

UNCORRELATED ORBITAL TRACK PROCESSING

SD2 Final Design Document

Kyle Galang, Aurela Broqi, Ruben Dennis, Aaron Nogues, Ezra Stone

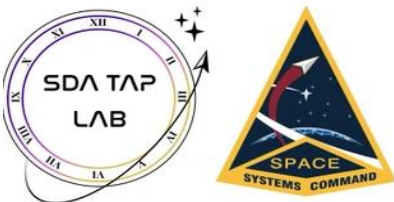
COP4935: Computer Science Senior Design II

Dr. Richard Leinecker

University of Central Florida

Orlando, Florida

April 03, 2026



AFRL

Table of Contents

1. Supplementary Material	5
1.1. Definitions	6-7
1.2. Document Breakdown	8
1.3. Researchers Personal Statements	9
1.3.1. Kyle Galang	10
1.3.2. Aurela Broqi	11
1.3.3. Ruben Dennis	12
1.3.4. Aaron Nogue	13
1.3.5. Ezra Stone	14
2. Project Concept of Operations	15
2.1. Executive Summary	16
2.2. Problem Statement	17
2.3. Design Plan	18
2.4. Minimum Viable Product	19
2.5. Agile Development	20
3. Uncorrelated Track Processor	21
3.1 Introduction	21
3.1.1. UCTP workflow and code	21
3.2 Dataset Generation Workflow	22
3.2.1. Data Shaping	23
3.3 Evaluation Workflow	24
3.3.1. Orbit Association Metrics	24
3.3.2. State Metrics	25
3.3.3. Residual Metrics	26
3.3.4. Binary Classification Metrics	27
3.3.5. Processor and Outputs	28
3.3.6. Conclusion	28
3.4 Definitions of Dataset	29-34
3.5 Unified Data Library	35-45

3.6	Conclusion	46
4.	OpenEvolve	47
4.1.	Introduction	47
4.2.	Background: Evolutionary Code Optimization	48
4.3.	OpenEvolve Framework Overview	49
4.3.1	Configuration and Extensibility	50
4.4.	OpenEvolve Basic Functionality & Process	51-53
4.5.	UCTP Integration to OpenEvolve	54-55
4.5.1	Early UCTP Integration Efforts	56
4.6.	Conclusion	57-58
5.	Symbolic Regression	59
5.1.	Introduction	60-61
5.2.	Foundation	62
5.3.	Expected Result	63
5.4.	Hypothesis Generation	64
5.5.	Sampling and Validation Process	65
5.6.	Optimization and Scoring	66
5.7.	Data Generation	67-68
5.8.	Testing and Evaluation	69
5.9	Nonlinear Oscillator Equations	70
5.10	Noise Robustness Analysis	71-75
5.11	LLM-Based Optimization Baselines	76
5.12	Summary	77
6.	Graphical User-Interface Development (GUI)	78
6.1.	Introduction	78
6.2.	OpenEvolve Architecture Versus Traditional Stacks	79
6.3.	Analytics Console Integration	80-83
6.4.	Analytic Console Development	84
6.5.	Future Development	85
6.6.	OpenOnDemand	86

7. Large Language Model Development (LLM)	87
7.1. Introduction	88
7.2. Purpose of the Large Language Model	89
7.3. System Architecture	90-91
7.4. Model Training and Data	90
7.5. Evaluation and Benchmarks	90-91
7.6. Website Integration	91
7.7. User Experience and Features	92
7.7. Security, Privacy, and Ethics	93
7.8. Limitations and Future Work	94
8. Overview of Project Completion	95
8.1. Overview	95
8.2. Documentation Finalization Overview	95-98
8.3. Research Finalization Overview	99-101
8.4. Symbolic Regression Research Finalization Overview	102-108
8.5. Analytics Console Finalization Overview	109-114
8.6. mini-Large Language Model Finalization Overview	114-117
9. References for Diagrams and Past Work	118
9.1. Introduction	118
9.2. UCTP & SDA TAP Lab	118
9.3. MITRE	118
9.4. Virginia Tech Applied Research Corporation	118
9.5. OpenEvolve	118-119
9.6. LLMSR-Bench	119
9.7. AlgoTune	119

Chapter 1. Supplementary Material

1.1 Definition of Terms

Throughout the document abbreviations of collaboration between different laboratories and institutions will be made. Alongside terminology, within the current scientific domain of research in Computer Science and Aerospace Engineering will be presented in abbreviated form in order to prevent redundancy. In order to ensure that all readers are able to interpret these terms in the same manner as intended, the following definitions are offered.

Uncorrelated Track Processor (UCTP):

Uncorrelated Track Processor underpins timely and accurate orbit estimation. Which is a critical step in detecting, correlating, and responding to potential threats in Space Domain Awareness.

OpenEvolve:

OpenEvolve is an open-source implementation of the AlphaEvolve system created by Google DeepMind. It introduces a powerful way to evolve code using Large Language Models through iterative optimization. It runs an evolutionary loop using island-based convergences which generates multiple candidates and selects best performers.

High-Performance Computing Cluster (HPC):

High Performance Computing cluster represents what we will be storing all of our datasets and computing into. In regard to more specific usage of HPC in this project. We are in collaboration with MITRE using their Federal AI Sandbox that is powered by NVIDIA DGX SuperPOD.

Space Domain Awareness Tools, Application, and Processing Lab (SDA TAP Lab):

Space Domain Awareness Tools, Applications, and Processing Lab. The main mission is to accelerate the delivery of space battle management software to operational units. They decompose kill chains, prioritize needs with operators, map needs to technologies, and onboard tech to existing platforms quickly. They partner with industry, academia, and across the government to succeed.

Air Force Research Laboratory (AFRL):

The Air Force Research Laboratory is the primary scientific research and development center for the Department of the Air Force. AFRL plays an integral role in leading the discovery, development, and integration of affordable warfighting technologies for our air, space, and cyberspace force.

Defense Advanced Research Projects Agency (DARPA):

The Defense Advanced Research Projects Agency (DARPA) is an independent research and development agency within the U.S. Department of War (DoW). The DARPA mission is to create and prevent technological surprises for our national security.

Two-Line Element (TLE):

Two-line element set, also referred to as elset in benchmarking outputs. This is a data format for encoding a list of orbital elements of an orbiting object around the Earth. TLEs represent trajectories only of Earth-orbiting objects.

Graphical User Interface (GUI):

Graphical User Interface will be mentioned consistently across the document as it is a part of the minimum viable product being developed by the students. It is also referred to as the Analytic Console which represents a real time dashboard for OpenEvolve and UCTP satellite data.

Unified Data Library (UDL):

The Unified Data Library is a cloud-based data repository that hosts a wide range of data, including data for space domain awareness. It contains more than 300 different data types in addition to commercially acquired data, enabling the Space Force government and allied partners access to a multitude of data sources from a centralized repository. This is a source of state vector and observation data used in UCT evaluation product.

High Multi-Dimensional Histograms (HMDH):

Hierarchical Multi-Dimensional Histograms are a software library used for data summarization. In this research domain we are primarily working with six-dimensional datasets that consist of positions and trajectories.

TrackTLE:

TLE generated by performing initial orbit determination on a single track of observations. These are used in the benchmark datasets to emulate UCT TLEs.

Common Epoch:

The epoch in which the Candidate Orbit state vector or TLE is generated.

Reference Object:

Satellite with known NORAD ID, state vector, TLE, and list of observations publicly available on the UDL that are made a part of a benchmark dataset.

Reference Orbit:

State Vector, TLE, a list of observations, and a list of trackTLEs generated from the list of observations that are associated with a Reference Object.

Candidate Orbit:

Output generated by the UCTP, defined by a Candidate State Vector and epoch or Candidate TLE, along with a list of observation IDs or trackTLE IDs the UCTP has correlated to that state vector or TLE.

Propagated Reference:

State vector or TLE of Reference Orbit propagated to Common Epoch.

Decorrelate:

The process in which the state vector and associated observations or TLE and track trackTLEs of a reference orbit are separated and stripped of their IDs. This is what makes observations uncorrelated, which is then fed into UCTP.

Downsample:

The process in which some observations associated with a reference orbit are strategically removed to simulate certain observation scenarios for true negative, limited observation window, etc.

Orbit Association:

Before evaluation of UCTP, each candidate's orbit must be associated with a reference orbit to generate a benchmark dataset.

Non-associated Candidates:

Candidate Orbits that cannot be associated with a Reference Orbit during the Orbit Association procedure.

Non-associated References:

State vector or TLE of Reference Orbit propagated to Common Epoch.

1.2 Document Breakdown

This section explains the structure and purpose of this document. Its primary goal is to present background information on the current problem, describe our proposed solution, and identify both sponsor and senior design requirements along with how we plan to satisfy them. How this document is structured is as the reader goes through the chapters, we build upon the understanding of the problem and its complexities and then showcasing how we are trying to solve them. This allows the reader to digest the document in a more natural way since there are many different research topics and multidisciplinary problems associated with solving the problems. By the end of this document the reader should be able to piece together how the research team is trying to solve the problem and alongside allow them enough background knowledge to follow along the intricate topics being presented throughout the document. Each chapter is interlinked with one another, where we slowly build the understanding of what UCTP is and its limitations and how we are trying to use OpenEvolve to solve them. This document and project is very research heavy but we aim to be able to break down everything for the reader to be able to follow along.

The architecture of how to solve this problem and meet requirements goes as follows:

- Uncorrelated Track Processing (Already created and need to extract JSON data from satellites)
- OpenEvolve AI architecture to help benchmark that output data to see how well UCT performs

- Symbolic Regression (OpenEvolve’s prime example on how they benchmark with already known solutions of 239 scientific domain problems)
- Graphical User Interface (Analytics Console that allows real time updates on OpenEvolve and UCTP satellite data)
- LLM (Trained from the example problems in order to help write out the initial and evaluator functions needed in the OpenEvolve architecture)

These five topics are divided among the team members, each providing a deeper explanation of the research and development being conducted to satisfy both the senior design software requirements and the sponsor’s expectations for research documentation and the final poster.

1.3 Personal Statements

This section serves to explain each student’s motivations and ideas about the project. As an undergraduate group working together within two semesters. This is to ensure that each group member gains something from this project.

1.3.1 Kyle Galang:

My motivation to work on this project is to gain experience with AI. I have worked with the aerospace industry for 3 years now. I feel that I have worked with optimizers and automation but never got the chance to truly work on AI as a computer scientist and software engineer. Hearing the project details excites me to be able to work with tech stacks that I haven’t had any experience with. Technology such as OpenEvolve which is an open-source version of Google DeepMind’s AlphaEvolve which is a Gemini-powered coding agent for designing advanced algorithms, working with MITRE supercomputers with Nvidia superpods, and working on an already existing framework for uncorrelated tracking.

Some of my concerns are that it is very heavy research which makes the workload very autonomous for my group. I’ve had experience contributing to open-source frameworks like this one called OpenMDAO which is a multidisciplinary optimizer for aircraft conceptual design. And my concern

there is that it took me a bit to really learn the existing code and how to contribute to the framework before pushing any code in during my time of the internship.

I've always liked the research roles in big industry with the flexibility and environment they uphold. So, my core motivation really relies on getting to know these new technologies and trying to rebuild my career in computer science to hopefully be able to land a role as a software engineer dealing with machine learning or AI as I move away from my experience in aerospace.

My idea for the project is that we will be learning how to use OpenEvolve on our local computers with small problem sets to see if we can create our own algorithms. As we get more familiar with it and wait on the partnering team to create our evaluation script. We also need to get access to high performance computing clusters to test them on a larger scale. From there we use the evaluation script to test if the AI generated algorithms perform better or worse than the previous mathematical model that was created in the black box. Of course, the first couple of months are going to be slow since there are a lot of research papers and going through the documentation on open-source technology. But I think the more we put into it, the easier it will get to implement it in the future.

Some ideas on how I think the work distribution will be due to the fact that we're all going for the same goal. We will most likely be doing the same thing. Which is hard in regard to the class structure of using Agile methodologies and splitting work up as a developer from front-end, database, back-end, etc. In this case we will all be researchers trying to come up with the solution in our own ways. Of course, we will all have the same environment for using OpenEvolve in our local machines which we will later be transferring into an HPC server. I think the best way to keep everyone at bay is setting up a GitHub with our own directories, where we can track our codes that we push. While also being able to keep everything in one place so when we do transfer our files from local machines, we can then just zip it up and transfer it over to the HPC when doing larger scale testing.

I think also a key component to this is weekly meetings. I've set 2 in order to keep the group closer so we can help each other out. Sometimes in research it feels like you're not moving or not making any progress. But being able to talk to each other more frequently allows us to hold each other accountable while also assisting others with research and methodologies on how they are solving the problems. The other key component is documentation as well for reproducibility.

One thing I also think is important is having what some may call scrum master or project lead. To be able to ensure everyone is following the contract and keeping things organized. That's one thing I've noticed in research-based environments is that you can easily stray away from things because of how flexible research is. But if we are a reinforcing person to delegate tasking and research, it will give better goals and sprint meetings to aim for which will move along the research project.

1.3.2 Aurela Broqi:

My motivation for successfully completing the Uncorrelated Tracking Processing Algorithms project, sponsored by the Air Force Research Lab, is to deepen my knowledge and skills in the field of Machine Learning and AI. My background in software development only began over the summer, and I want to continue building my foundation by learning from those with more experience. I am also motivated to grow as a team member by contributing to an effective workflow environment while making impact and progress throughout the project. I also see this project as an opportunity to expand my own skill set and prepare myself for future professional roles.

My concern is that much of the work will focus on understanding how an existing model functions and integrating it into a pre-built algorithm, rather than giving us opportunities to build new algorithms or develop software from scratch. Since one of my goals is to improve my software engineering skills, I am concerned this project may not provide as much direct coding experience as I hoped. I plan to continue communicating actively with my teammates, staying prepared for each meeting, and engaging directly with the project leads to secure meaningful work throughout these eight months.

I am particularly interested in the technical aspects of this project. I hope to gain experience with the underlying models and to work on algorithm design, not just analyzing how existing models' function. Ideally, I would like to extend or improve the algorithm already developed in the previous project and possibly learn a new programming language. I am motivated to provide proof that our concept is worth further investment, especially since there may be an opportunity to work on the NVIDIA SuperPod. I did not realize that this project could require resources like that, which makes me want to take it seriously and gain as much learning as possible. More than anything, I hope to come out of this project with stronger engineering and problem-solving skills.

1.3.3 Ruben Dennis:

The Uncorrelated Orbital Track Processing doesn't sound daunting until you start looking under the hood. We're going to be working with a newer LLM through Open Evolve, which will provide us with skills for working with future models in the industry. On top of that, we're dealing with the concept of evolutionary coding and how to iteratively improve algorithms. This project will provide some more insight into topics like benchmarking and performance tuning.

As for my motivation, I have always had interest in pretty much anything space-related and have kept tabs on projects such as the James Webb telescope. So, the opportunity to work on the Uncorrelated Orbital Track Process (UCTP) with relevant data about the objects orbiting us was super intriguing. On top of that, we will be conducting research on a newer LLM (Open Evolve), which I'm sure will be very difficult, but we also have the chance to become deeply knowledgeable about how this specific LLM runs. I have also never done proper research project in regard to computer science, so this will provide insight into how many companies operate when conducting their own research. From my perspective the most exciting part of this entire project is exploring evolutionary coding and seeing if Open Evolve can make iterative improvements. I want to look more into how benchmarking and profiling can be integrated.

This project has the capability to pave the way for even more research into things like AI-assisted collision avoidance, orbital mechanics, and tracking already known objects like

satellites. Since this is the Air Force, this research could also lead to further development of guidance systems for aircraft and missiles. In terms of my career in cybersecurity, this will provide skills in finding and fixing vulnerabilities. By working with Open Evolve to automate bug fixing and test algorithm improvements, it will allow me to gain experience in secure coding practices and automated vulnerability repair. The same skills that help verify whether an evolved algorithm is correct and efficient can also be applied to auditing software for flaws, building more resilient systems, and ensuring that code can withstand unexpected inputs or adversarial conditions.

In addition, cybersecurity often requires a “trust but verify” mindset when dealing with new tools and technologies. This project will improve that mindset as I evaluate AI-generated code for correctness, robustness, and potential risks. Designing fitness functions, implementing fuzz tests, and considering edge cases will all mirror the kinds of systematic verification methods used in security testing. Beyond the technical overlap, this experience will also help me practice adapting to rapidly evolving technologies, something that is critical in a field where threats and defense strategies change constantly.

Some ideas for the project are to be an automated repair assistant so that we can feed intentionally buggy algorithms to see if the LLM can provide insight into what we are trying to do and make the correct changes but then add some comments to where things could be improved. A self-testing code where we prompt Open Evolve to generate unit tests to catch failures. Also seeing if it can provide a fundamentally different approach to the algorithms. All these would aid in providing the best performance for UCTP. Overall, I am super excited to work on this project and the technologies that come with it. It concerns a field that is intriguing to me with space and has difficult problems to solve.

1.3.4 Aaron Nogues:

My interest in the Uncorrelated Orbital Track Processing project stems from a long-standing fascination with applying complex AI concepts to solve challenging, real-world problems. My coursework in both Computer Vision and Artificial Intelligence was theoretically fascinating, but I always felt that the real-world application was what I was missing. This project, with its focus on researching and applying OpenEvolve, is the perfect opportunity to

bridge that gap. When the team decided to tackle this, what really stood out to me was the potential to create a genuinely more efficient algorithm for a critical and complex domain.

From a technical standpoint, I am particularly motivated by the challenge of adapting the OpenEvolve framework in general in addition to our specific use case. The opportunity to work with an open-source version of a sophisticated system like Alpha Evolve is incredibly exciting. It's a chance to move beyond the textbook examples from my classes and into a hands-on, practical application of evolutionary algorithms. I believe that successfully leveraging this framework will not only be a valuable contribution to the team's goals but will also be a significant step in my own professional development as I pursue a career in applied AI.

From a technical perspective, most of the effort would be just in reviewing OpenEvolve and actively implementing examples and other problems. This effort contributes to the main goal of determining the value of Open Evolve and by extension Alpha Evolve as it relates to the Airforce's research department. Perhaps a visual focus on improvements created by the Open Evolve in action, comparing iterations of the algorithms that are returned.

1.3.5 Ezra Stone:

The Uncorrelated Orbital Track Processing (UCTP) project presents an exciting opportunity to build upon my little experience with LLM technologies. My journey so far through Computer Science has had a large emphasis on data structures and algorithms. Instilling in me the critical need for efficient and reliable algorithms to process large scale data. The work on benchmarking UCTP algorithms as outlined in the original project plan is a testament to this. The ability to accurately classify and characterize space events is paramount for both civilian and military applications and I am driven by the idea of developing a solution that can improve these existing methods.

The core motivation for me lies in the idea of building a system that can improve itself. From my point of view the current methods being used are inefficient and can be greatly improved if done correctly. The concept of an agent which can autonomously generate, test, and tweak algorithms represents to me the possibility of a profound shift. I am excited by the challenge of applying this groundbreaking AI system to solve a problem as complex and crucial

as UCTP. It possesses an opportunity to move beyond the manual optimization of individual algorithms and explore an automated approach to discovery.

My interest in AI is a direct result of my prior projects and research that I've done in course work and my own free time. Through my own research I have seen the exponential growth of data in every aspect of technology. We've all probably witnessed how traditional methods struggle to keep pace with the sheer volume and velocity of information. This has made it very clear that the future of data analysis lies in more intelligent, autonomous systems. This is especially true in domains such as government, military, and space operations.

From my personal point of view, the opportunity to learn and contribute to OpenEvolve is particularly exciting because of how new it is. The openEvolve framework, being a key component of this project, represents a new frontier in AI assisted code development. It's not just about using pre-trained models to generate code; it's about building a system that learns to "evolve" and improve its own solutions. This is a level of agentic intelligence that will move beyond automation and into the realm of co-creation. OpenEvolve's modular design with distinct components for prompt sampling, LLM ensembles, and evaluation provide a flexible and powerful foundation for experimenting.

I am particularly interested in the evaluation process and how it can be used to provide a quantifiable fitness score for the evolving algorithms. The ability to objectively measure performance is what makes an evolutionary approach viable, and I am excited to develop this component for our specific UCTP task. In addition to this the chance to work with the Gemini API and integrate it into a real-world project is an invaluable learning and project experience. I am confident that my skills and enthusiasm for this kind of work will allow me to make a significant contribution to this project's success.

Beyond the technical aspects of the project, I am also deeply motivated by the opportunity to work with the Air Force Research Laboratory (AFRL). The chance to contribute with the SDA TAP Lab and its mission to enhance space situational awareness is incredibly exciting. I am eager to collaborate with the team at the SDA TAP Lab and to gain a deeper understanding of the unique requirements and operational constraints of the space domain.

Chapter 2. Project Concept of Operations

2.1 Executive Summary:

This senior design project is conducted in collaboration with the Air Force Research Laboratory (AFRL), the Space Domain Awareness Tools, Applications, and Processing Lab (SDA TAP Lab), MITRE, and a parallel student team. The effort centers on evaluating OpenEvolve, an open-source adaptation of Google DeepMind’s AlphaEvolve agent, as a benchmarking framework for Uncorrelated Track Processing (UCTP). OpenEvolve combines large language models (LLMs) with evolutionary search using island-based convergences to automatically discover algorithms and optimize code, offering a promising approach for systematically exploring solutions in complex scientific domains.

Current UCTP pipelines can produce observations and tracks that do not correlate to any known objects, but there is no widely accepted “ground truth” solution set against which candidate algorithms can be evaluated with. This lack of benchmarked problems and metrics makes it difficult to measure progress, compare approaches, or justify real-time dynamic analysis. Our project addresses this gap by researching OpenEvolve’s architecture and operations, starting from its symbolic regression benchmark suite, which spans 239 scientific domain problems with known solutions. These problems provide a controlled environment in which OpenEvolve’s performance can be studied and its evaluation methodology adapted to the UCTP context.

The project is organized into two tightly coupled tracks. The research and development track focuses on understanding and extending OpenEvolve for UCTP-like tasks. We will design smaller, UCTP-relevant problem sets and initially run them using Google AI Studio APIs to query LLMs such as Gemini. If OpenEvolve’s AI architecture proves suitable for benchmarking UCTP algorithms at this scale, we will transition to larger datasets more representative of real UCTP outputs and deploy experiments on MITRE’s high-performance computing (HPC) clusters to assess scalability and robustness.

The software design and development track targets senior design requirements by creating a graphical dashboard and LLM-assisted tooling around OpenEvolve. The dashboard will connect locally to the OpenEvolve repository, visualize real-time iteration and efficiency scores, and provide an interface for monitoring algorithmic progress. In parallel, we will develop an LLM-driven feature that generates two

key Python artifacts required by OpenEvolve: initial program files that encode candidate algorithms or equations, and evaluator files that define scoring metrics per iteration. Together, these capabilities enable sponsors to extend or replicate our work and also help train the LLM to be more accurate overtime.

By the end of the project, the team will deliver four minimum viable products: (1) a research report assessing the viability of OpenEvolve as a benchmarking framework for UCTP, (2) a poster summarizing findings and visualizations, (3) a real-time OpenEvolve monitoring GUI, and (4) an LLM-based code generation pipeline for OpenEvolve program and evaluator files. These outcomes are intended to satisfy sponsor objectives while meeting senior design expectations in both research and software engineering practice.

2.2 Problem Statement

Uncorrelated Track Processing (UCTP) is critical for Space Domain Awareness, enabling the detection and tracking of objects that do not match any known cataloged body. Although existing UCTP pipelines can generate such uncorrelated observations and tracks, there is no standardized set of benchmark problems with known solutions or agreed-upon performance metrics. Without this, researchers and operators cannot objectively evaluate algorithm quality, compare competing approaches, or quantify improvements over time.

OpenEvolve, an open-source evolutionary AI framework built around large language models, offers a potential solution by providing a structured way to generate, test, and score algorithms on benchmark tasks with known ground truth. The real objective of SDA TAP Lab is to have a UCTP do real time analysis while also making it dynamic to where the algorithms are improving over time. With such a complex nature of data coming in from satellites, the algorithms need to evolve over time.

Therefore, the problem this project addresses is twofold:

1. **Scientific Problem:** Determine whether OpenEvolve's AI architecture and evaluation methodology can be adapted to create reliable, UCTP-relevant benchmark problem sets and performance metrics that accurately reflect UCTP challenges.

2. Engineering Problem: Design and implement a usable software interface and supporting LLM-based tools that integrate with OpenEvolve, enabling stakeholders to run experiments, visualize performance, and automatically generate the program and evaluator files needed to benchmark UCTP algorithms at scale.

Solving this problem will provide AFRL, SDA TAP Lab, MITRE, and the broader research community with a principled framework and practical tools for evaluating UCTP solutions. All of the developments will be publicly available for everyone in the research community. The only proprietary code that cannot be shared is the UCT processor created by SDA TAP Lab. This is very beneficial to the group doing research and development since we are able to showcase our work to the public for AI development in the UCTP domain.

2.3 Design Plan

Currently our team is split between two parts: first half being mainly to fulfill sponsor's requirements for research documentation and posters. This side of the team is pure research and development of applying UCTP into OpenEvolve's AI architecture, while the second half is to fulfill senior design requirements for software design and development. This side of the team focuses on researching OpenEvolve's architecture and how to contribute to it with a GUI and LLM chat bot. Both teams are coherently working together where the research team is doing pure research on background knowledge on already developed code for UCTP evaluation metrics and the development team doing research on OpenEvolve architecture. Each side meets twice a week where they collaboratively share and transfer knowledge to both sides in order to solve the main problem of assessing the viability of OpenEvolve to UCTP's limitation for benchmarking.

2.4 Minimum Viable Product

As stated, before in the executive summary and problem statement our group is expecting to meet four minimum viable products stated in the following:

- I. A research report assessing the viability of OpenEvolve as a benchmarking framework for UCTP
- II. a poster summarizing findings and visualizations
- III. A real-time OpenEvolve monitoring GUI
- IV. An LLM-based code generation pipeline for OpenEvolve program and evaluator files.

These minimum viable products have been accepted on both sides of the student team and sponsors.

2.5 Agile Development

Our sprints are depicted in JIRA where we have sprints spanning from the first 2 weeks of Senior Design I to the end of March in Senior Design II presentations. Although future blockers cannot be foreseen, our team plans on keeping on track based off of our JIRA sprints. The main blockers we foresee in the future are sponsor blockers. Since we are working with a government agency, we expect blockers such as access to databases, government shutdowns, communication, and multidisciplinary collaboration. At the moment, our team is still waiting to get access to the HPC from MITRE's side so we can work with the AI sandbox for larger data scales. But currently we are working with smaller datasets for UCTP and OpenEvolve. As of progress within the team towards the ending of senior design I, we are currently ahead of schedule on development as shown in the figures 2.51, 2.52, and 2.53. Our sponsors also have access to our JIRA which tells them what we are working towards each sprint alongside our weekly meetings on what we are doing specifically.

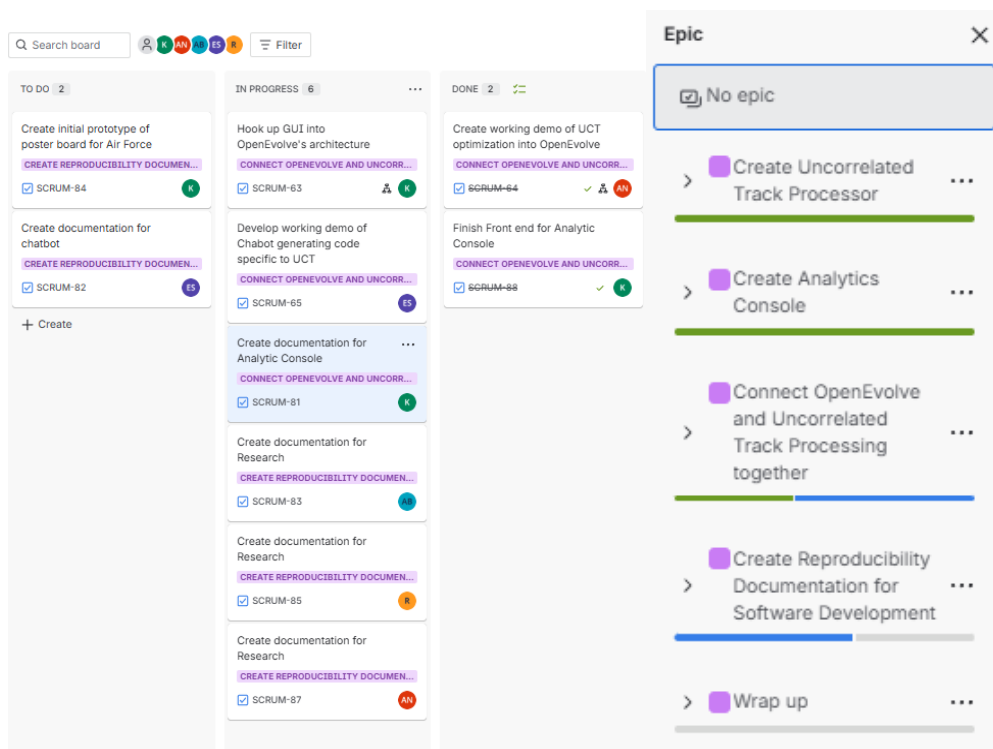


Figure 2.5.1: This figure show the Jira Board for current sprint and Epics that we have completed/in progress.

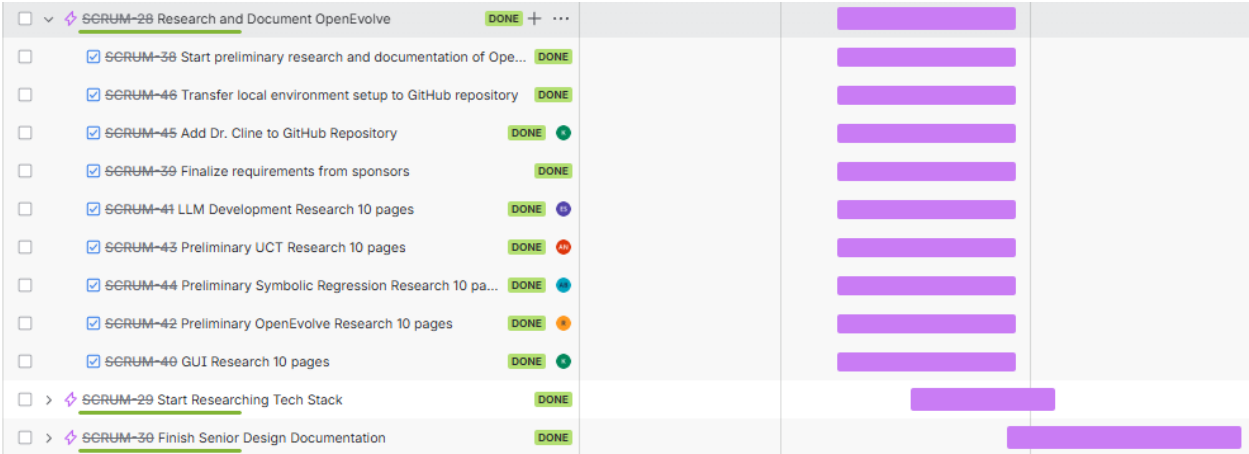


Figure 2.5.2: This figure shows an in-depth look into the tasks filled inside each sprint.

Project Maturation

Category	Item	Status	Notes
Environment Setup	GitHub Repository	Mature	
Environment Setup	SDA TAP Lab & UDA Access	Mature	
Environment Setup	Open OnDemand	Identified Later	Through HPC access
Environment Setup	Documentation	Ongoing	Updated as symbolic regression benchmarking continues
Design Requirements	Design Schematics	Mature	
Design Requirements	Tech Stack	Mature	
Development	Graphical User Interface	Mature	
Development	LLM Development	Development Phase	
Development	Validity Environment Testing	Identified Later	Pending Open OnDemand setup

Legend: Mature Ongoing Development Phase Identified Later

Figure 2.53: This figure shows the critical tasks presented each week during sponsor meetings with the team. (Green: mature, yellow: in progress, red: blockers) Current tasks reached for the team are Analytic Console (GUI) designs are finished, LLM development is already developed but has bugs that need to be fixed, and Open OnDemand is a web feature that allows us to SSH into the HPC but has not been fully set up yet so validity testing for GUI and LLM development have not been set up yet.

Chapter 3. Uncorrelated Track Processor

3.1 Introduction

In a brief introduction UCT processing underpins timely and accurate orbit estimation. Which is a critical step in detecting, correlating, and responding to potential threats in Space Domain Awareness. Adversaries exploit CCDDM to degrade sensor data quality, leaving traditional algorithms unable to reconcile sparse and/or noisy observations into reliable orbital estimates.

But in a simpler and more in-depth explanation, uncorrelated Track Processor (UCTP) is a part of a Space Domain Awareness (SDA) system that tries to make sense of uncorrelated sensor observations that do not match any known tracks from known databases. In practice, sensors (radar, optical, etc.) produce tracks of objects moving through space. Most tracks can be correlated to existing orbits in the catalog. But when a track cannot be matched it becomes uncorrelated. A UCT processor:

- Takes these uncorrelated tracks and **rapidly estimates a plausible orbit** from incomplete or noisy data. (i.e., downsampling or simulating)
- Decides whether the track likely represents a **new object**, a **maneuvering object**, a **mis-association/cross-tag**, or just **bad data**.
- Produces candidate orbits and scores/metrics that downstream systems can use for **threat detection, catalog updates, and tasking sensors**.

Since the data is often sparse, noisy, or deceptive (i.e., CCDDM: Camouflage, Concealment, Deception, and Maneuver), UCT processing is considered a very difficult problem to solve. Since it isn't just a single algorithm to solve everything but rather a diverse number of approaches to solving such a problem.

3.1.1 UCTP Workflow and Code

Currently we are working with the SDA TAP Lab's UCTP benchmarking code, which has already been developed by previous groups and professionals over the years. The framework can be broken down into two pipelines, dataset generation and evaluation.

3.2 Dataset Generation

The basic dataset generation workflow allows the user to choose the desired dataset characteristics, such as object type, orbital regime, and data quality. Which is then downsampled or simulated observations again matching data. This converts them to trackTLEs, which then the result is stored into a database that is fed into UCTP.

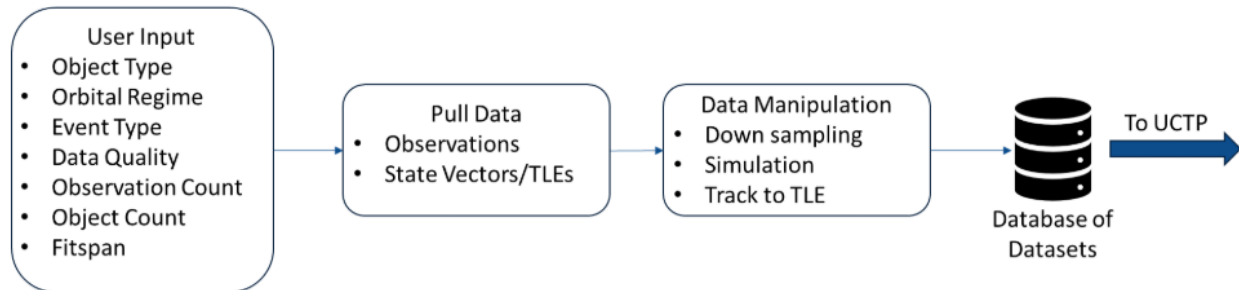


Figure 3.2.1: This figure showcases the dataset generation workflow as explained previously in chapter 3.2.

3.2.1. Data Shaping

When creating the database from the results, there are many ways to shape the data:

- **Window Selection:** Allows us to select a slice of time or which part of the orbit or viewing geometry to keep. This allows us to create a tier system that allows us to score the current time window of the data. The first step in building a dataset is choosing a good time window. A Tier-1 window is ideal: all user criteria are met using real data only, with no downsampling or simulation. A Tier-2 window meets the criteria but requires downsampling (removing some observations), with no simulation. A Tier-3 window needs simulated observations for existing objects in the dataset to reach the desired orbital coverage and/or number of observations. A Tier-4 window needs simulated observations for new reference objects to reach the desired number of target objects. In Tier-3 and Tier-4, downsampling may or may not be used, but it does not change the tier. Tier-5 means the selected criteria cannot be satisfied at all.

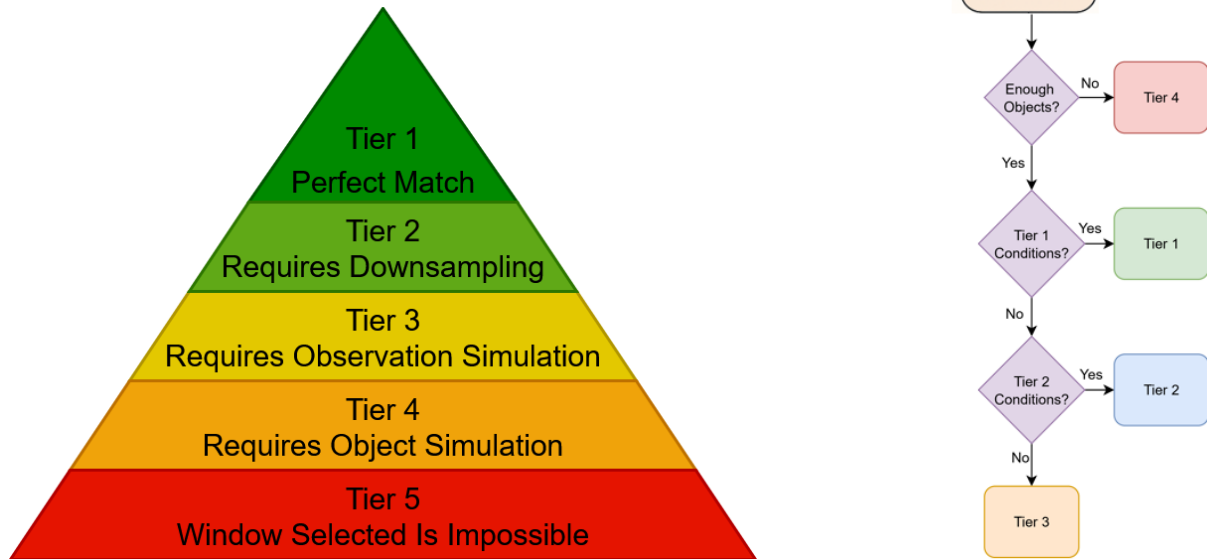


Figure 3.2.11: This figure showcases the tier system on how we score the data into the dataset.

- **Downsampling:** Which allows us to simulate poor data quality from good measurements by removing observations. This is done through three metrics: Orbital Coverage, Maximum Track Gap, and Total Observation Count.

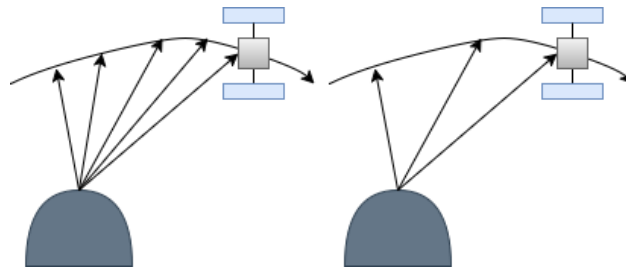


Figure 3.2.12: This figure showcases how we are downsampling data by removing observations.

- **Simulation:** This applies Gaussian noise to both the propagated positions and inverted right ascension and declination values. This determines simulated azimuth and elevation values from the simulated angular values and known site locations of latitude, longitude, and altitude.

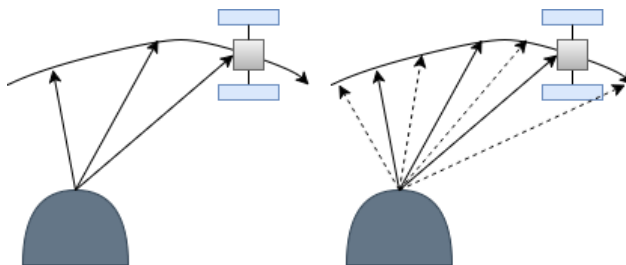


Figure 3.2.13: This figure showcases how we are simulating data by taking the state vector of the reference orbit and propagate to the observation epoch.

Datasets are generated according to desired characteristics input by the end user through a 16-character string:

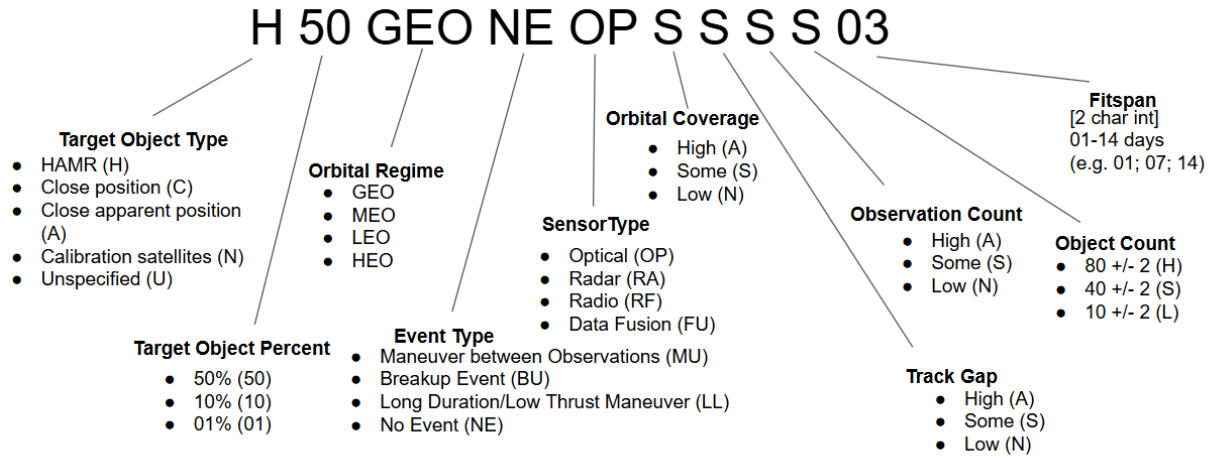


Figure 3.2.14: This figure showcases the dataset code which is a unique 16-character string encoding from the user selected data parameters.

3.3 Evaluation Workflow

The evaluation pipeline is how the data is set up in the output file through the evaluation metrics of orbit association, state metric, residual metric, and binary metric. This is then comprised into one file consisting of all these metrics alongside the current satellite id and all of its observations.

3.3.1 Orbit Association Metric

The orbit association metric is the preprocessed data to allow the other metrics to be created. This is only edited once more sensor types are integrated. The evaluation framework will associate Candidate Orbits with Reference Orbits by solving an optimal assignment problem using a modified Jonker-Volgenant algorithm with no initialization. The assignment will also be using a linear sum assignment function from the Python library `scipy.optimize`.

The assignment cost matrix is constructed following:

- For every permutation of Reference Orbit/Candidate Orbit pair, the Reference Orbit is propagated to the Common Epoch using Orekit (see Propagation).

- If the Reference Orbits and Candidate Orbits are given in TLEs, they are both converted to 6D state vectors (see TLE to State Vector)
- The 6D vector difference between the Propagated Reference and the Candidate Orbit is taken.
- The L2 norm of that difference vector is assigned as the cost.

Then propagation occurs through state vector propagation, covariance propagation, and TLE propagation. This step is to confirm/validate/verify.

3.3.2 State Metrics

The state metrics refers to comparing the Candidate orbit to the Reference orbit either as state vectors or TLEs to determine how accurate the orbit estimation is.

For state metrics for state vectors, the following metrics are computed for each pair of the Reference orbit/Candidate orbit:

- **L2 error:** Length of the difference vector between the reference and candidate states, computed for position only, velocity only, and the full 6-D state.
- **Bias:** Average offset of the candidate from the reference in each position and velocity component.
- **Mahalanobis Distance (MD):** Measures of how far the candidate state is from the reference state, taking both covariances into account:

$$MD = (x_{\text{ref}} - x_{\text{cand}})^T \Sigma^{-1} (x_{\text{ref}} - x_{\text{cand}})$$

$$\text{where } \Sigma = \Sigma_{\text{ref}} + \Sigma_{\text{cand}}.$$

- **MD p-score:** Computed from a chi-squared distribution with 6 degrees of freedom as $1 - P(\chi_6^2 \leq MD)$.

This is the probability (0.0–1.0) that the reference and candidate are the **same orbit**, given their covariances. Higher values mean higher confidence.

- **NEES (Normalized Estimation Error Squared):** Similar to MD but uses only the candidate covariance:

$$NEES = (x_{\text{ref}} - x_{\text{cand}})^T \Sigma_{\text{cand}}^{-1} (x_{\text{ref}} - x_{\text{cand}}).$$

- **NEES p-score:** Also, from a chi-squared distribution with 6 degrees of freedom:

$$1 - P(\chi_6^2 \leq NEES).$$

This measures how **well-calibrated** UCTP's covariance is.

- p-score < 0.5 → processor is **overconfident** (covariance too small).
- p-score > 0.5 → processor is **underconfident** (covariance too large).

For state metrics for TLEs, for a given pair of TLE/Candidate. TLE is propagated to the common epoch. Both TLEs are converted to state vectors to generate L2 Norm and bias metrics from state metrics for state vectors. The other metrics are not computed due to TLEs lacking covariance and uncertainty.

3.3.3 Residual Metrics

Residual metrics measure how far the predicted observations are from the actual ones, on the unit sphere (i.e., angular error on the sky). We propagate the Candidate Orbit to each observation time and compare:

- **Reference vs. Candidate:** Compare observations (or trackTLEs) from the Reference Orbit to those predicted by the Candidate Orbit → measures accuracy relative to truth.
- **Candidate vs. Candidate:** Compare Candidate Orbit's own observations (or trackTLEs) to its predicted ones → measures precision and internal consistency.

For each case, we compute the root mean squared (RMS) error between the actual and predicted observations or trackTLEs.

3.3.4 Binary Classification Metrics

The binary classifier labels each observation or trackTLE in the benchmark dataset as one of four types:

- **True Positive (TP):** The observation belongs to both the Candidate Orbit and the Reference Orbit it is associated with.
- **True Negative (TN):** The observation does not belong to any Candidate Orbit or Reference Orbit.
- **False Positive (FP):** The observation is assigned to a Candidate Orbit, but it does not actually belong to the associated Reference Orbit.

- **False Negative (FN):** The observation is not assigned to a Candidate Orbit, but it does belong to the associated Reference Orbit.

From these counts, we compute standard binary classification metrics:

- **Accuracy:**

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

- **Recall / Sensitivity:**

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **Balanced Accuracy:**

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right)$$

- **Cohen's Kappa:**

$$\kappa = \frac{2(TP \times TN - FN \times FP)}{(TP + FP)(FP + TN) + (TP + FN)(FN + TN)}$$

- **Matthews Correlation Coefficient (MCC):**

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

- **Specificity:**

$$\text{Specificity} = \frac{TN}{TN + FP}$$

These metrics together describe how well the UCTP identifies correct associations and avoids incorrect ones.

3.3.5 Processor and Outputs

All of these metrics that are explained earlier are the preprocessor portion of the framework. They allow us to create output JSON data files that we then feed into the UCTP to get out evaluation and benchmark.

3.3.6 Conclusion

So, what is the importance of these metrics and evaluation outputs? UCTP is trying to figure out the objects in a dataset. There are two main things that would be nice to optimize, which are the accuracy of the fitness. An example is state metrics: comparison between reference state vector where the object really is versus the Candidate state vector. Residual metrics where it references the root means squared error, which is the difference between true observation versus reference observation. This calculation is done through square root of the average of the sum of the residuals. Binary calculation metrics which are important to return and understand how these qualities affect the overall fitness. Or a 6x6 covariance uncertainty, where we calculate a distance which is scaled by the sum of the covariance matrices, normalized estimation error, and standard distance in 6-dimensional space. Or lastly, the speed of the convergence to a solution.

These are the many problems we aim to solve that UCTP has trouble with. So now we understand what UCTP is and what its limitations are. How do we go about solving it? As stated, before in this document the problem with UCTP is that pipelines have been around for quite a while now. But the problem is that UCT processing does not have a well-known solution to compare it to. This makes it hard to improve upon the algorithms for future use and also what Space Domain Awareness (SDA) aims to do with real-time monitoring for UCTP. We need benchmark datasets with known solutions and performance metrics to evaluate UCTP performance.

3.4 Definitions of Datasets

These given data sets will be given according to the desired characteristics input by the end user. The characteristics of the dataset will be encoded in a 16-character string shown below:

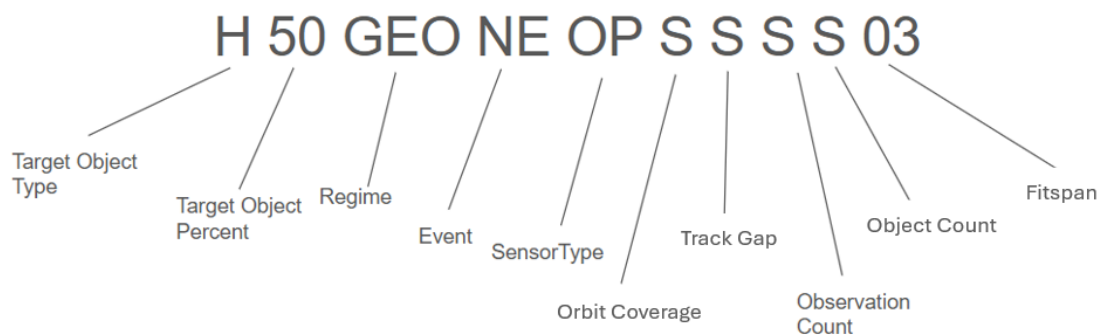


Figure 3.4.1: This figure showcases the 16-character string that is generated for the dataset being fed into the processor.

Target Object Types:

The target object types are a simple overall classification of the type of object to be given and/or later simulated. The relationship between the letter of classification and characteristics of this classification is defined by the graph:

High Area to Mass Ratio (HAMR)	Close Objects	Close Apparent Objects	Unspecified	Calibration
$> 1 \text{ m}^2/\text{kg}$	distance $< X \text{ km}$ velocity $< X \text{ m/s}$	Angles within: X°	Datasets unconstrained by object types	Well-tracked objects only
H	C	A	U	N

Target Object Percent:

The target object percentage is the percentage of the target object to be included in the given dataset. Any value from 100 to 01 or UN (unspecified) is valid. When UN is chosen it leaves the makeup of the data set to the choice of the algorithm, populating the data set with a natural distribution of objects. The reason it is directly correlated with the object type is because objects of type Unspecified must have UN as their target object percent.

Orbital Regime:

The orbital regime is the measurement of the semi-major axis a in kilometers. There are 4 regimes Low Earth Orbit (LEO), Middle Earth Orbit (MEO), Geosynchronous Orbit (GEO) and Highly Elliptical Orbit (HEO) and the orbital regime of the object can take up any number of combinations of these 4 regimes. These combinations are LMO, LGO, LHO, MGO, MHO, GH0, LMG, LGH, MGH, and ALL. Combinations of 2 are the first letters of the unique word from the 4 regimes (L, M, G, H) followed by an O to stand for “orbit” and combinations of 3 are the first letters paired together. All letters are sorted by

the order of their regime, L being first, H being last and so on. The range of a by its regime are defined by the chart:

Manuever Between OBS	Breakup	Long Duration/low thrust	No Events
MB	BU	LL	NE

Sensor Type

The type of sensor that would pick up the object in orbit (when searched or generated).

Optical	Radar	Radio Frequency (RF)	Fusion of all sensor types	Optical/RF	Optical/Radar	Radar/RF
OP	RA	RF	FU	OR	RO	RR

Orbit Coverage:

The orbital coverage is the amount of the object's path covered by the data, based on its orbital regime there are accepted minimum values for identification based on how covered the object in orbit is.

The relationship between the orbital regime and the minimum (not Low coverage) acceptable orbital coverage to identify the object in orbit is shown by the chart.

LEO (Low <0.0213% covered)
MEO (Low <0.0449% covered)
GEO (Low <41.656% covered)

The letters that represent orbital coverage are A, S, N, and the values are defined by the chart below:

>90% of objects have low orbital coverage	40-60% of objects have low orbital coverage	<10% of objects have low orbital coverage
A	S	N

Track Gap:

The track gap is the longest duration of time between observations of an object, the shorter the better. According to our sponsors, a 'long' track gap is defined as a duration of two orbital periods of the object. When preparing the example data, Orbital Regime and Fitspan take precedence over track gap which means the track gap; when given, is more likely to be considered 'long'. This infers that track gap as a metric for identifying objects is less important, which is possibly why the generated data is less 'reliable' than the data directly from the UDL. The convention also uses the same letters that Orbital Coverage uses but it is important to note that the value of representation is different than Orbital Coverage:

>90% of objects have a long track gap	40-60% of objects have a long track gap	<10% of objects have a long track gap
A	S	N

Observation Count:

The observation count is the total number of unique observations per object per 3-day-timespan. A 'low' observation count is less than 50, standard' observation count is between 50-150 and objects with greater than 150 observations will not be used or will be down sampled to a 'standard' observation count. This uses the same letters that Orbital Coverage uses but it is important to note that the value of representation is different than Orbital Coverage:

>90% of objects have low observation count	40-60% of objects have low observation count	<10% of objects have low observation count
A	S	N

Object Count:

The object count is the total number of objects to be included in the object data set give or take 2 objects, it is identified by three letters H, S, L. The values of the letters are defined by the chart below:

High number of objects: 80 +/- 2	Standard Number of Objects: 40 +/- 2	Low number of objects: 10 +/- 2
H	S	L

The fitspan is the duration of the dataset spans in days from 01 to 14.

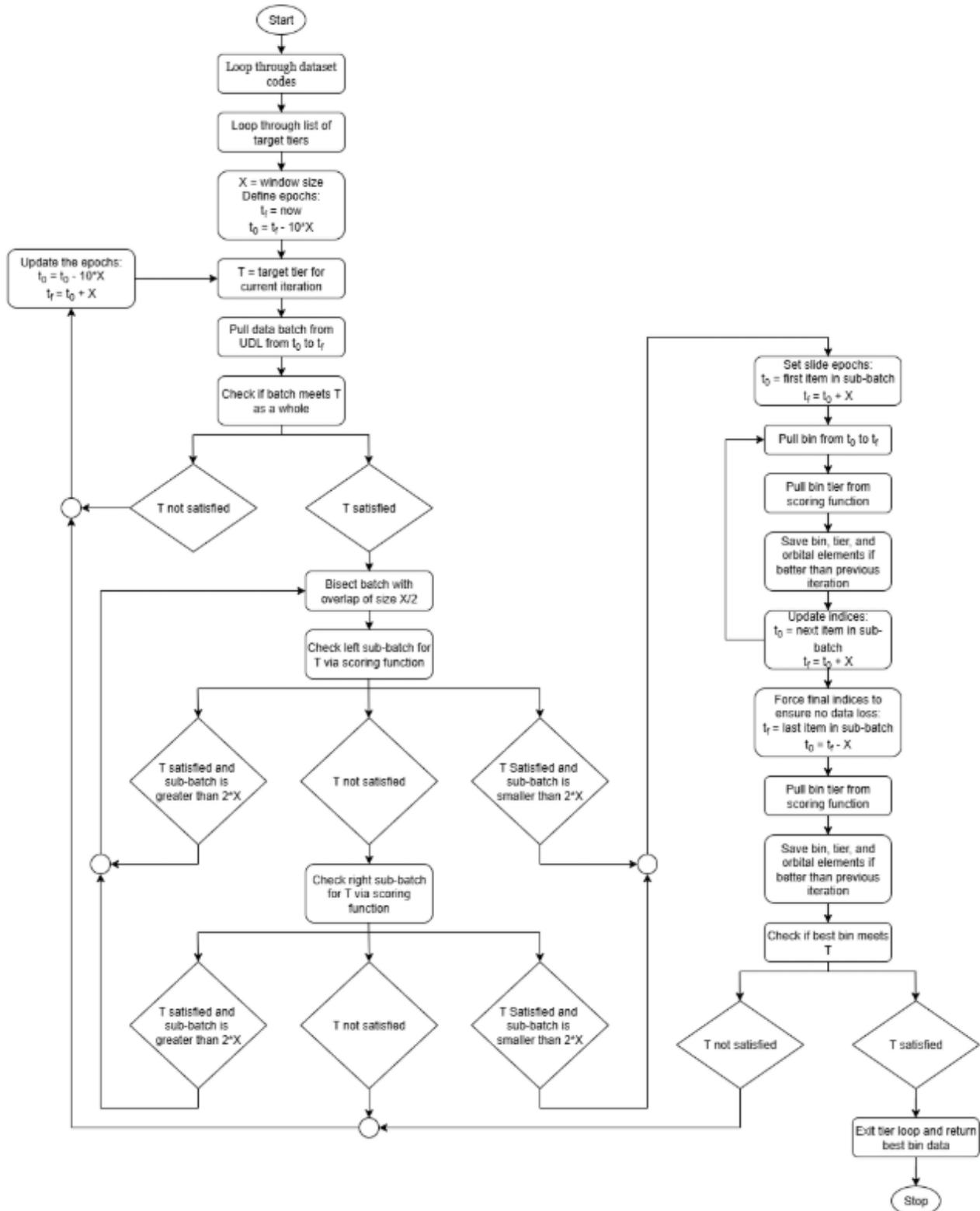
Actively Assembling the Dataset. Fit metrics determines the unit-sphere projected great circle residuals between the actual observations and the propagation observations. The function is split into two categories:

- Comparison of the reference (truth) observations with the associated orbits (how accurate a correlated state is)
- Comparison of the reference (truth) observations with the UCTP output (how precise a correlated state is)

The character string makes up the queries that. The steps to then use these parameters for the given datasets are as follows: (on the following page)

The first step in creating a dataset is to identify the window as stated previously. The most optimal time is when there is no simulation or down sampling required. Which is shown on the following chart.

Figure 3.4.2: This diagram showcases the workflow for evaluation and data generation for UCTP.



Time Window Selection

The first step to fully preparing and giving the data is identifying an ideal time window based off of the fitspan and other variables present in the string. The resulting quality of time window selection is separated into tiers according to this chart:

Tier 1	Tier 2	Tier 3	Tier 4	Tier 5
(optimal) No downsampling No simulation	No simulation	Simulation of objects already present	Simulation of new objects	(worst) Criteria is impossible

3.5 Unified Data Library (UDL)

The Unified data library has literally hundreds of API endpoints for information collected across different disciplines. These range from aircraft to objects in space. Our sponsor specifies that the schemas of the information will be of the form EEObservation_Full or Elset_Full, the parameters of the schema for the EEObservation_Full is as follows:

EEObservation_Full: Model representation of observation data for electro-optical based sensor phenomenologies.

Id string: Unique identifier of the record, auto generated by the system.

classificationMarking string: Classification marking of the data in IC/CAPCO Portion-marked format. (IC/CAPCO Portion-marked is the standardized way the U.S. Intelligence community applies classification markings to individual portions of a document)

obTime string (date-time): Object detection time in ISO 8601 UTC, up to microsecond precision. (For example, 3pm EST = 2025-11-20T20:00:00Z)

corrQuality *number (double)*: Object Correlation Quality score of the observation when compared to a known orbit state, the value ranges differ from provider to provider (non-standardized).

idOnOrbit *string*: Unique identifier of the target on-orbit object, if correlated.

idSensor *string*: Unique identifier of the reporting sensor.

idSkyImagery *string*: Unique identifier of the Sky Imagery.

collectMethod *enum (string)*: Method indicating telescope movement during collection, while this is a standardized enumerator how each provider utilizes these options and the exact description of these values varies from provider to provider. The enumerators are as follows: AUTOTRACK, MANUAL AUTOTRACK, RATETRACK, MANUAL RATE TRACK, SIDEREAL, MANUAL SIDEREAL

satNo *integer (32 bit)*: Satellite/Catalog number of the target on-orbit object.

taskId *string*: Optional identifier to indicate the specific tasking which produced this observation.

transactionId *string*: Optional identifier to track a commercial or marketplace transaction executed to produce this data. Like all ids, this is for traceability and uniqueness for the observation event, however in this case it is to distinguish where this information is from on the commercial side of data.

trackId *string*: Optional identifier of the track to which this observation belongs. (Track is the record of an object's position at a specific timestamp.)

obPosition *enum (string)*: An optional value of the position of this observation within a track, the values are as follows:

obPosition	Description
FENCE	The observation was taken at the start of a collection window or boundary condition.
FIRST	This is the first detection of the object in a sequence of observations.

IN	Intermediate observation within a sequence. (not first or last observation)
LAST	Final observation in a sequence before the object leaves the sensor's fields of view or the collection ends.
SINGLE	There is only one observation of the object, not a sequence
null	No assumption should be made regarding whether other observations may or may not exist to compose a track.

origObjectId *string*: Optional identifier provided by observation source to indicate the target on-orbit object of this observation. This may be an internal identifier and not a valid satellite number.

origSensorId *string*: Optional identifier provided by observation source to indicate the sensor identifier which produced this observation. This may be an internal identifier and not a valid sensor ID.

uct *Boolean*: Boolean value that indicates whether this observation is part of an uncorrelated track or was unable to be correlated to a known object. This flag should only be set to true by data providers after an attempt to correlate to an on-orbit object was made and failed. If unable to correlate, the 'origObjectId' field may be populated with an internal data provider specific identifier.

azimuth number (*double*): Line of sight azimuth angle in degrees and topocentric frame. Reported value should include all applicable corrections as specified on the source provider. The definition of this value and a visual representation are as follows:

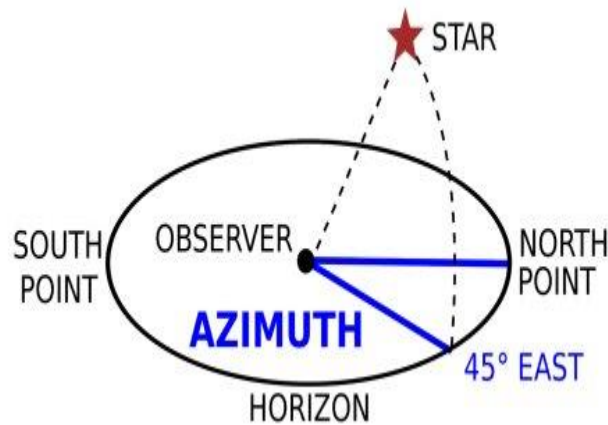


Figure 3.5.1: This figure showcases the azimuth which is the direction of a celestial object from the observer, expressed as the angular distance from the north or south point of the horizon to the point at which a vertical circle passing through the object intersects the horizon.

azimuthMeasured *Boolean*: Optional flag indicating whether the **azimuth** value is measured [true] or computed [false]. If null, the provider must be contacted for the proper value. If measured, the limit of the accuracy of the **azimuth** is relative to the sensor, if computed the limit of the accuracy is relative to the program that computes it.

azimuthBias *number (double)*: Sensor line of sight azimuth angle bias in degrees.

azimuthRate *number (double)*: Rate of change of the line-of-sight azimuth in degrees per second.

elevation *number (double)*: Line of sight elevation in degrees and topocentric frame. Reported value should include all applicable corrections from the source provider.

elevationMeasured *Boolean*: Optional flag indicating whether the **elevation** value is measured [true] or computed [false]. If null, the provider must be contacted for the proper value. If measured, the limit of the accuracy of the **elevation** is relative to the sensor, if computed the limit of the accuracy is relative to the program that computes it.

elevationUnc *number (double)*: One sigma uncertainty in the line-of-sight elevation angle, in degrees.

elevationBias *number (double)*: Sensor line of sight elevation bias in degrees.

elevationRate *number (double)*: Rate of change of the line of sight elevation in degrees per second.

range *number (double)*: Line of sight range in kilometers (km). If **referenceFrame** (another value defined later) is null, Julian Date 2000 (J2K) should be assumed. All corrections should be provided by the data provider.

rangeMeasured *Boolean*: Optional flag indicating whether the **range** value is measured [true] or computed [false]. If null, the provider must be contacted for the proper value. If measured, the limit of the accuracy of the **range** is relative to the sensor, if computed the limit of the accuracy is relative to the program that computes it.

rangeUnc *number (double)*: One sigma uncertainty in the line-of-sight range, in kilometers.

rangeBias *number (double)*: Sensor line of sight range bias in km.

rangeRate *number (double)*: Range rate in km/s. If **referenceFrame** is null, J2K should be assumed. All corrections should be provided by the data provider.

rangeRateMeasured *Boolean*: Optional flag indicating whether the **rangeRate** value is measured [true] or computed [false]. If null, the provider must be contacted for the proper value (measured or computed). If measured, the limit of the accuracy of the **rangeRate** is relative to the sensor, if computed the limit of the accuracy is relative to the program that computes it.

rangeRateUnc *number (double)*: One sigma uncertainty in the line-of-sight range rate, in km/s. **ra (right ascension)** *number (double)*: Line of sight right ascension, in degrees, in the specified **referenceFrame**. If **referenceFrame** is null, J2K should be assumed. All applicable corrections should be specified by the provider. Here is a diagram of **right ascension**:

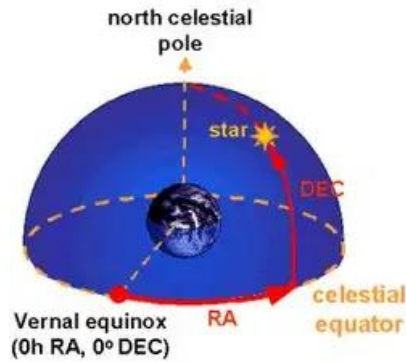


Figure 3.5.2: This figure showcases the right ascension.

raMeasured *Boolean*: Optional flag indicating whether the **ra** value is measured [true] or computed [false]. If null, the provider must be contacted for the proper value (measured or computed). If measured, the limit of the accuracy of the **ra** is relative to the sensor, if computed the limit of the accuracy is relative to the program that computes it.

raRate *number (double)*: Line of sight right ascension rate of change, in degrees/sec, in the specified **referenceFrame**. If **referenceFrame** is null, then J2K should be assumed.

raUnc *number (double)*: One sigma uncertainty in the line-of-sight right ascension angle, in degrees.

raBias *number (double)*: Sensor line of sight right ascension bias in degrees.

declination *number (double)*: Line of sight declination, in degrees in the specified **referenceFrame**. If **referenceFrame** is null, then J2K should be assumed. Here is a diagram of **declination**:

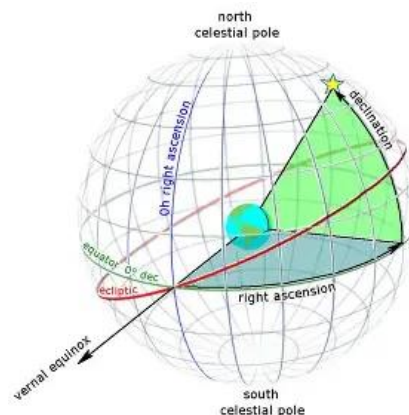


Figure 3.5.3: This figure showcases the idea of declination.

declinationMeasured *Boolean*: Optional flag indicating whether the **declination** value is measured [true] or computed [false]. If null, the provider must be contacted for the proper value (measured or computed). If measured, the limit of the accuracy of the **declination** is relative to the sensor, if computed the limit of the accuracy is relative to the program that computes it.

declinationRate *number (double)*: Line of sight declination rate of change, in degrees/sec, in the specified **referenceFrame**. If **referenceFrame** is null then J2K should be assumed. **declinationUnc** *number (double)* One sigma uncertainty in the line-of-sight declination angle, in degrees.

declinationBias *number (double)*: Sensor line of sight declination angle bias in degrees.

los (Line of sight) *number (double)*: Line of sight cartesian position of the target, in km, in the specified **referenceFrame**. If **referenceFrame** is null, then J2K should be assumed. This line of sight is divided into **losx**, **losy**, **losz**, which are relative to the respective cartesian axis.

losUnc *number (double)*: One sigma uncertainty in the line of sight pointing in micro-radians.

losvel *number (double)*: Line of sight cartesian velocity of the target, in km, in the specified **referenceFrame**. If **referenceFrame** is null then J2K should be assumed. This line of sight is divided into **losxvel**, **losyvel**, **loszvel**, which are the velocities relative to the respective cartesian axis.

senlat *number (double)*: Sensor World Geodetic System 1984 (WGS84) latitude at **obTime** (if mobile/onorbit) in degrees. If null, can be obtained from sensor info. **senlat** ranges from -90 to 90 degrees (negative values south of equator)

senlon *number (double)*: Sensor WGS84 longitude at **obTime** (if mobile/onorbit) in degrees. If null, can be obtained from sensor info. **senlon** ranges from -180 to 180 degrees (negative values west of Prime Meridian)

senalt *number (double)*: Sensor WGS84 altitude at **obTime** (if mobile/onorbit) in km.

sen *number (double)*: Cartesian position of the observing mobile/onorbit sensor at **obTime**, in km, in the specified **senReferenceFrame** (defined later). If **senReferenceFrame** is null, then J2K should be assumed. This value is split into **senx**, **seny**, **senz** which are relative to the respective cartesian axis.

senvel *number (double)*: Cartesian velocity of the observing mobile/onorbit at obTime, in km/sec, in the specified **senReferenceFrame**. If **senReferenceFrame** is null then J2K should be assumed. This value is split into **senvelx**, **senvely**, **senvelz** which are relative to the respective cartesian axis.

senQuat *Array[number (double)]*: The quaternion describing the rotation of the sensor in relation to the body-fixed frame used for this system into the local geodetic frame at observation time (**obTime**)/ The array element order convention is scalar component, followed by the three vector components (qc,q1,q2,q3).

fovCount *integer (32 bit)*: The number of Resident Space Objects (RSOs) detected in the sensor field of view.

fovCountUCT *integer (32 bit)*: The number of uncorrelated tracks in the field of view.

expDuration *number (double)*: Image exposure duration in seconds. For observations performed using frame stacking or synthetic tracking methods, the exposure duration should be the total integration time.

zeroptd *number (double)*: Result of this formula: $2.5 * \log_{10}(\text{zero_mag_counts (constant)} / \text{expDuration})$

netObjSig *number (double)*: Result of this formula = **counts** (constant) / **expDuration**

netObjSigUnc *number (double)*: Net object signature uncertainty = **counts** (constant)/ **expDuration**

mag *number (double)*: Measure of observed brightness calibrated against the Gaia G-band in units of magnitudes.

magUnc *number (double)*: Uncertainty of the observed brightness in units of magnitudes.

magNormRange *number (double)*: Formula: **mag** - 5.0 * $\log_{10}(\text{georange (defined later)} / 10^6)$

geolat *number (double)*: For GEO detections, the longitude in degrees north.

geolon *number (double)*: For GEO detections, the longitude in degrees east.

geoalt *number (double)*: For GEO detections, the altitude in km.

georange *number (double)*: For GEO detections, the range in km.

skyBkgrnd *number (double)*: Average Sky Background signal in Magnitudes. Sky Background refers to the incoming light from an apparently empty part of the night sky.

primaryExtinction *number (double)*: Primary Extinction Coefficient, in Magnitudes. The Primary Extinction is the coefficient applied to the airmass to determine how much of the observed visual magnitude has been attenuated by the atmosphere. Extinction, in general, describes the absorption and scattering of electromagnetic radiation by dust and gas between an emitting astronomical object and the observer.

primaryExtinctionUnc *number (double)*: Primary Extinction Coefficient Uncertainty, in Magnitudes.

solarPhaseAngle *number (double)*: The angle, in degrees, between the target-to-observer vector and the target-to-sun vector

solarDecAngle *number (double)*: Angle from the sun to the equatorial plane.

shutterDelay *number (double)*: Shutter delay in seconds.

timingBias *number (double)*: Sensor timing bias in seconds.

rawFileURI *string*: Optional URI location for a separate API endpoint to retrieve the raw data file that produced this record.

intensity *number (double)*: Intensity of the target for infrared (IR) observations, in kw/sr/um.

bgIntensity *number (double)*: Background intensity for IR observations, in kw/sr/um.

descriptor *string*: Optional source-provided and searchable metadata of the data.

source *string*: Source of the data.

origin *string*: Originating system or organization which produced the data, if different from the **source**. The **origin** may be different than the **source** if the **source** was a mediating system which forwarded the data on behalf of the origin system. If null, the **source** may be assumed to be the **origin**.

dataMode *enum (string)*: This is the “mode” of the data, or how accurate to real life the data is itself.

dataMode	Description
EXERCISE	Data pertaining to a government or military exercise. May include both real and simulated data.
REAL	Data collected or produced that pertains to real-world objects, events and analysis.
SIMULATED	Synthetic data generated by a model to mimic real-world datasets.
TEST	Specific datasets used to evaluate compliance with specifications and requirements, and for validating technical, functional and performance characteristics.

tags *Array[string]*: Optional array of provider/source specific tags for this data, **tags** are used for implementing data owner conditional access controls to restrict access to the data.

createdAt *string(date-time)*: Time the row was created in the database, auto-populated by the system.

createdBy *string*: Application user who created the row in the database, auto populated by the system.

zeroPtdUnc *number (double)*: This is the uncertainty in the zero point for the filter used for this observation/row in units of magnitude. This value is used for differential photometry.

eoobservationDetails *EOObservationDetails_Full*: The EOObservationDetails_Full is another data schema entirely, whose schema is written out in the UDL. It is a model representation of additional detailed observation data for electro-optical based sensor phenomenologies.

onOrbit *Onorbit_Full*: The Onorbit_Full is another data schema entirely, like the EOObservationDetails_Full. It is a model representing on-orbit objects or satellites in the system.

referenceFrame *enum (string)*: The reference frame of the EOObservation measurements. If null, assume J2K. Here are the values for **referenceFrame**: [J2K, GCRF ,ITRF ,TEME]

senReferenceFrame *enum (string)*: The reference frame of the observing sensor state. If null, assume J2K. Here are the values for **senReferenceFrame**: [J2K, EFG/TDR, ECR/ECEF, TEME, ITRF, GCRF]

umbra *Boolean*: Boolean indicating that the target object was in umbral eclipse at the time of this observation.

penumbra *Boolean*: Boolean indicating that the target object was in a penumbral eclipse at the time of this observation.

origNetwork *string*: The originating source network this record was created, auto-populated by the system.

sourceDL *string*: The source data library from which this record was received . This can be a remote or tactical UDL or another data library. If null, the record should be assumed to have originated from the primary Enterprise UDL.

type *string*

Read only field specifying the type of observation.

Richness in Data: The richness of the data provided by every UDL API endpoint cannot be understated, each measurement coupled with an uncertainty value, multiple enumerators changing the exact context of the observation itself. Some values are just results of simple calculations using the given data values. With this richness in data came a large problem the team had, which was trying to understand what this data truly means in relation to the UCT Processing pipeline. For example, the **eoobservationDetails** and **onOrbit** values inside the **EOObservation_Full** schema are schemas themselves which have a multitude of parameters all with similar precision and paper trails for the information itself. An AI just to look through the schemas and give a crash course to people getting into the UDL or any system like it would be a project in itself.

Effectively Using the Data: The data itself is rich, very rich; however, our sponsor provided a narrower list of the elements from each observation with the formatting that they wanted the Open Evolve evolved program to output as. This means that of the 90 elements of the schema given originally, the output contains much less elements compared, the exact value varies depending on whether the data is Elset data or an EOObservation_Full data.

Elset_Full: The Elset data contains information pertaining to orbital characteristics of on-orbit objects. However, going into detail about its parameters isn't as important as only some of the expected values are included inside the format provided by the sponsor, these values are **epoch, cov covReferenceFrame, linarSolar, solarRadPress, solidEarthTides,inTrackThrust,rms and algorithm**.

3.6 Conclusion

UCTP draws on a wide range of data sources. Many of its orbit estimation and accuracy metrics are built from publicly available services that track known objects in orbit. When you look at how this data is generated, transformed, and fed into the processor for evaluation, it becomes clear just how complex and multidisciplinary UCTP really is. That naturally leads to the key question: **how do we tame such a complex multi-disciplinary system with many moving parts into an AI architecture?**

Chapter 4. OpenEvolve

4.1 Introduction

OpenEvolve is an open-source framework designed to **automate algorithm discovery and code optimization**. Instead of the usage of UCTP algorithms and tuning them manually, OpenEvolve uses large language models (LLMs) together with evolutionary search using island-based convergences to automatically generate, modify, and evaluate candidate programs based off of evaluation metrics. At each iteration, the system proposes small, structured “mutations” to the code, scores them using user-defined metrics, and then keeps the best performers for the next generation. This allows the program to keep diversification to combat again premature convergences. Over time, this process explores a wide range of algorithmic designs and converges toward higher-quality solutions.

For UCTP, OpenEvolve directly addresses the lack of standardized benchmarks and systematic evaluation. By pairing our benchmark datasets and metrics (state, residual, binary classification) with OpenEvolve’s automated search, we can rapidly test many different UCTP algorithms, compare them in a consistent, quantitative way, and discover novel approaches that might not emerge from manual design alone. In this way, OpenEvolve becomes both a **benchmarking engine** and a **code-generation tool**, helping us identify, refine, and justify UCTP algorithms for Space Domain Awareness.

4.2 Background: Evolutionary Code Optimization

Traditional algorithm development depends on manual experimentation. Developers have to iteratively adjust parameters, rewrite portions of code, and measure performance until wanted results are reached. This process simply put is slow.

Evolutionary optimization looks to a different angle to the problem inspired by natural selection. In the examples, a population of candidate solutions evolves under selective pressure. Poorly performing algorithms are discarded, and the promising solutions are retained, modified, and combined to produce new generations. Over many iterations, the population converges toward more effective solutions without requiring explicit design rules.

OpenEvolve applies this principle directly to source code rather than just numeric parameters. Each candidate in the evolutionary population is a block of executable code that can be modified, executed, and evaluated. The system uses LLMs as the means to mutate, capable of generating new code that is syntactically correct and logically sound. These LLMs can reason about structure and mathematical relationships, transforming random search into a guided, semantically aware exploration.

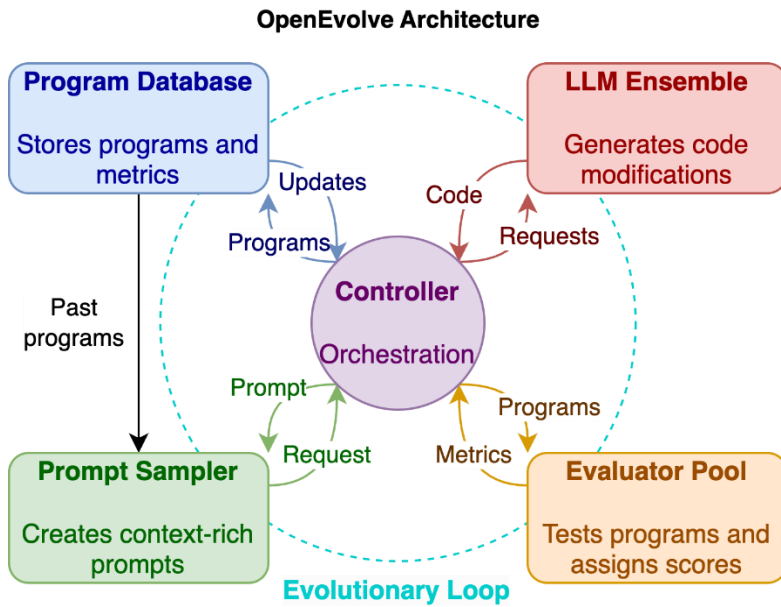
By combining the adaptability of evolutionary algorithms with the contextual reasoning of LLMs, OpenEvolve can have continuous, autonomous experimentation. This capability has relevance to UCTP research, where algorithmic improvements depend on subtle interactions between filtering logic, regression models, and data association functions that are difficult to discover manually.

4.3 OpenEvolve Framework Overview

OpenEvolve is an open-source adaptation of DeepMind’s AlphaEvolve framework. It automates algorithm discovery by coordinating multiple components, a distributed controller, one or more LLM “islands,” evaluation pipelines, and a database of program artifacts. Through these components, OpenEvolve supports thousands of concurrent experiments, allowing large portions of the algorithmic search space to be explored at the same time.

Each OpenEvolve experiment follows this structured cycle:

1. **Generation:** LLMs propose new versions of a target code segment, modifying existing logic or generating entirely new constructs.
2. **Evaluation:** Each candidate program is executed against a predefined test harness that measures objective performance metrics (e.g., accuracy, error, or runtime).
3. **Selection:** Candidates are ranked according to fitness, and the top performers are retained for further mutation.
4. **Reproduction:** High-performing code fragments are recombined and reintroduced into the population, forming the next generation.
5. **Persistence:** All artifacts, code, logs, metrics, and metadata are stored in a version-controlled database, ensuring deterministic reproducibility.



Asynchronous pipeline optimized for maximum throughput

This loop continues until convergence criteria are met or a user-specified iteration limit is reached. The framework's design supports multi-objective optimization, allowing users to balance accuracy, robustness, and computational efficiency simultaneously.

OpenEvolve also utilizes a Quality-Diversity search strategy based on the MAP-Elites algorithm. Rather than retaining only a single highest-

performing program, MAP-Elites maintains an archive of diverse candidate solutions along multiple feature dimensions including runtime, memory usage, and code structure complexity. This prevents the search from collapsing into a small region of solution space and allows OpenEvolve to find very different algorithmic solutions that turn out to be superior to traditional designs in particular contexts.

OpenEvolve's architecture includes several key technical components:

- **Controller Loop:** Orchestrates population management, parallel evaluations, and iteration control.
- **LLM Ensemble:** Multiple language models (e.g., OpenAI, Gemini, Anthropic) operate as independent "islands." They periodically exchange elite solutions to prevent premature convergence and encourage diversity.
- **Evaluator:** A user-defined Python script that scores each candidate program's output. For this project, evaluators will measure regression accuracy or related tracking metrics relevant to UCTP.
- **Database and Artifact Tracker:** Captures all source code versions, execution logs, and performance data to guarantee reproducibility across runs.

Since OpenEvolve integrates with any OpenAI-compatible API, it can run using cloud based models or local deployments depending on project constraints. OpenEvolve also uses deterministic seeds which allow us to fully reproduce our tests.

What is very important in this architecture as explained above from previous bullet points is that OpenEvolve has a non-blocking (asynchronous pipeline). Besides bringing the benefit of speed execution, the asynchronous controller architecture supports scalability across a distributed computing environment. Each island independently requests LLM mutations, executes evaluations, and stores artifacts, all without waiting on other islands. This non-blocking design makes it possible for OpenEvolve to execute thousands of concurrent experiments, which enables large algorithmic search spaces to be explored in parallel and significantly quickens convergence. This means that there is parallelism when computing where the main control orchestration does not wait to finish one task to run another. This is important because it allows the Evolution to generate more quickly. One main problem we have with introducing OpenEvolve's AI architecture into the UCT workflow is that we will need to create a pipeline that corresponds to UCTP.

4.3.1 Configuration and Extensibility

A major strength in OpenEvolve is the modular configuration system that enables adaptation of the framework to different problem domains without changes in its central architecture. Configuration in OpenEvolve is done through a single file in which the mutation strategy, LLM model provider, population size, island topology, and evaluation metrics are defined for a given experiment. This enables researchers to quickly change the behavior of an experiment through configuration parameters rather than by redesigning the evolutionary process.

As such, the evaluator is the central extension point within the framework. It requires a user to provide a Python-based scoring function that executes each candidate program and returns performance metrics. Provided that the evaluator outputs quantitative measurements, OpenEvolve can integrate them into its selection and reproduction process. This allows the system to support a wide range of objectives, be it accuracy, runtime performance, memory footprint, or domain-specific scoring criteria. More specifically for UCTP applications, the evaluation functions may incorporate orbital state error, residual calculations, or binary classification performance when comparing candidate and reference orbits.

The configuration system also supports multiple LLM providers through an OpenAI-compatible interface, which lets the framework operate using cloud-based models or locally hosted language models depending on resource constraints. In the context of UCTP, this configurability becomes essential

because the various evaluation criteria to score candidate algorithms will change due to mission needs, sensor availability, and even orbital prediction models. As new sensor suites are introduced or requirements for tracking change, the various UCTP evaluation functions can be updated within the configuration file without redesigning the evolutionary system. This is how OpenEvolve can evolve with the UCTP workflow, continuously optimizing algorithms over time based on updated residual models, orbital state estimates, or binary association logic.

4.4 OpenEvolve Basic Functionality & Process

Due to the complexity of OpenEvolve's architecture and process, this section serves to showcase a simple problem that goes through the process for the reader to understand. Which will then be connected to the UCTP framework and how we plan to integrate UCTP into OpenEvolve's AI architecture. We first start off with a simple program in computer science. That program being Maximum Subarray Sum taken from LeetCode:

Example 1:

Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
Explanation: The subarray [4,-1,2,1] has the largest sum 6.

Example 2:

Input: nums = [1]
Output: 1
Explanation: The subarray [1] has the largest sum 1.

Example 3:

Input: nums = [5,4,-1,7,8]
Output: 23
Explanation: The subarray [5,4,-1,7,8] has the largest sum 23.

Using a brute force method, we can get the answer with a runtime of $O(n^3)$. Which is not the best when it comes to the most optimal solution. Using data structure techniques, we know we can get an optimal solution of $O(n)$ using a left pointer by skipping negative evaluations and a right pointer to iterate through. How OpenEvolve works here is using this example of trying to get the most optimal solution to this maximum subarray sum problem. OpenEvolve will need three inputs:

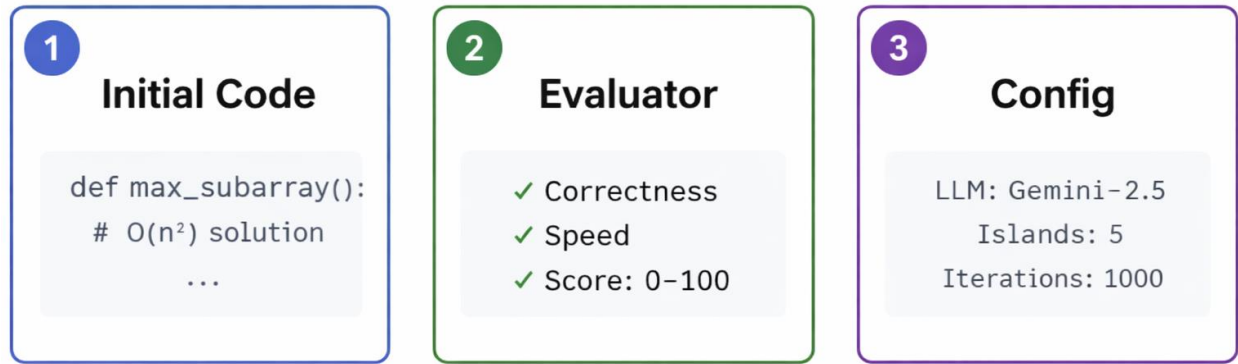


Figure 4.4.1: This figure showcases the required inputs to run OpenEvolve. Where we input the $O(n^3)$ code of maximum subarray sum problem. And with the evaluator file, we ask OpenEvolve to find a more optimal solution through speed and correctness. With these metrics we can score each program through each iteration. The Config is what we use to code generate and manipulate. In OpenEvolve’s benchmark research, they find that using simple LLMs like Gemini-2.5 performs the best.

Now that we have the required files needed to run OpenEvolve, how does it find the best program? OpenEvolve uses a technique called island-based convergence which comes from island-model evolutionary algorithms. Where instead of evolving one big population of programs, we can evolve smaller populations (“islands”) to promote parallelism. The following figure will give a better example of how OpenEvolve finds the best program:



Figure 4.4.2: This figure showcases the different runtimes OpenEvolve tries to test when finding the optimal solution to the maximum subarray sum problem. Where every generation, we do migrations where we take the best performing programs from one island and move them to another. This allows us to have fast convergence times and also allows us to maintain diversity to allow the best solutions to spread throughout the migration process. This prevents

Where every N generation, we do migrations where we take the best performing programs from one island and move them to another. This allows us to have fast convergence times and also allows us to maintain diversity to allow the best solutions to spread throughout the migration process. This prevents the islands from becoming similar too quickly and also prevents premature convergences. This

technique helps promote exploration and parallelism by isolating populations instead of having to converge on one big population or code. Each island has its own evolutionary loops, which is what we know from the current outputs of OpenEvolve's directory.

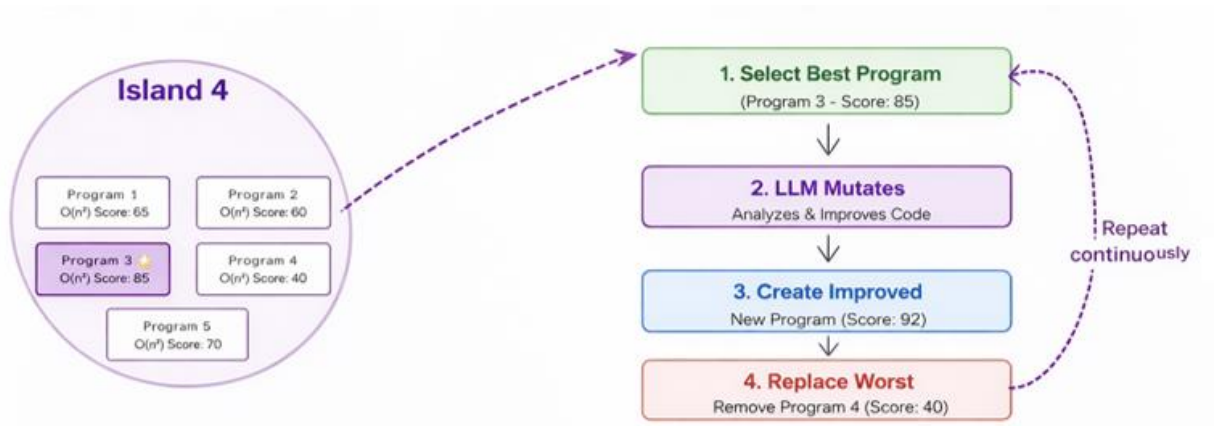


Figure 4.4.3: This figure showcases OpenEvolve uses the evaluation metrics to score each program in order to find which program to migrate when trying to find the best solution.

To showcase how dynamic OpenEvolve is when it comes to evolving code, in this case when solving the maximum subarray sum problem. It knows that based off the user metrics of scoring how well a program does base off of speed and accuracy. It knows that the most optimal solution we can get is $O(n)$. In this case it did find a solution which is using prefix sum approach. But the scoring of the solution is still 85/100. Is there a better way to solve this problem?

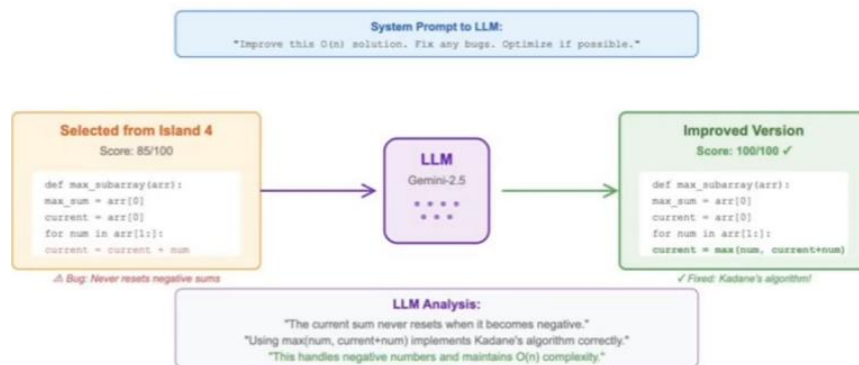


Figure 4.24: This figure shows why the usage of LLM works well when working with dynamic algorithms that need to evolve over time. Not only did it find a solution of $O(n)$ runtime, but it also works to improve it.

In this case OpenEvolve did catch a big component where you are able to reset on 0. Hence, Kadane's Algorithm. This is a very important part of OpenEvolve when trying to solve UCTP limitations.

4.5 UCTP integration to OpenEvolve

So now that we understand how OpenEvolve works and its strength, how do we integrate UCTP into this AI architecture? Knowing the limitations of UCTP, we understand that it has no benchmarks we can compare it to, its algorithms cannot evolve over time with new sensors being implemented into the satellites, and there is a big interest in having a real-time implication of Space Domain Awareness. We understand from previous chapters about UCT's workflow of data generation and evaluation generation. We then feed that into a processor to understand how we can group uncorrelated tracks. OpenEvolve introduces a wide variety of solutions to optimize the workflows of UCTP.

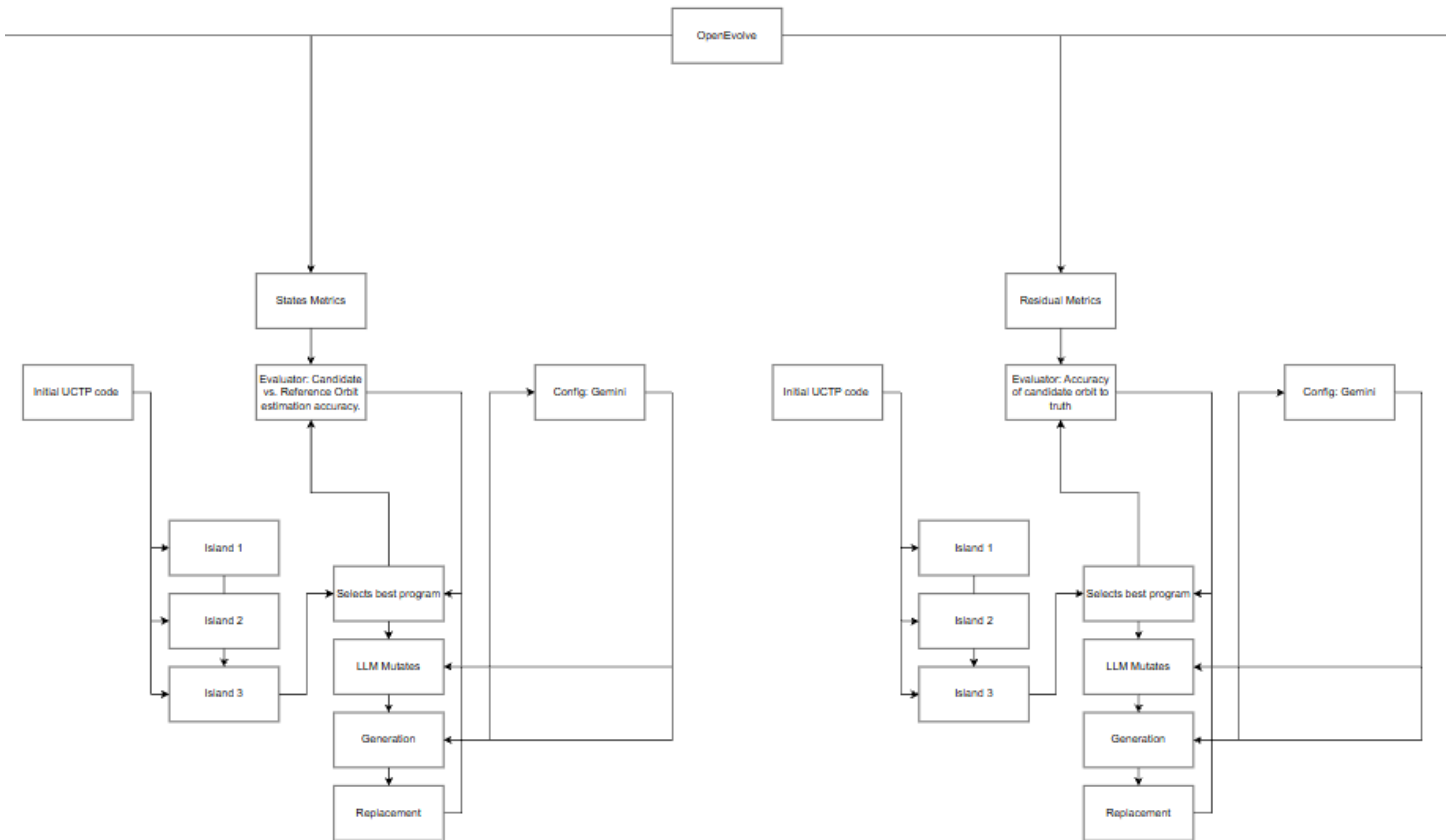
The whole idea of the processor is trying to figure out the objects in a dataset. There are two main things that would be nice to optimize for the accuracy of the fit. As stated before, UCTP has metrics it takes account of when evaluating (state, residual, binary). These are all just different algorithms on trying to create a dataset based off of different equations used to find how accurate our Candidate orbits are versus our Reference orbits. We can use OpenEvolve to optimize these workflows. Alongside looking further into the future where UCTP can function in real time for new sensor suites. Where algorithms are able to evolve with new data and new approaches.

But this raises the question of how large and how computationally expensive this framework can get. If we look at UCTP workflows and pipelines, anything can become an island to optimize. The following questions need to be answered:

- Are we using OpenEvolve on a small-scale where we're already mutating from already existing algorithms used by UCTP to optimize and find better algorithms.
- Are we using OpenEvolve on a larger scale where we have a dynamic benchmarking system between OpenEvolve and UCTP as new data comes in where the algorithm is always mutating?

These questions we asked to the sponsor. The current direction at the moment is solving OpenEvolve on a smaller scale for UCT processors. But looking at how complex UCTP is, it will become a lot larger in the future. Especially with the implementation of real-time UCTP where it evolves through time. On a

smaller scale, our group has created a diagram showcasing how large the framework can get when integrating UCTP into OpenEvolve. This current idea is to pull the algorithms and equations used to evaluate the accuracy of Candidate Orbit versus Reference Orbit. Where each evaluation can be turned into an island:



This UML diagram only shows two out of the four evaluation metrics that UCTP uses when creating the dataset. We also have to keep in mind that this is only the first part of UCTP where we were creating a dataset that the processor can handle. We also need to handle the benchmarking for UCTP. This proposes another big problem of the main reason we are using OpenEvolve. Which is to benchmark UCTP with a known solution. Does this mean that we would also need to integrate the processor portion of UCT and compare the solutions OpenEvolve generates to the current processor?

4.5.1 Early UCTP Integration Efforts

As the development continued, we were provided with sample output data produced by UCTP. This consisted of observational entries and their corresponding reference groupings, so we could begin experimenting with the end-to-end integration of OpenEvolve and a realistic UCTP dataset. To explore this, an initial candidate program was written to parse the JSON-formatted dataset and perform a simple grouping algorithm. The initial program reads the observational part of the dataset and populates observation identifiers into state vector structures that match the UCTP output format, generating a complete JSON file that can be processed by our defined evaluator.

This output from the initial program is fed into the evaluator, where the program's produced groupings are compared against the reference truth contained within the sample UCTP output. This evaluation leverages the scoring mechanism of the evolutionary framework so that quantitative judgments can be made about how close the initial program's grouping is to the expected associations. With this evaluation setup working, OpenEvolve was used to modify and run variations of the program in the hopes of improving it. In such a workflow, we were able to see whether evolutionary techniques would be able to refine the initial grouping behavior and move the output closer to the reference truth. Although these integration efforts are still in the early stages, this work demonstrates that the system is capable of executing, evaluating, and evolving candidate UCTP algorithms using real data.

4.6 Conclusion

OpenEvolve gives us a practical way to move UCT processing out of slow, hand-tuned loops and into a more continuous, evidence-driven search. By combining evolutionary strategies with LLM-guided code mutations, we can explore many competing algorithmic ideas at once, measure them against clear UCTP metrics (state, residual, binary classification), and keep only what demonstrably works. The island-based model and asynchronous controller make this search fast and scalable; MAP-Elites keeps the search diverse, so we don't prematurely converge to a solution. Most importantly, every candidate, score, and artifact is reproducible, so there are traceable results.

For UCTP, this changes the conversation. Instead of arguing over which filter, regressor, or association methodology to try next, we define the evaluators and let the system run competitive trials at scale. In early integration tests, even simple grouping programs could be evolved toward the reference truth, showing that the framework can ingest real UCT outputs, execute candidates in end-to-end loops, and improve them iteratively. As benchmark datasets grow and sensor suites change, configuration—not

rewrites—will adapt the search. That’s the path to a living, real-time UCT pipeline that learns as conditions evolve.

What remains is disciplined engineering:

- Scoping impactful evaluation metrics, primarily in the binary functions.
- Sustaining computing costs while preserving diversity and throughput, with such a complex framework, it can easily get of the budgeting scope.
- Hardening the evaluator/test harness to reflect real operational environments with computing power (latency, memory, failure modes).

If we execute those items, OpenEvolve becomes more than a research tool: it’s a repeatable benchmarking engine and an algorithm foundry for Space Domain Awareness. This can evolve into a more dynamic real time solution which will improve the overall functionality of Space Domain Awareness.

Immediate Next Steps

1. **Lock evaluation ground truth & metrics:** finalize state, residual, and binary scoring specs and edge-case handling.
2. **Develop a “small-scale” island set with a small UCT processor:** run seeded, reproducible experiments on two evaluator targets; establish baseline fitness curves and time-to-improvement.
3. **Compute & cost guardrails:** We are currently running basic LLMs like gemini-2.5 to minimize costs which should not be a problem since in OpenEvolve’s research gemini-2.5 did the best when it came to benchmark against other open-source LLMs.
4. **Test viability of OpenEvolve’s architecture against UCTP:** if baselines exceed pre-agreed lift, extend to processor-in-the-loop benchmarking.

Delivering this chapter’s plan means we’ll know whether OpenEvolve can accelerate UCT processing or not, and we’ll have a clear roadmap to scale it to real-time dynamic solutions that SDA needs in the future.

Chapter 5. Symbolic Regression

5.1 Introduction

Understanding from previous chapters of UCTP and OpenEvolve, we know that we need to use OpenEvolve's AI architecture to benchmark how well UCTP is performing and see if we can optimize workflows and pipelines. The idea of this section is to introduce the idea of symbolic regression. There are two main portions we are looking at in this topic, **(1) LLM-SRBench** which is a benchmarking framework for data driven scientific equation discovery. It integrates a pre-trained LLM with symbolic regression, evolutionary optimization, and parameter fitting, allowing it to infer underlying physical or mathematical relationships from raw data without prior knowledge of domain specific equations. **(2) AlgoTune** is a benchmarking tool for widely used mathematics, physics, and computer science functions. Its main goal is to try to answer if language models speed up general-purpose numerical programs. Alongside AlgoTuner which is an agent that enables language models to iteratively optimize code.

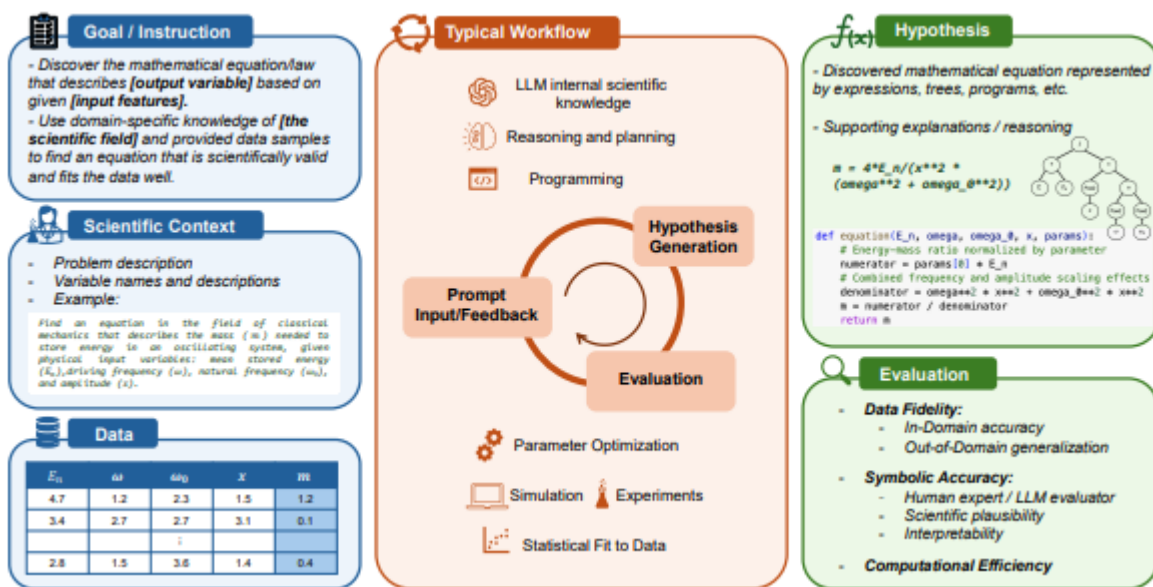


Figure 5.1.1: This figure showcases the overview of the LLM-based Scientific Equation Discovery used in LLM-SRBench. The blue (left side) section are benchmark tasks that combine scientific context with numerical data. The orange (middle) section is the discovery process that iteratively leverages LLM. The green (right side) section are the discovered hypotheses, represented as equation strings, trees, and programs.

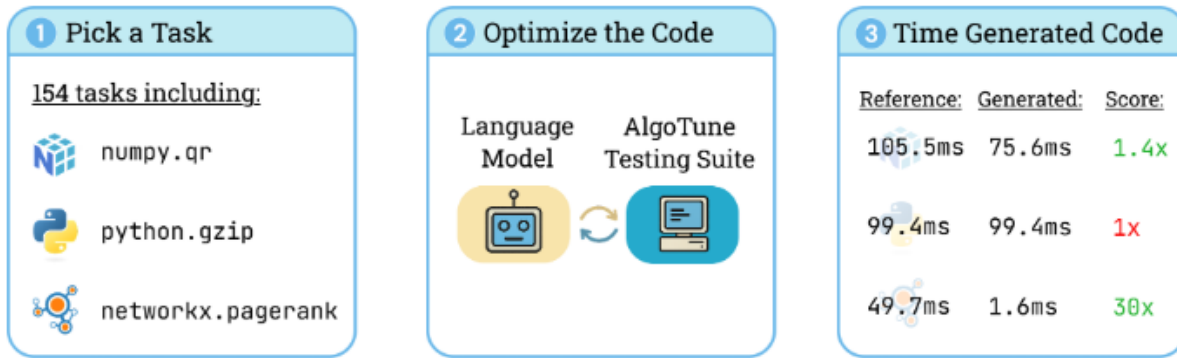


Figure 5.1.2: This figure showcases the overview of AlgoTuner agent that enables language models to iteratively optimize the code.

So why are these benchmarking tools important and how do they help us solve our problem with UCTP? To give a brief high-level explanation of these benchmarking tools:

LLM-SRBench: The idea of this science and research is that there is too much optimism around LLM based scientific equation discovery. When a lot of time they are memorizing famous formulas. This current benchmarking tool makes it impossible to memorize such equations when making discoveries on their own. Which is discovered that even with the best systems, it is still pretty bad. This benchmarking tool has 238 scientific domain problems (4 domains: Chemistry, Biology, Physics, Materials Science), and it splits it into two categories:

1. **LSR-Transform (111 problems):** Transforms known equations into **uncommon but equivalent mathematical forms**.
2. **LSR-Synth (128 problems):** Synthetic but realistic problems that mix known standard terms with novel, **synthetic terms that are unlikely to be memorized**.

AlgoTune: Is the same idea of LLM-SRBench. Where can a model write code that produces that same outputs as Python libraries used to solve scientific problems. But instead of focusing on scientific equation discovery like LLMSR-Bench, it focuses more on the performance optimization of numerical programs. Consequently, AlgoTune collects coding problems from real-world numerical libraries (NumPy, SciPy, scikit-learn, etc.). AlgoTune focuses on two things:

1. **Benchmarking (154 problems):** Numerical/ scientific programming tasks where the goal isn't just to get the **answer correct** but more of an emphasis on **how can we get the right answer the fastest?**

2. **AlgoTuner (agent):** A loop around an LLM **that manipulates, runs, and profiles code for the fastest accurate answers.**

We talk a lot about these benchmarking tools, but when does symbolic regression come to play and what is it? Symbolic regression is the basic idea of given data, can we find a formula that's generated it, and not just using a numerical fit. It is similar to how machine learning algorithms work, where we have a large sum amount of data and how can we make sense of that data through algorithms and patterns. So how is symbolic regression normal from regular regression? Regular or Ordinary Regression usually comes into a form of Linear Regression/Polynomial Regression where we have a form we can choose and then we fit the parameters based off of that. In Symbolic Regression we don't assume a form. The algorithm searches over possible structures and then fits the parameters based off of that.

So, understanding that, we can see how OpenEvolve and these benchmarking tools come to play. Where we can use genetic programming / evolutionary algorithms to start from random formulas and mutate / crossover them. The idea of these benchmarking tools is to showcase the existing problems to this solution. Where in most cases when doing classical symbolic regression, the process usually goes as follows:

- Scientists pick ground-truth equations from any scientific domains
- They simulate data from those equations
- Then that data gets fed into the algorithm and sees if they can rediscover the original equation.

In a pre-LLM world this is fine because there is no risk of **memorization**. We have to understand that LLMs aren't blank states. They are pretrained from the internet, papers, code, etc. So, what is the idea of symbolic regression?

- Benchmarks for symbolic regression define their **ground truth** using **already-known equations**
- With LLMs, those same benchmarks become **contaminated by pretrained data**
- We need to define why symbolic regression needs to **break memorization** to remove leakage and force methods to actually do data-driven discovery and reasoning and **not just recall**.

So, in the following topics, we can go into a deeper dive of symbolic regression. To really understand how it works and the process of why it is important to solve UCTP limitations.

5.2 Foundation

Goal: build a baseline equation discovery. A type of machine learning algorithm; predict y given x . Searches for a mathematical equation that best fits a given dataset. Instead of just learning numbers (like linear regression) it tries to discover both the form and parameters of the equation. Symbolic Regression algorithm generates random combinations of math operations, constants, and variables, evaluates fitness where each equation is tested on your dataset. So, the closer its prediction is to the higher its fitness score.

It then takes the crossover where two good equations are selected and the parts of their structure combining genetic material:

```
Parent 1:  $y = x^2 + 3$   
Parent 2:  $y = \sin(x)$   
Start with symbolic regression: discovering the analytical form of a  
function  
Offspring:  $y = \sin(x) + 3$   
on  $f(x) = y$ 
```

Figure 5.2.1: This figure showcases how we use symbolic regression to discover the equation of the function.

Represent equations as binary trees of mathematical operators (+, -, /, *, sin, etc). Apply genetic programming to evolve equations — mutate operators, and crossovers between good equations, and score each on dataset fitness. Fitness is measured with Mean Squared Error between the predicted and true outputs.

Maintaining interpretability: in machine learning means designing or choosing models that remain understandable to humans, so that we can explain why a model makes a certain prediction, not just what the prediction is.

5.3 Expected Results

Baseline SR achieves interpretability but limited exploration efficiency due to random search over a large equation space. LLM-SR Guided Equation Generation

Goal: Replace brute-force symbolic regression search with LLM-based hypothesis generation.

By using a pre-trained LLM ($\pi\theta$) we can generate equation program skeletons, such as:

```
def f(x, params):
    return params[0]*np.sin(x) + params[1]*x**2
```

Figure 5.3.1: This figure showcases how we use symbolic regression to discover the equation of the function.

F is the set of all the sampled equations. The notation $f \sim \pi\theta$ means each f is drawn from the model. This will continue to generate multiple functions and store into a dataset $D = \{(x_i, y_i)\}$. Then we use:

$$\text{Score}_T(f, D) = -\text{MSE}(f(x_i), y_i)$$

Which will keep the highest scoring equation and discard invalid or non-executable ones. Aims to maximize the reward $\text{Score}_T(f, D)$ for a given scientific problem T and dataset D :

$$f^* = \arg \max_f \mathbb{E}_{d \in D} [\text{Score}_T(f, D)]$$

After each equation is generated, they test how well it fits the data D . The score is a fitness measure (measured with negative mean squared error ($-\text{MSE}$) between the predicted and actual data points). Higher score = better match to real data. Trying to find the equation that gives the best fit to the dataset. It will also use **interpretation**, so if the prompt mentions "*oscillator*" it will generate (*sin/cos/exponential*) terms rather than random operators, which will drastically reduce the combinatorial search space compared to the symbolic regression approach. For a given scientific problem T and dataset D :

$$\text{Score} = -\text{MSE}(\hat{y}, y)$$

- T = the current problem (discovering the equation for an oscillator)
- D = the data for that problem (input/output pairs)

$$f^* = \arg \max_f \mathbb{E}_{d \in D} [\text{Score}_T(f, D)]$$

Finds the equation f^* that achieves the best average score across the dataset. E_d means average over all data points in D . $\arg \max_f$ the function f that gives the highest score. Overview: The LLM ($\pi\theta$) is repeatedly asked to generate possible equation templates and each generated equation is tested on the real data, scored based on how accurate it is, and the process repeats until it finds the equation f that best matches the data for that scientific problem.

Summary: Navigates through search space more efficiently and discovers more accurate equations with better out-of-domain generalization. Traditional symbolic regression uses **search heuristics** (genetic algorithms, reinforcement learning, or random sampling). These methods **do not know** physics, math, or domain logic, they just try operators until something fits the data. LLM-SR changes that by adding an LLM that already understands math structure, physics, and code syntax. It navigates the search space more efficiently. Traditional methods treat equation discovery as blind search, where each equation is a random combination of operators and constants, and they evaluate millions of times to find one good fit.

5.4 Hypothesis Generation

Goal: Leveraging a pre-trained LLM to automatically propose diverse and scientifically meaningful equation skeletons by acting as a hypothesis engine. It will propose new mathematical functions $def f(x, params)$: using context, problem descriptions, and evaluation instructions to produce an interpretable orbital model.

What the Prompt Needs:

- **Instruction:** Directs the LLM to complete the function so that it reflects physical meanings and relationships among input variables. Tells the LLM how to write the equation.
- **Problem Specification:** Gives context about what problem it's solving, includes key variables, constraints, and objectives. Example: "model an oscillator using position x_{pos} , y_{pos} , z_{pos} , x_{vel} , y_{vel} , z_{vel} ."
- **Evaluation and Optimization Function:** Explains how the generated equations will be judged (e.g., scored based on MSE). This helps it produce structures that are measurable and testable.
- **Experience Demonstration:** Gives examples of previous good equations, showing what a successful equation looks like and how it improved in past rounds. This lets the LLM learn from proper attempts without retraining.

5.5 Sampling and Validation Process

At each iteration t , the LLM-SR will generate a batch of candidate question skeletons for diverse hypothesis exploration by adjusting temperature:

- Higher temperature $\rightarrow$ more *creative*, **random outputs**.
- Lower temperature $\rightarrow$ safer, **deterministic outputs**.

Tune to get a good balance of new ideas and sensible ones. Sampled equation programs are executed, and those failing to execute or exceeding a maximum execution time threshold are discarded.

Once the LLM writes code, it gets tested. If it crashes (syntax error, undefined variable) or takes too long, it's thrown out. Only valid runnable equations move to the next stage where they'll be tested on data.

The LLM's in-context learning and crossover capabilities are employed to refine the suggested equation skeletons iteratively.

By representing equations as Python programs, we take advantage of the LLM's ability to generate structured, executable code while providing a flexible and effective way to represent general mathematical relations.

The program representation also facilitates direct and differentiable parameter optimization to better optimize the coefficients or constants in the generated equations.

5.6 Optimization and Scoring

Goal: To adjust the parameter vector, `params`, to minimize the prediction error using **Mean Squared Error loss function**.

- **NumPy and BFGS (`scipy.optimize`)**: A **nonlinear optimization** algorithm that uses gradients to find a local minima. It is effective for smooth, continuous functions and problems with relatively few parameters.
- **PyTorch + Adam (`torch.optim`)**: A **stochastic gradient-based optimizer** to handle large or noisy datasets.
- **BFGS (Broyden Fletcher Goldfarb Shannon)**: provides accurate deterministic fitting ideal for smaller problems with fewer parameters. Adam combines flexibility and scalability, which is good for noisy or high-dimensional data.

An **optimizer** uses gradient descent of a loss function to know which direction to move the parameters. “Stochastic” means random, in optimization, it refers to updating parameters using a random subset of the data rather than the entire dataset. **The randomness helps make training faster, escape local minima, and introduce exploration in optimization.**

After optimizing the skeleton parameters, we assess the fitness of the equation hypothesis by measuring its ability to capture underlying patterns in the data. We compute predicted target values such as:

$$\hat{Y} = f(x, \text{params}^*)$$

The **fitness evaluation score** s is then calculated as the **negative mean squared error** between predicted and true target values. Mean squared error is a loss we want to minimize — lower MSE means a better model, and the higher MSE means worse. But in search-based or reinforcement learning frameworks, we maximize a score (reward), not minimize a loss.

This is defined as:

$$\text{Score} = -\text{MSE}(\hat{y}, y)$$

This flips the direction of optimization ,—, if MSE is small (good), s becomes a large negative number (closer to zero, higher). If MSE is large (bad), s becomes a more negative number (lower).

5.7 Data Generation

Benchmarker flow chart



Figure 5.7.1: This figure showcases how UCTP generates data files to feed into the processor.

Test datasets are created based on the equation-based output (e.g., predicted orbital positions, velocities, or observed data) formatted according to the Unified Data Library schema to ensure compatibility with the UCTP processor.

- **The I/O:** This is generated and transferred into the UCTP benchmark system through standardized I/O operations. This is where we will store the JSON structure that contains observational and reference information to be scored accurately.
- **Scoring:** Compare with the truth dataset and compute performance metrics to quantify how well the LLM-SR generated equation explains or predicts observed data.
- **Packaging:** Export as a JSON and summarize into a report for detailed analysis. This validates the LLM-SR's discovered equations against the UCTP standardized benchmarking framework.

Plausible Orbit Example from UCT:

- Uses the **observations** from the **dataset_obs**
- Produces a **6D** state vector that makes sense (**position** (physically possible)) and **velocity** (physically realistic))
- The estimated orbit must have a realistic **speed** for a satellite if it's in **Earth's orbit**.
- How far is the **candidate** (*predicted*) **orbit** from the **reference** (*true*) **orbit** (in all 6 dimensions)
- Fits the noise measurement

To connect the sparse/noisy observations into one consistent orbit

- When you have a bunch of observations that are sparse (not many, maybe far apart in time) or noisy (each one is a bit wrong) and you want one consistent orbit (one 6D state + covariance) that explains them all.
 - The system will do this in two main steps:
 - **Association**: decide which observation belongs to the same object, how will it do this? It'll look at the time, angles, and ranges. Motion must look like an orbit. Group observations that could be explained by one physically plausible trajectory. This will build “tracks” or “observations” groups where each group is treated as one object history. **groupedObsIds** in the dataset is the answer key for which observations belong to which true object.
 - **Estimation**: fit one orbit into those observations, despite noise and gaps. What Orbit will best explain this group? Well once a group has a group of observations $\{z_1, z_2, \dots, z_n\}$ believed to be from the same object it starts with an initial guess of the orbit (a rough 6D state + epoch). Propagates that orbit forward/backward in time to each observation time. For each observation, it computes a residual (error) ($Residual_{error} = z_i^{observed} - z_i^{predicted\ from\ orbit}$). It will then adjust the orbit to reduce the residuals overall (Least-squares/Kalman). Basically, finds all the 6D state that when propagated comes closest to all the observed points, even if there are few points (sparse), each point has error (noise). Sparse will use the model (orbit equations) to bridge the gaps in time. Noisy will use statistics (least-squares, covariance; to average out the errors)

5.8 Testing and Evaluation

Each team member completed evaluation logs from running the symbolic regression benchmark for the physical oscillator problem. This evaluation script tests the generated symbolic equation within `initial program.py` against the ground-truth dataset (`X train for eval.npy` and `y train for eval.npy`) to assess how well the discovered function models the target dynamics. In terms of what the model expected:

- 3 input variables
- 10 tunable parameters to be optimized during evaluation
- Loading input and output training data

Evaluation Results:

- **Combined score:** 1.6927 → this is the overall metric combining accuracy and interpretability, where higher values represent better performance.
- **Negative mse:** -0.0203 → this number represents the negative MSE between predicted and true outputs; since it's closer to zero, it indicates higher accuracy.

Overview: Since we generated predictions with a low error rate, we can conclude that the model successfully suggests the discovered equation is both executable and reasonably accurate for this oscillator dataset. The evaluation module has correctly loaded, validated, and scored the generated equation structure using the defined scoring.

Dataset	Time range	initial values	F	α	β	δ	γ	ω
Oscillator 1	(0, 50)	$\{x=0.5, v=0.5\}$	0.8	0.5	0.2	-	0.5	1.0
Oscillator 2	(0, 50)	$\{x=0.5, v=0.5\}$	0.3	0.5	1.0	5.0	0.5	1.0

Table 2: Parameter values for Oscillator datasets.

Oscillator 1:

$$\dot{v} = F \sin(\omega x) - \alpha v^3 - \beta x^3 - \gamma x \cdot v - x \cos(x)$$

Oscillator 2:

$$\dot{v} = F \sin(\omega t) - \alpha v^3 - \beta x \cdot v - \delta x \cdot \exp(\gamma x)$$

Figure 5.8.2: This figure showcases the parameter values for oscillator datasets.

5.9 Nonlinear Oscillator Equations

Nonlinear Oscillators are chosen to test whether the model can discover true physical relationships when evaluating and not just memorizing equations. Nonlinear equations can also involve non-convex optimization challenges and chaotic behaviors forcing the LLM-SR to reason symbolically about the equation under the hood. This is an important benchmark stress test, ensuring the algorithm can consistently discover accurate and interpretable equations, even when faced with highly complex or unstable dynamics.

5.10 Noise Robustness Analysis

To evaluate whether imperfect data conditions still work when dealing with real world settings we can conduct systematic analysis of model performance under varying levels of noise. To test this, we will use controlled Gaussian noise with different standard deviations ($\sigma = \{0, 0.01, 0.05, 0.1\}$) to the **training data**. This analysis provides insight into how the incorporation of prior knowledge through LLM-SR affects equation discovery **performance** when dealing with **noisy measurements**.

For Example: Bell Curve.

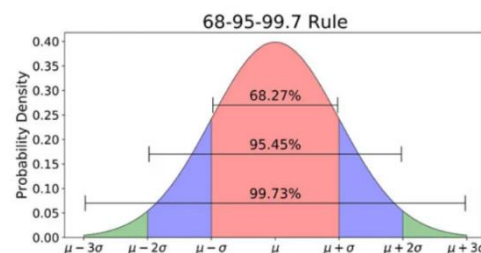


Figure 5.10.1: This figure highlights how data is spread around the mean (μ) based on the standard deviation (σ).

Each region represents how much of the data falls within certain ranges. The **red** region (central maximum) (**68%**) means that most measurements stay close to the mean. The **blue** region (**95.45%**) of all data points that come from this distribution. And the **green** region (**99.7%**) is the extremely rare distribution and rarely any points fall here. The height of the curve shows **probability density** (area under the curve corresponds to probability), so how likely values around that point are. Values near the

mean (the center) are very likely and the sides of the chart have a low probability density, so extreme values are unlikely. We don't know exactly how far away an object is, we want to define it as most likely a certain amount of distance away with a relative uncertainty of such and such amount. We want relative confidence that this thing is within an area of where we think it is. We are trying to encapsulate that with probability distribution. As we move further away there is a smaller chance that the object is further away than what you originally thought. The high standard deviation we would have a wider distribution. Skinny function (**small variance**), short function (**large variance**), so as we get further away there is still a high chance that the object is much further away than what we thought, whereas a small covariance there is a smaller chance that the object is further away than originally thought.

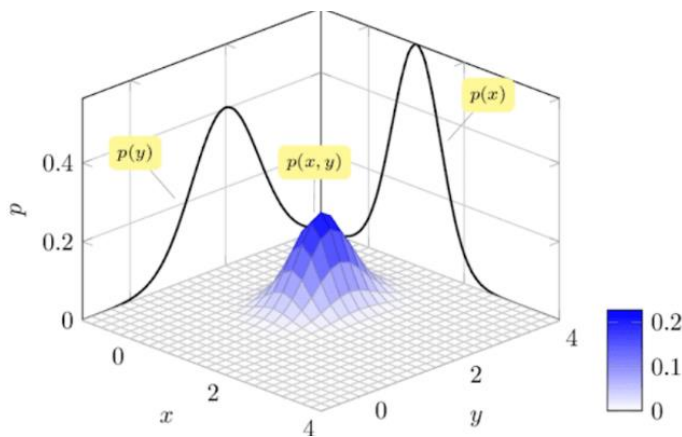
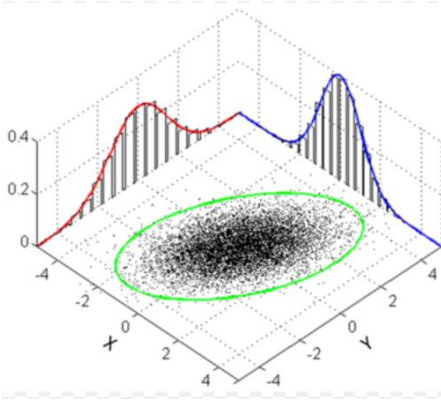


Figure 5.10.2: This figure showcases how in UCTP we are working with 3-dimensional data which is then translated into 6-dimensional data when accounting for covariance with positioning and trajectory.

When we move to 2 dimensions, if we have an x and y component, we are now trying to find something in a 2-dimensional space instead of just how far it is along one line. In 1-dimension, we only asked, “how far is it?” but in 2-dimensions we now ask, “how far, and how far to the left or right?”. We can have the same probability distribution of the x-component and y-component so we will need **two individual variances**. There is also the possibility that if x and y are **correlated** (not completely independent) if every time Y is positive x is also positive (and the same if negative) in addition to variances we also need **cross-variance**. These values define where the **mean of the probability distribution is in 2-dimensional space**, and the other **three pieces of information** (variance in x, variance in y, covariance of x–y) and package that into a 2x2 matrix which is now called the **covariance matrix**. This is going to tell us the same information of the 1-dimension (how uncertain are we about where this thing actually is relative to where we thought it is), now we are using more information like how uncertain are we in each direction and also how uncertain are we if its slightly in **front** then it's also off to the **left** whereas if its **behind** than its to the **right**. There is **less** of a probability that if it's behind it will be to the left. This is what covariance is telling us.

In 6-dimensions where we have 3 positions and 3 velocities, now we have a 6x6 covariance matrix, that's telling us about the variance in each one of those 6 components and telling us how each one of those components might be related to each one of the other components. So, using this information we can fully define the probability of the distribution of where we think the object will most likely be. But also, what is the possibility that the object is slightly displaced from where we thought it was.



What we should be thinking about is what other questions we will need to ask to start thinking in the 6-dimensional state.

So, if we took thousands different measurements of something, they are not going to be the exact same value (just about the same with small differences) there will be random fluctuations in our detector (noise or random effects) so we get a probability distribution. If we get a bunch of measurements and try to measure the same thing a whole bunch of times, we will not get

the same thing every time because of noise or some other form of uncertainty.

Figure 5.10.3: This figure showcases the complexity of working with 3-dimensional data alongside all of the noise and random effects we also generate. How well can UCTP and OpenEvolve handle these noises?

Measuring the same thing a whole bunch of times, we will not get the same thing every time because of noise or some other form of uncertainty. If we get something that looks like this, we can define all of these data points by probability distribution. If we were to make one more measurement of whatever we are measuring, where will it be? We won't know exactly where it is, but with relative uncertainty we can say the most likely place it will be, we take the average of all of them. The average distribution will be the most likely spot for it to be. The covariance will be telling us where it will be but how likely it will be a little off from that spot. If we were to make what is the relative probability that it will be somewhere.

There will be some spread in the x-direction and the y-direction, which are the two variances. There is also the possibility that x and y are correlated. For example, if it is more likely that x and y are **both positive** at the **same time** than it is for **x to be positive** while **y is negative**, then **x and y have a correlation**. So, in addition to needing the variance in the x-direction and the variance in the y-direction,

we also need the covariance between the x and y components, such as x-y component, x-z component, and z-y component. Covariance tells us how the two variables vary together, as opposed to how each varies individually. The same idea extends to higher dimensions. In 3D, we can have spread in the x-direction, y-direction, and z-direction, but we also have covariances for the x-y component, x-z component, and y-z component. As the number of dimensions increases, the covariance must capture all the pairwise relationships. The covariance describes how wide or narrow the distribution is, and how far the probability distribution extends before the probability drops off. To fully define all this uncertainty information, we need a matrix whose size depends on the dimensionality of the state vector we are working with. For a 2-dimensional vector, the covariance is a 2x2 matrix; for a 3-dimensional vector, it is a 3x3 matrix; and for a 6-dimensional orbit state vector, the covariance is a 6x6 matrix. So essentially, the covariance tells us not just how each dimension varies on its own, but how different dimensions vary together, and the full covariance matrix grows with the number of dimensions in the state.

The **orbit** is most likely **near** a particular state but with **some probability of being in nearby orbits**. Because of this **uncertainty**, the UCT processor **output** should **give both a mean and covariance for each candidate orbit**, and the **reference dataset** should also **provide a mean and covariance** for each input **reference object**. In addition to comparing only the mean state vectors, we can use the covariance to determine not just the difference in the means but also a statistical difference between the two probability distributions represented by their mean and covariance. The 6 parameters of the state vector completely define the orbit we are trying to estimate. The **covariance** for this 6-dimensional position and velocity vector is a 6x6 matrix that defines how **uncertain** we are about that estimated position and velocity; The covariance essentially represents the spread of where the object might be. The 6-dimensional position and velocity vector is the center of the distribution (the mean) and the covariance defines the region of highest probability around that mean. By specifying both the mean and covariance instead of just the mean alone, the UCT processor can represent that the orbit is most likely near the estimated state, with some probability of being in nearby orbits. So, each UCT processor output should include a mean and covariance for every candidate, and the reference dataset should include a mean and covariance for each reference object. This allows us to compute not only the difference in mean state vectors but also a statistical difference between the two probability distributions described by their mean and covariance.

Covariance has a central maximum and drops off on both sides, and the mean of the distribution is the average of every data point underneath that bell curve. The standard deviation (how far apart the gaps are) is related to the width of that bell curve. The standard deviation is the square root of the variance, 1D probability distribution. How confident are we that we

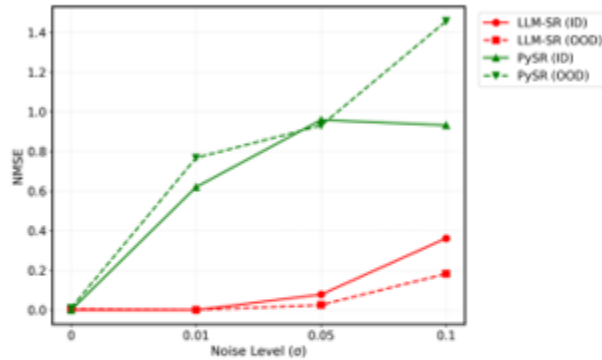


Figure 5.10.4: This plot analysis compares both LLM-SR and PySR on the Oscillation 2 benchmark. This shows normalized mean squared error for both in-domain (testing on data that comes from the same distribution as the training data) and out-of-domain (model is tested on data that is different from the training distribution) predictions under different levels of Gaussian noise. LLM-SR stays more robust out-of-domain, meaning that the test data is different from what it's trained on, and it still works well.

Where PySR's errors **increase** with noise, PySR gets much **worse** when the data is **noisier** than what it was **trained** on. While all methods degrade in performance when noise increases LLM-SR exhibits better resilience compared to traditional approaches. This is because it uses physics/knowledge built into the prompts, this gives it physical intuition and helps it avoid nonsense equations. Therefore, it doesn't get confused when the data is noisy. The domain knowledge limits the model to only search for realistic and physically meaningful equations. Focuses on what makes sense in your domain. So even with noisy data it finds good solutions. Which is why it's robust. When the data gets worse the model will use prior knowledge and outperform those that don't. So, with good data(clean), the model can **rely** mostly on the **data itself**. But with bad data (noisy or unreliable), the model must **rely** more on **prior knowledge**.

5.11 LLM-Based Optimization Baselines

There have been attempts to implement symbolic regression as an experimental task, as proof of concept. But their implementation **lacked** domain-specific knowledge, generating complete

equations with LLM instead of optimizable skeletons, uses older LLM models, and lacks several design elements present in LLM-SR, such as equation-as-program representation and **multi-island dynamic memory management** (which is how we're implementing it) for diverse exploration.

Model	Oscillation 1		Oscillation 2	
	ID↓	OOD↓	ID↓	OOD↓
DSR	0.0087	0.2454	0.0580	0.1945
uDSR	0.0003	0.0007	0.0032	0.0015
PySR	0.0009	0.3106	0.0002	0.0098
LMX	0.5031	48.93	1.004	0.9371
FunSearch	0.4840	8.059	0.7234	0.5861
LLM-SR (w/o skeleton+optimizer)	0.1371	0.6764	0.3780	0.3751
LLM-SR (w/o Prior)	0.0001	0.0029	4.65e-5	0.0071
LLM-SR (w/o Program)	0.0001	0.0035	3.22e-5	0.0043
LLM-SR	4.65e-7	0.0005	2.12e-7	3.81e-5

Figure 5.11.1: This table showcases the different benchmarks in different models in regards to oscillations.

These results compare the different optimization approaches on two oscillator benchmarks. The metric they use to compare is NMSE (lower is better). As you can see, the traditional methods (DSR, uDSR, PySR) do okay on in-domain data but bad on out-of-domain data. LMX and FunSearch (LLM-based algorithmic search methods) both have poor performance, high errors. But LLM-SR is 10,000 times better than PySR, 1,000,00 times better than DSR, and about 1,000,000,000 times better than LMX.

Also noting that removing the skeleton code hurts the LLM-SR model drastically compared to just removing the prior knowledge or equation-as-program representation.

5.12 Summary

To fully grasp everything in this chapter, we started by understanding the OpenEvolve AI architecture and why Symbolic Regression is needed. We now know that to test whether or not the model is evolving and optimizing equation discovery so that we are then able to benchmark the UCTP to see if it produced better results. Our main goal is to ensure that the model is discovering the correct equations (e.g. mathematical/physics/computer science) and not memorizing famous formulas but instead building up ones for the best optimization based on the given code. We also introduced AlgoTune which is the engine of the entire LLM-SRBench, it's handling the performance optimization of numerical programs. But now we want to understand what Symbolic Regression is. The idea is that we are able to

give it data and with little information it is able to discover the equation that generated that data. They define their ground truth using already-known equations. A scoring function is used to keep track of the highest scoring equation. After each equation is generated, they test how well it fits with the dataset. The score is a fitness measure, higher score = better match to real data. Prompting becomes extremely important here since we want to be specific on how the model discovers these equations without creating anything useless. The prompt must include instruction, problem specification, and evaluation and optimization function to propose meaningful skeletons. Adjusting the temperature allows us to get a mix of creativity and deterministic outputs for new ideas. We want to train on random sets of data to make training faster, escape local minima, and introduce exploration in optimization by evaluating the score and calculating the negative mean squared error between the predicted and true target so that we reward the maximized score. Then by breaking down what we are receiving from the JSON file and understanding how to use that information is crucial for the next steps of this project as we start to think about how it'll be developed. We want to define and understand what kind of data the satellites are returning and what will be important enough to train the model and not confuse it with useless information. We also want to make sure that we are able to compile, test, and output information on sampled data from the OpenEvolve GitHub. This will help us understand any errors early on rather than dealing with them when it gets more serious. We also get a better idea of what OpenEvolve's processes are and how we can use that to implement in our own architecture. We understand that we need performance and accuracy when testing out the model's evaluation results. The results are the error rates. Nonlinear oscillators are mentioned to ensure that the model is able to discover and evolve complex equations and stress test it for future high complex and dynamic systems. Then we move to bell curve and 6-dimensional implementation. First, we start with linear just to get a foundational understanding of what we should be asking ourselves as we move up in dimensions. From what we gather we are using position and velocity to try and find an object's specific point in time along with its orbit. This is quite difficult, so therefore, the concept of covariance, mean, and standard deviation are introduced to explain that even if the position found isn't exact, we can at least classify it with ranges and probabilities. As we moved to 2-dimensions we have to think about different movements (right/left) and distance (front/behind) and use that information to build off from 1-dimensional, which is why cross-variance is introduced, to grasp the idea of matrixes. It becomes harder to really know what questions you have to ask and what you need to think about when you are looking for something in a higher dimension. Those questions and ideas will have to be written down and researched. Also included information on controlling noise level and how that affects the LLM-SR and that it performs very well and

is more robust when tested on data that is different from the training distribution. Lastly, include a few charts to show the high-performance rate LLM-SR has against other models

Chapter 6. Graphical User Interface Development (GUI)

6.1 Introduction

The analytic console is created in order to be able to keep track of OpenEvolve's mutation to UCTP algorithms in real time. This analytic console also allows researchers to have an all-in-one dashboard where they are able to pull UCTP data from the output files and into OpenEvolve AI architecture. The user is also able to prompt a chatbot created within the dashboard to automatically generate evaluation metric codes from what the user specifies it to. This large language model is trained under OpenEvolve's example problems that have been run and tested already. This allows a starting point for that chat bot to train on. Which is then also trained under the researchers using it in the future. So overtime the model gets better and more efficient at generating these evaluation files needed for the OpenEvolve Architecture.

6.2 OpenEvolve Architecture Versus Traditional Stacks

OpenEvolve already has a graphical user interface to showcase how the algorithms are evolving and so we are following that architecture and the tech stack to use so. The difference here is that instead of creating a web application where we need to host a server for a database. Everything will be held within the directory of the files being created and manipulated from each OpenEvolve iteration. The current tech stack that was identified from going over the visualizer python file embedded in OpenEvolve files are:

OpenEvolve Visualizer.py tech stack:

- JSON data creation and retrieval
- Python + Flask
- HTML/CSS/JSS

Environment:

- Real-time API scans per checkpoint
- All local based data not using a server since the High-Performance Computing cluster itself will be storing everything

The main problem of OpenEvolve's architecture is the idea that we can't have a consistency of how the metrics are per algorithm. What this means is that each different problem in OpenEvolve has their own schemas on the metrics that they use to evaluate how efficient OpenEvolve is doing. An example of this is that our sponsor wants a real-time line graph to show how OpenEvolve is doing so they can check up on the status of it and also identify when to stop the iterations. Since in some cases OpenEvolve finds the optimized code and solution fairly quickly even though you set the iteration bound to 100.

The next two figures will show how the different algorithms have different metrics. Talking to the sponsor, they wanted to be able to use this GUI in the future for other problems as well and one thing that was very beneficial from research that we gathered is the fact that we can make our dashboard dynamic. What we plan to do is we will loop through the metrics to find what is being evaluated to whatever algorithm we are using and then create the dashboard graphs from that. And so, our website will have dynamic sizing based off how many metrics are being used. The user will also have the ability to filter certain graphs(metrics) to their liking if they want to just focus on certain portions of the dashboard.

So how these are specified are:

Diversity and **Complexity** is provided per iteration, so we will just need to create a time series line chart to show the efficiency changes per iterations.

Some cases to look at are the **feature_stats** and the **combined_score** parametrics provided in the **metadata.json**.

- **value_score**: how close we got to the minima in function_minimization. (1.0 best possible result)
- **distance_score**: how far solution is in coordinate space from true optimum. (1.0 exactly optimum)

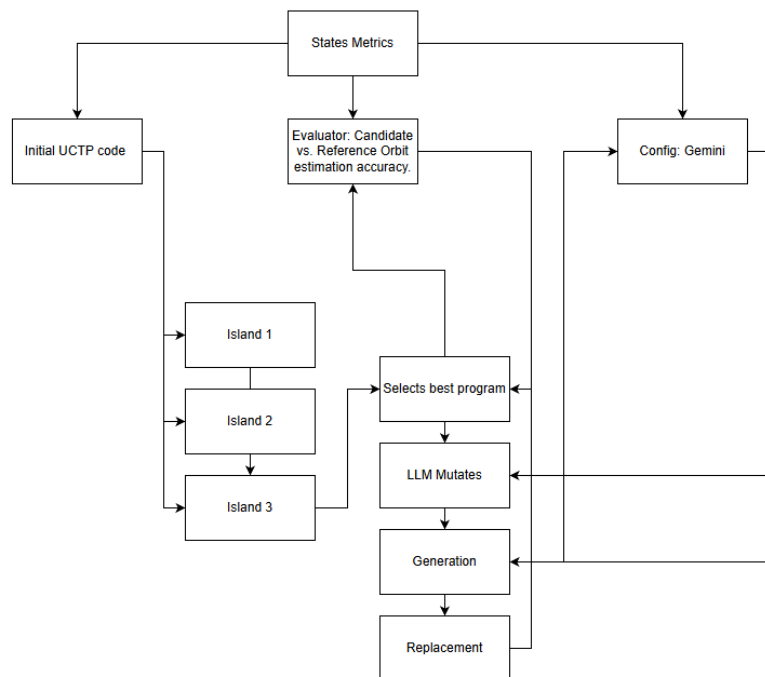
- **combined_score:** overall fitness combining value and distance.
(combined_score=value_score+distance_score)
- **reliability_score:** consistency and robustness (1.0 all inputs tested are stable no failed states)

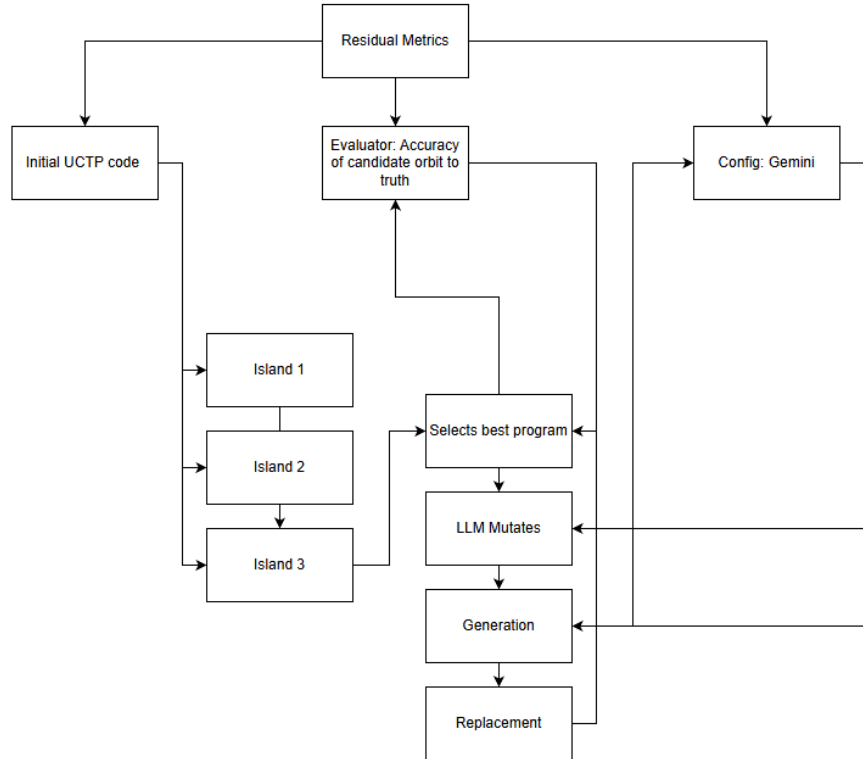
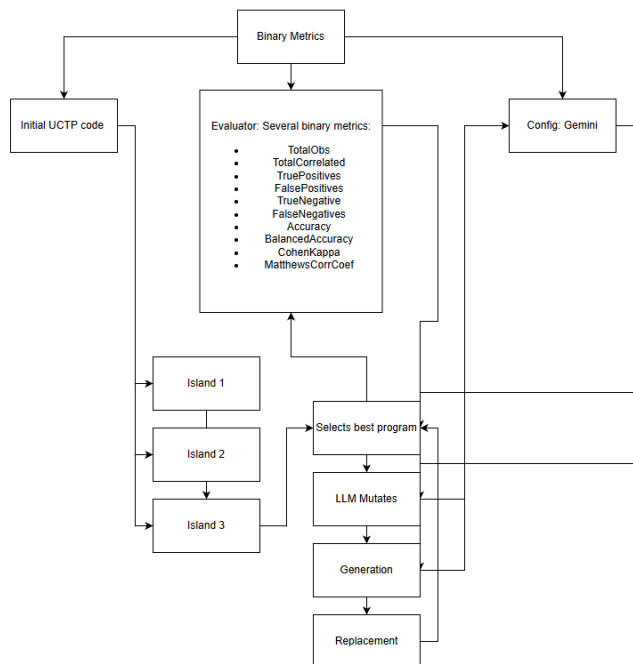
In the Metadata json file, this is how we will showcase in the dashboard of the dynamically created graphs based off each metric. The overall performance graph will be combined_score.

6.3 Analytic Console Integration

So now that we understand how OpenEvolve structures their evaluation, we can now try to integrate that with UCTP evaluation techniques. We are solely basing our evaluation file that we are developing in OpenEvolve based off of UCTP evaluation metrics as stated in Chapter 3. The following are UML diagram showcasing how we implement UCTP evaluation metrics into OpenEvolve's AI architecture.

State Metrics:



Residual Metrics:**Binary Calculation Metrics:**

These UML diagrams showcase how each metric will be an island in OpenEvolve’s evolutionary process. Where is island has it’s own evolutionary loop of selecting the best program, mutating through LLM, generating a new and improved program, and replacing the worst.

Understanding OpenEvolve’s architecture, the Analytic Console will live within the directory of where you’re running OpenEvolve in. We don’t need a database like normal technology stacks because running OpenEvolve will be done through MITRE’s HPC. OpenEvolve also creates their own files in the directory that it is being run through, so in order to get real time updates, we will need to create a script that automatically checks the meta-JSON file in each iteration. This can be seen in figure 6.31 which shows the whole workflow of the Analytic Console.

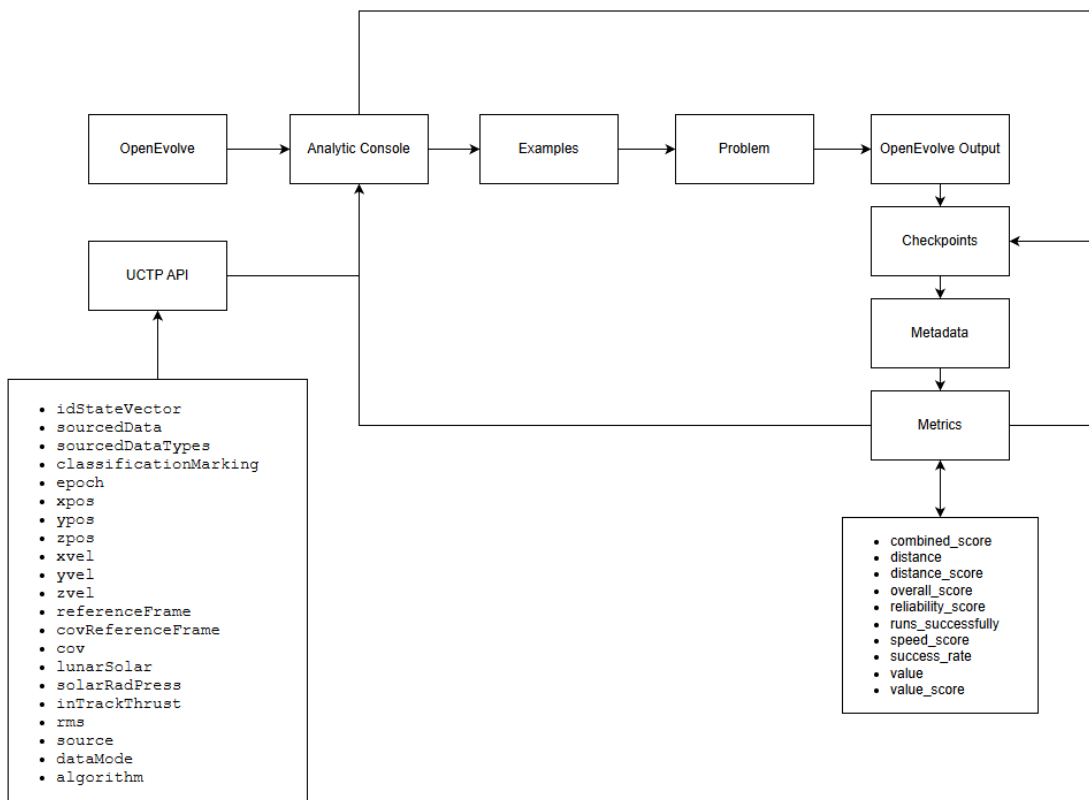


Figure 6.3.1: This figure showcases the flow of how the Analytic Console updates data from OpenEvolve in real time. The current Analytic Console doesn’t just do real time updates from OpenEvolve but also takes in data from UCTP as well so researchers have an all-in-one dashboard that showcases both sides of the problem.

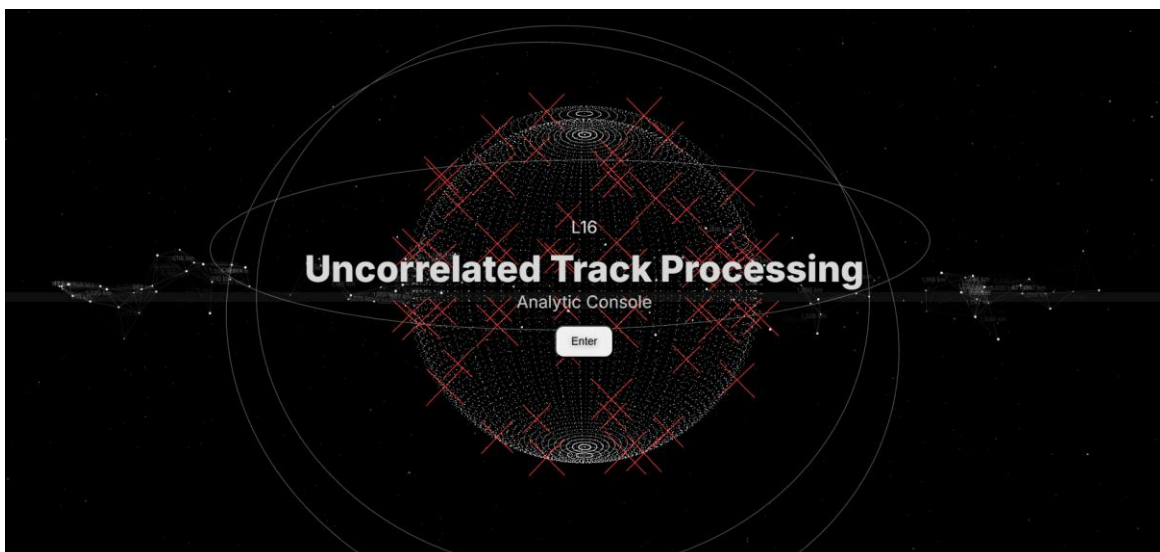
6.4 Analytic Console Development

Currently, the Analytics Console has gone through two design iterations. We began with a simple dashboard only layout, but for usability and long-term vision the team added a dedicated home page and expanded the dashboard to display UCTP data. The graphical user interface design is now largely mature; the remaining work is to connect it to OpenEvolve so it can pull live data and validate the dashboard's logic. Progress has paused for the moment, as development efforts are focused on integrating UCTP into OpenEvolve to enable meaningful end-to-end testing.

Design iteration no.1:



Final design iteration no.1 (Home Screen):



Final design iteration no.2 (Dashboard):



6.5 Future Development

Since the Analytics Console is ahead of schedule in terms of core functionality, the development team is taking the opportunity to incorporate more real world software engineering practices into the project. In particular, we plan to introduce Continuous Integration/Continuous Deployment (CI/CD) pipelines for the graphical user interface. These pipelines, implemented through GitHub, will run automated unit and integration tests on both the API and GUI to quickly detect whether any new changes break existing functionality or degrade performance.

These efforts are currently considered stretch goals, but they align with our long-term vision of making the Console maintainable and production ready. In addition, we aim to experiment with environment testing using containerized development setups, potentially orchestrated with Kubernetes. This would allow team members—and eventually sponsors—to spin up consistent, isolated environments for development and testing, improving reproducibility and easing future handoffs or scaling.

6.6 OpenOnDemand

OpenOnDemand provides a web-based way for us to access MITRE's supercomputers (HPC resources). One of the main challenges in using these systems is maintaining consistency across all the software environments—things like Python versions, installed packages, and file layouts. To address this, our GitHub repository serves as a centralized development space for both the GUI and the LLM components. While each researcher still has their own local OpenEvolve setup for experimentation and documentation, the shared repository ensures that the code we intend to run on the HPC is developed against a common, agreed-upon structure.

A major blocker for the project right now is obtaining and working within MITRE's HPC environment, which has strict licensing and software constraints. As a result, we can continue developing and testing the GUI on our personal machines, but we cannot yet guarantee that everything will behave identically on their systems. To close this gap, Dr. Cline (Senior Researcher at MITRE and SDA TAP Lab) and the team are exploring the use of Docker. By containerizing our environment, we can match the specific software stack allowed on the HPC and test our code in a controlled, reproducible way. The goal is to make the transition from our GitHub-based development to running on MITRE's HPC as seamless as possible, like running example problems locally from the command line, but now encapsulated within a Docker workflow.

Chapter 7. Large Language Model Development (LLM)

7.1 Introduction

As computational research environments grow increasingly complex, the tools that support them must evolve to meet the demands of both expert and novice users. One of the most notable shifts in recent years is the emergence of Large Language Models (LLMs), which offer natural-language interfaces capable of assisting with tasks that traditionally required advanced programming knowledge, theoretical background, or extensive documentation review. These models have introduced new possibilities for human–computer interaction, reducing barriers that previously limited engagement with specialized systems. Within this broader technological landscape, the open-source project OpenEvolve stands out as a framework that enables experimentation with evolutionary algorithms in a way similar to Google’s AlphaEvolve. However, despite OpenEvolve’s accessibility and flexibility, many find it difficult to write a functional evaluator.py.

An evaluator script acts as the scoring mechanism in the evolutionary process. It encapsulates problem definitions, scoring criteria, performance thresholds, and structural requirements that allow agents to understand what constitutes improvement. Although experienced programmers may find the evaluator design process straightforward, new users often encounter conceptual and technical obstacles. They may misunderstand the logic behind fitness scoring, misconfigure the agent-environment interface, or overlook essential structural components such as initialization routines or evaluation loops. These errors can lead to incorrect results, model stagnation, or complete task failure. For many potential users, these difficulties become a major source of frustration and may discourage further experimentation.

To address these challenges, the website incorporates a specialized Large Language Model based on a lightweight variant of Google’s Gemini architecture. Unlike general-purpose chatbots, this model is designed specifically to assist with generating, explaining, and refining evaluator scripts for OpenEvolve. The system is calibrated to operate efficiently, minimizing computational cost while still providing high-quality, domain-specific assistance. This chapter examines the role of the LLM within the website, explaining how it supports evaluator creation, enhances user understanding, and ensures that OpenEvolve becomes accessible to a broader range of researchers and students.

Through this comprehensive analysis, the chapter demonstrates how even a small, inexpensive LLM—when trained and integrated effectively—can play a transformative role in expanding the usability and reach of a complex computational research framework. By providing contextual explanations, generating functional code templates, and guiding users through conceptual difficulties, the LLM not only improves workflow efficiency but also contributes to the democratization of advanced algorithmic experimentation.

7.2 Purpose of the Large Language Model

The purpose of the LLM within our website is multifaceted, centering on the generation, clarification, and refinement of evaluator scripts for OpenEvolve. At its core, the LLM functions as an intelligent assistant designed to bridge the gap between user intent and technical implementation. The evaluator is one of the most important components in an evolutionary algorithm framework because it establishes the rules by which agents measure progress and determine success. Misunderstanding or incorrectly designing an evaluator results in flawed evolutionary dynamics, ineffective optimization, or ambiguous performance signals. Therefore, ensuring that users can build accurate evaluators is critical for meaningful experimentation.

One purpose of the LLM is to translate natural-language descriptions of problems into functional evaluator code. Many users prefer to describe their tasks in informal terms rather than in strict programming syntax. The model analyzes these descriptions and identifies key components, such as input formats, expected outputs, scoring mechanisms, and problem constraints. Which generates code that aligns with OpenEvolve’s structural requirements. This functionality reduces the time required for evaluator creation and lowers the entry barrier for individuals without deep programming expertise.

Another option is when the user uploads an `initial_program.py` file, the AI system analyzes the structure, functions, classes, and expected program behavior using code-understanding capabilities built into the LLM. The model parses the file to identify key inputs, outputs, and any constraints or error-handling requirements. Based on this analysis, the AI generates an `evaluator.py` script that can automatically validate the program’s correctness by constructing relevant test cases, executing the functions defined in `initial_program.py`, and comparing the produced outputs to the expected results. Because the AI uses semantic code comprehension rather than simple pattern matching, it can adapt the generated evaluator to different coding styles and program architectures, ensuring that the evaluation logic aligns with the

intention of the uploaded code. This allows the website to dynamically create tailored evaluators for any assignment or custom program a user provides.

Another central purpose is to explain the underlying concepts that govern evaluator design. Users frequently struggle with questions such as:

- How should fitness be calculated?
- What constitutes an “optimal solution”?
- How should reward shaping be handled?
- What role does the evaluator play in guiding evolutionary behavior?

The LLM provides detailed, context-appropriate explanations that enhance user understanding. This educational component is particularly important for new users who may not grasp the principles of evolutionary algorithms or the rationale behind OpenEvolve’s design. By presenting concepts in accessible language, the model promotes deeper conceptual clarity and encourages more informed experimentation.

Additionally, the LLM serves as a debugging and error-correction tool. Users often create evaluators with missing methods, incorrect imports, uninitialized variables, or logical inconsistencies. The LLM can analyze user-submitted code, identify errors, and provide corrected versions. This feedback loop helps users learn from mistakes while ensuring that evaluators remain compatible with the OpenEvolve framework.

Finally, the LLM provides iterative refinement capabilities. Users may request modifications to generated scripts, ask for alternative scoring strategies, or request more detailed explanations of evaluator components. The model supports these iterative adjustments, enabling a flexible and dynamic workflow that aligns with research and experimentation needs.

Overall, the purpose of the LLM is not merely to generate code but to function as a comprehensive, accessible, and intelligent resource that supports every stage of evaluator creation. By doing so, it contributes significantly to the accessibility, effectiveness, and educational value of OpenEvolve.

7.3 System Architecture

The system architecture responsible for delivering the LLM functionality is designed with modularity, efficiency, and reliability in mind. Although the model at this time is based on Google's Gemini architecture, its deployment is intentionally lightweight to minimize computational costs. This decision reflects the project's goal of maximizing accessibility and ensuring that the website can serve users without requiring powerful hardware or expensive cloud resources.

Front-End Interface

The front-end interface provides the environment through which users interact with the LLM. It includes a text input field, output display area, and optional code upload feature that allows users to submit partial evaluator and initial program scripts for review. The interface is designed to be intuitive and minimalistic so that users can focus on describing their evaluator needs without navigating complicated menus. It also supports dynamic context display, enabling the system to retain relevant conversation history so that follow-up queries remain informed and coherent.

Backend Application Server

The backend application server functions as the coordinating layer that manages communication between the front-end and the model. This layer handles several critical tasks, including:

- **Session management** – ensuring that user conversations maintain context
- **Prompt preprocessing** – augmenting user requests with relevant metadata
- **Rate limiting** – ensuring fair resource usage
- **Post-processing** – cleaning model outputs for readability and consistency
- **Verification routines** – checking evaluator completeness and structural correctness

This backend layer ensures that the LLM receives inputs in a structured format that maximizes accuracy while also guaranteeing that outputs adhere to OpenEvolve conventions. For example, when the model generates an evaluator, the backend verifies that it includes required components such as the `evaluate ()` function, proper initialization, and necessary imports. If any components are missing, the server automatically triggers a regeneration request before returning the output to the user.

Gemini Inference Layer

At the core of the architecture lies the Gemini inference engine. This component processes user prompts and generates evaluator code or explanations. The model is optimized for both efficiency and accuracy, allowing it to provide domain-specific output without requiring the computational resources associated with larger LLMs. The inference layer operates with constrained decoding strategies that reduce hallucinations and ensure adherence to OpenEvolve’s structural conventions.

To support consistency, the inference layer incorporates parameter settings such as reduced temperature, controlled beam search, and weighted scoring functions that promote deterministic and reliable code generation. These constraints help ensure that the model prioritizes factual correctness and structural validity over creative variation.

Verification and Reliability Mechanisms

One notable feature of architecture is the emphasis on reliability through verification. Since evaluator scripts must follow strict structural guidelines to function correctly, the system includes automated checks that verify the presence of essential methods, proper indentation, Python syntax validity, and compatibility with example templates. This ensures that even when the model produces imperfect output, the system can catch errors and request improved generations.

Architecture Summary

By integrating these layers into a cohesive system, the overall architecture provides a robust environment for evaluator generation. The combination of lightweight inference, structured verification, and user-friendly interaction yields a system that is both powerful and accessible. This architecture ensures that users receive high-quality support without requiring advanced technical knowledge or specialized computational resources.

7.4 Model Training and Data

The training process for the LLM, although built upon an already capable Gemini foundation, is tailored to the specific needs of evaluator generation. Rather than performing full-scale training from scratch, a process that demands extensive computational power, the system relies on fine-tuning and curated data exposure. This approach allows the model to learn the structure, logic, and conventions of OpenEvolve evaluators without sacrificing performance efficiency.

Use of OpenEvolve Examples: The primary dataset consists of evaluator scripts drawn from the official OpenEvolve GitHub repository. These scripts represent a variety of tasks, including mathematical operations, symbolic reasoning problems, optimization challenges, and LLM optimization. Each evaluator is carefully selected to illustrate core patterns such as:

- Initialization procedures
- Fitness calculation strategies
- Agent-environment interaction logic
- Error handling
- Reward Shaping Techniques
- Integration with evolutionary loops

By training on these examples, the LLM develops an internal representation of evaluator structure that enables it to generate accurate templates and provide meaningful explanations.

Supplementary Documentation Data: In addition to code examples, the model is exposed to explanatory documentation that describes how evaluators function within OpenEvolve. This includes high-level conceptual explanations, flow diagrams, and text descriptions of evolutionary mechanics. This supplementary data enhances the model's ability not only to generate code but also to articulate the purpose and function of evaluator components when users request clarification.

Fine-Tuning Strategy: The fine-tuning strategy emphasizes small, high-quality datasets rather than large volumes of general code. The goal is not to create a general Python generator but to produce a specialized

assistant capable of operating precisely within OpenEvolve’s evaluator framework. This strategy ensures that the model retains efficiency and maintains short response times.

Error Patterns and Corrective Training: During development, common errors in evaluator generation were identified and incorporated into the training process. These included:

- Missing evaluation methods
- Improperly defined initialization parameters
- Incorrect handling of agent outputs
- Inconsistent scoring logic
- Lack of return values

By exposing the model to these error cases and their correct solutions, the training process improves the model’s ability to recognize and avoid similar mistakes in future outputs.

Training Outcomes: The training process results in an LLM that is capable of:

- Generating complete evaluator templates
- Explaining evaluator structure in user-friendly language
- Analyzing and debugging user-submitted scripts
- Providing suggestions for improving evaluator accuracy
- Adapting evaluator design based on user needs

This targeted training approach ensures that the model supports both learning and practical implementation.

7.5 Evaluation and Benchmarks

To ensure that the LLM performs consistently and accurately, a series of benchmarks and evaluation metrics are used to assess its output. These evaluations focus on structural correctness, functional reliability, and explanatory quality.

Structural Completeness: The structural completeness metric evaluates whether the model-generated evaluator contains all required elements. This metric checks for:

- presence of required functions
- proper imports
- correct indentation
- variables defined before use
- initialization blocks

The benchmark tests dozens of scenarios in which users request evaluators with varying levels of detail. The LLM consistently scores high in structural completeness due to the verification layer that corrects incomplete outputs.

Functional Correctness: Functional correctness measures whether the evaluator executes successfully within an OpenEvolve environment. To test this, generated evaluators are run against simple agent inputs.

The benchmarks test:

- execution without syntax errors
- logical correctness of scoring functions
- compatibility with OpenEvolve’s evaluation loop
- proper handling of agent-generated outputs

The model achieves high accuracy on these tests, particularly when generating evaluators for familiar problem types.

Explanatory Clarity: Because the LLM plays an instructional role, clarity is an essential metric. The explanatory clarity benchmark evaluates whether the model can:

- Explain evaluator logic in natural language
- Describe the purpose of functions
- Identify potential issues in user-submitted code
- Provide conceptual context

The model consistently provides articulate, accurate explanations that support user learning.

Adaptability and Responsiveness: To measure adaptability, the system evaluates how well the model handles:

- vague user requests
- iterative refinement
- follow-up questions
- domain-specific constraints

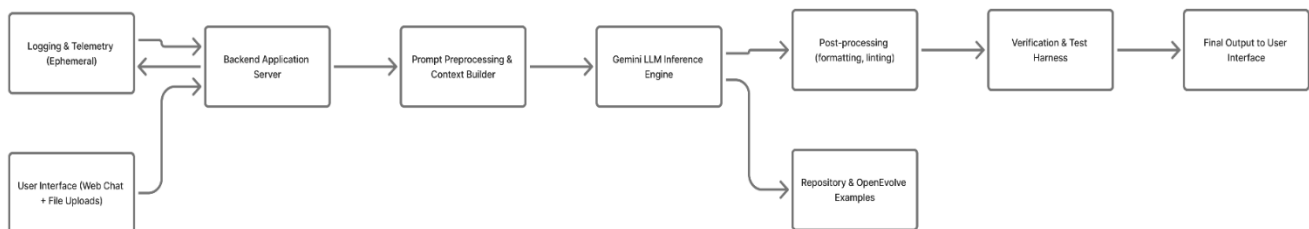
These tests confirm that the model maintains consistent internal logic and adapts effectively to evolving user needs.

7.6 Website Integration

The integration of the LLM into the website is designed to create a smooth, intuitive user experience that supports the entire evaluator creation process. The integration follows a workflow that emphasizes accessibility, consistency, and reliability.

Prompt Processing Workflow: When a user submits a request, the website follows a structured workflow:

1. User input received
2. Backend preprocessing
3. Relevance extraction and context building
4. Model inference
5. Post-processing and verification
6. Output delivered to user



Context Management: The website maintains conversation context, allowing users to build evaluators iteratively. This reduces the need for repeated explanations and supports a more natural interaction style.

Error Correction Integration: Users can upload incomplete or incorrect evaluators, and the website integrates the model's debugging capabilities to highlight errors and offer corrections. This feature supports users in refining their evaluators and learning from mistakes.

Template Generation Tools: The website includes tools that allow users to request templates based on:

- problem descriptions
- scoring strategies
- input-output examples

These tools leverage the LLM to provide ready-to-use starter files, accelerating experimentation.

Performance Optimization: Because the model is lightweight, inference latency is minimal, ensuring smooth interactions even under high traffic. This makes the website accessible to a wide audience without requiring expensive infrastructure.

7.7 User Experience and Features

The user experience is designed around the principles of accessibility, clarity, and flexibility. The LLM enhances the user experience by offering features that support evaluator creation at every skill level.

Natural-Language Interaction: Users can describe their evaluator needs in plain language. The model interprets these descriptions and generates corresponding code or explanations. Users can also request modifications or improvements to generated evaluators, allowing for fine-tuning and customization.

Debugging Assistance: The model can analyze user-submitted code, identify errors, and propose corrections, helping users improve their evaluators through guided feedback. This is useful to help those trying to learn how to improve their OpenEvolve skills.

Template Generation: The model will provide ready-to-use templates based on example problems, reducing time spent on manual implementation.

7.8 Security, Privacy, and Ethics

Security, privacy, and ethical considerations play an essential role in the integration of the LLM within the website, particularly because evaluator scripts may contain proprietary algorithms, unpublished research concepts, or experimental scoring logic. To ensure responsible handling of user data, the system follows strict privacy protections. User inputs are not stored beyond the duration of an active session, and evaluator scripts are processed only transiently to produce the requested output. None of the information users provide is incorporated into any model retraining or long-term datasets. These safeguards protect the confidentiality of user-generated evaluators and ensure that sensitive intellectual content remains under the user's control.

Ethically, the system is designed to constrain the model's behavior to its intended purpose of assisting with evaluation generation and explanation. This focused scope reduces the likelihood of inappropriate or irrelevant outputs that could mislead or confuse users. Transparency is also emphasized as a central ethical principle. Users are clearly informed that the system employs a Large Language Model and that all generated code should be reviewed before deployment in real experiments. This expectation encourages responsible use and prevents users from assuming that the model's recommendations are inherently flawless or authoritative.

Another ethical concern involves the risk of hallucinations or misleading information, issues that can occur in any generative model. To minimize these risks, the system incorporates verification routines and structured decoding strategies that restrict the model to deterministic, rule-based outputs whenever possible. These mechanisms ensure that generated evaluators remain aligned with OpenEvolve's structural requirements and that the model does not introduce unsupported or speculative logic.

7.9 Limitations and Future Work

Despite the strengths of the current LLM integration, several limitations remain that shape the direction of future improvements. The most significant constraint arises from the model's intentionally lightweight design. While this architecture supports efficient inference and low operating costs, it also limits the depth of reasoning the model can perform. As a result, highly complex or unconventional evaluator designs may exceed the model's current capabilities, requiring manual adjustments or expert intervention.

Another limitation stems from the model's reliance on example data drawn primarily from the OpenEvolve GitHub repository. Although these examples provide strong coverage of common evaluator structures, they do not encompass every possible task or scoring strategy a user may wish to implement. Rare, novel, or highly domain-specific evaluator patterns may therefore be challenging for the model to generate accurately.

Future work offers several opportunities to expand and refine the system. Increasing the size and diversity of the training dataset would enhance the model's generalization capabilities and reduce its reliance on a limited set of examples. Integrating multimodal inputs, such as diagrams, logs, or performance traces could provide richer context for evaluator generation and enable more sophisticated debugging assistance. Additional features, such as real-time execution testing or advanced visualization tools, would further support users in understanding and refining evaluator behavior. Collectively, these enhancements would strengthen the system's performance, improve user experience, and broaden the range of evaluator types the LLM can effectively support.

Chapter 8. Overview of Project Completion

8.1 Overview

The purpose of our project has stayed the same throughout the duration of our project. But the focus on the deliverables changed a lot as we got closer to completion. Before seeing what has changed our original minimum viable products were:

- Research report assessing the viability of OpenEvolve as a benchmarking framework for UCTP
- Poster summarizing findings and visualizations
- Real-time OpenEvolve monitoring GUI
- LLM-based code generation pipeline for OpenEvolve program and evaluator files.

The scope of the research paper and poster was ultimately reduced because the Space Domain Awareness group already appeared to have a strong understanding of OpenEvolve, its capabilities, and its intended use. As a result, the project focus shifted toward improving the speed and efficiency of workflows built around OpenEvolve through the development of the Analytics Console. By Senior Design II, many new features and design changes had been implemented in the Analytics Console because it had become the sponsor's primary priority. Meanwhile, the LLM-based code generation pipeline was reduced to a stretch goal, as the funding required to fully train the model was not feasible within the scope of the project.

Overall, the most significant outcome of the project was the Analytics Console itself. Its purpose is to support researchers developing uncorrelated track processors and related algorithms by giving them a more efficient way to monitor how their algorithms evolve in real time. The underlying motivation is that algorithm development for these processors is highly intensive and time-consuming. By combining OpenEvolve with the Analytics Console, the project aimed to help optimize this process by allowing researchers to observe algorithm performance as it changes over time. This enables them to better identify when and where an algorithm performs well, where performance begins to decline, and how those changes develop throughout the evolution process.

8.2 Documentation Finalization Overview

This section of the document serves to walk through the GitHub environment that all the researchers are working on. This environment is accessible to the public alongside additional contributors from sponsors and scientists we are working with. All code serves for transparency towards Space Domain Awareness. The following link is provided:

[Project](#) [Code](#)

Figure 8.1.1: These links are connected to both GitHubs being used. First one being Project which is a link to publicly available UCTP evaluator code. Second one being the Code which is a link to the following GitHub environment being shown in this section of the document.

Our documentation workflow is managed primarily through GitHub and presentation slide decks, which together provide a structured and traceable record of the project's development. Within the GitHub repository, we maintain a custom LaTeX template that was specifically designed to keep all written deliverables consistent in style, professional in appearance, and easy to revise as the project evolved. This template was used across formal write-ups, technical summaries, and supporting project documents, including the final design document. By standardizing documentation in this way, the team was able to maintain a cohesive format across deliverables while also making updates more efficient throughout both Senior Design I and Senior Design II.

In addition to the written documentation, all presentation slides are also stored and version-controlled in the same GitHub repository. These slide decks reflect the material presented during weekly sponsor meetings and serve as a chronological record of the team's progress, design changes, research findings, and task progress. Because these materials are preserved in version control, they do more than document final outcomes; they also capture the evolution of the project over time. This makes it easier for team members, sponsors, and future collaborators to review the reasoning behind design decisions, understand how requirements shifted, and trace the progression from early concepts to the final implementation.

Beyond reports and slide presentations, the full development of the Analytics Console is also maintained within the GitHub repository. As a result, the repository serves not only as a storage location

for source code, but as a complete record of the project’s technical development. This centralization is especially valuable for long-term maintainability, since future teams can review both the implementation and the supporting documentation in one location rather than relying on separate disconnected sources.

The Analytics Console itself also includes an internal documentation webpage designed from both a developer and user perspective. From the developer standpoint, this page provides notes on the current build, explanations of how data is retrieved and displayed, and guidance for future contributors who may continue expanding the console after the current team’s work is complete. From the user standpoint, the documentation page serves as a tutorial and onboarding guide, helping first-time users understand how to navigate the console, configure data directories, and interpret the information being displayed. It also explains how to set up the console in different environments, including local machines and high-performance computing systems, so that users can deploy and adapt the platform without having to manually search through the source code.

Together, the GitHub repository, LaTeX documentation, slide decks, final design materials, and built-in Analytics Console documentation create a unified and well-documented project record. This documentation structure not only supports the current team and sponsors but also provides a strong foundation for future developers and researchers who may build on the project later. By maintaining both technical and user-focused documentation in an organized and version-controlled format, the project is better positioned for continuity, transferability, and long-term use.

Uncorrelated Track Processing Algorithms

Senior Design 2025-2026

Kyle F. Galang
Aurelia Brosi
Ruben Dennis
Aaron Noguez
Ezra Stone



University of Central Florida
Department of Computer Science

1.16 - Uncorrelated Track Processing Algorithms

Documentation, XX/XX/XXXX - XX/XX/XXXX

Contents

1 Kyle's Section: Senior Design I Documentation Build-up	2
1.1 Week 1 Research Break-down	2
1.1.1 Project Concepts and Organization	2
1.1.2 Software Integration into Research	2
1.1.3 OpenEvolv Environment Setup	3
2 Aurelia's Section: Senior Design I Documentation Build-up	4
2.1 OpenEvolv Environment Setup	4
2.1.1 Topic no.1	4
2.1.2 SCRUM	4
2.1.3 UI Development Tech Stack	5
3 Ruben's Section: Senior Design I Documentation Build-up	6
3.1 Week 1 Research Break-down	6
3.1.1 OpenEvolv Environment Setup	6
3.1.2 UI Development Tech Stack	6
3.1.3 Topic no.3	7
4 Aaron's Section: Senior Design I Documentation Build-up	8
4.1 Week 1 Research Break-down	8
4.1.1 OpenEvolv Environment Setup	8
4.1.2 Evaluator Script Automation	8
4.1.3 UI Development Tech Stack	9
5 Ezra's Section: Senior Design I Documentation Build-up	10
5.1 Week 1 Research Break-down	10
5.1.1 OpenEvolv Environment Setup	10
5.1.2 UI Development Tech Stack	10
5.1.3 Topic no.3	11

Figure 8.2.1: This figure showcases the LaTeX template the group created to summarize a professional document for our findings.

Each team member has installed OpenEvolve on their local machine so they can independently experiment, track progress, and modify files as needed. To avoid losing work and to keep our changes synchronized, we also push our environments and key configuration files to GitHub, giving us version control and an easy way to roll back if something breaks.

Because OpenEvolve’s architecture is fairly complex, even small edits to the evaluator or initial Python files can significantly affect how iterations are run and how metrics are computed. To better align the framework with our goals, we are developing our own subset of benchmark problems that more closely resemble UCT scenarios. Most of our file changes focus on customizing the symbolic regression problem definitions, allowing us to study and adapt OpenEvolve’s behavior for UCT-like use cases.

Name	Last commit message
..	
Aaron	Delete Researchers/Aaron/Personal-Documetation/TrackAssociation/_py...
Aurela	Document Symbolic Regression Code Changes
Ezra	Add files via upload
Kyle	updating UML diagrams
Ruben	Moved my setup to a folder

Figure 8.3.1: This figure showcases the different directories per researcher. This allows us to isolate our changes when developing our own approaches on how to integrate UCTP into OpenEvolve.

Name	Last commit message
..	
Personal-Documetation	Delete Researchers/Kyle/Personal-Documetation/ clean up
UML-DIAGRAMS	updating UML diagrams
.gitignore	Add files via upload
CLAUDE.md	Add files via upload
CONTRIBUTING.md	Add files via upload
Dockerfile	Add files via upload
LICENSE	Add files via upload
MANIFEST.in	Add files via upload
Makefile	Add files via upload
README.md	Add files via upload
openevolve-architecture.png	Add files via upload
openevolve-logo.png	Add files via upload
openevolve-run.py	Add files via upload
openevolve-visualizer.png	Add files via upload
pyproject.toml	Add files via upload
setup.py	Add files via upload

Figure 8.3.2: This figure showcases the OpenEvolve environment in each researcher’s directory. Of course, because we are uploading our OpenEvolve environments into GitHub, we also set up secrets which are basically environment variables for our OpenAI API keys.

8.3 Research Finalization Overview

The project set out to evaluate whether Open Evolve is suitable for the U.S. Air Force's development of uncorrelated track processing algorithms. In doing so, the team identified three main requirements: Open Evolve must be flexible with different data formats, it must be able to understand specialized concepts within a larger system context, and it must reduce development effort rather than increase it.

Early testing showed that Open Evolve could improve a simple processor from 0% to 51% correctness in under 15 iterations with minimal prompting and setup. However, this progress quickly plateaued, and each iteration required enough time that larger runs would become costly and impractical. When compared to a manually developed processor using a commercial LLM, which reached 98% accuracy after some tuning, Open Evolve appeared less efficient as a practical solution.

Because of these results, the team shifted its focus away from proving that Open Evolve could solve the problem at all and instead toward understanding how useful it is from a user and research standpoint. The team then explored several categories of "hard data," including prompting style, non-prompt configuration changes, language compatibility, and performance across different problem types.

The findings showed that Open Evolve could improve programs across several example problem categories, including circle packing, function minimization, online judge programming, and signal processing. Prompt wording and "niceness" had little impact, since Open Evolve only accepts valid code outputs. Language testing showed that English, German, Spanish, and mixed German-Spanish prompts performed similarly, while Chinese prompts consistently performed worse, likely due to tokenization and structural language differences. Some configuration-based testing was limited because of token restrictions, so the team could not fully evaluate all aspects of Open Evolve's island and genetic architecture.

Overall, the team concluded that Open Evolve can make some progress in improving code, but its usefulness is limited by long iteration times, token costs, and the amount of setup required. In its current form, it does not appear efficient enough to justify replacing traditional development methods for the Air Force's needs. For Open Evolve to become more practical in this domain, it would require a highly capable subject-specific LLM, strong prompting strategies, and a more sophisticated system design to support complex astrophysical and tracking-related problems.

Insights Gained: (Ruben Dennis)

One of the changes that I would make differently is using the Open Evolve configuration file appropriately. As a result of token limitations, I had to make changes to some of the main parameters, including the number of iterations and islands. Although OpenEvolve can handle much higher limits of iterations, in my case, the testing could not exceed several dozen, which did not allow me to get the best results. For this reason, I could not investigate other aspects of algorithm improvement, including migration or long-term convergence.

I also found that Open Evolve is somewhat confusing to use overall, particularly when working with the configuration file. There are many parameters across different sections LLM settings, evolution settings, database configuration, and prompt configuration and it is not always clear which ones have the greatest impact on performance. Although it is possible to achieve a desired outcome through extensive experimentation, many questions remain about which parameter provides more significant changes than others. Therefore, despite the fact that the system works and can indeed help improve the algorithm, it takes a lot of effort to use the application efficiently.

A major change I would make to this project would be to increase the available computational resources and funding. With more tokens and the ability to run a larger number of iterations, it would be possible to explore Open Evolve more deeply and better evaluate its true capabilities. This would allow for more thorough testing of configuration parameters, longer evolution cycles, and more complex problem setups. Without these resources, the conclusions drawn from this research is based on a constrained version of the system, rather than its full potential.

Overall, what has been learned from this process is that Open Evolve is a tool that can yield positive changes but one that requires significant resources. Depending on the decisions made in terms of how to configure the system and which prompts to use, it is possible to get better results with sufficient resources.

Lessons learned and Insights gained: (Aaron Nogues)

Even after interfacing with Open Evolve so much, I find the process to be very tedious. Even when it's in the middle of running, I can't help but think of stopping the process for just one more change to be made to improve. Thinking of changing the prompt diction so the LLM avoids iterating with

no substantial changes or responds in text and context that is flagged as non-code changes. Feeling pressure to know which parameters the evaluator should be tracking despite not having fully intimate knowledge of the subject matter. Personally, the commercial AI as it currently stands with individual prompting guided by iterative human fed prompts process as a whole seems superior. Even while prompting I can use it to improve my knowledge of the subject, at least enough to explain it, as it currently stands Open Evolve is just a black box where I hope it improves. If I were to use Open Evolve personally it would need not only what was developed by Kyle, but an environment to manipulate Open Evolve and the files that it uses. I shouldn't need multiple tabs in my code editor for changing configuration files and then an additional web page open with documentation. All this especially when most parameters given are optional.

I will take this lesson of interfacing with new technology and always remind myself to think of the user experience. No software that I ever develop should confuse new users and still be tedious to those with experience. When reading documentation, it shouldn't feel like trying to find a needle in a haystack.

8.4 Symbolic Regression Research Finalization Overview

Right now, the focus has been on running OpenEvolve experiments in a way that is conceptually similar to AlgoTune. This includes evolving programs across multiple iterations, evaluating them using state, binary, and residual metrics and observing how the solutions improve over time. Instead of using pre-defined symbolic regression benchmarks, we are applying these ideas directly to our own evaluation setup.

From this, we started to see how symbolic regression connects to what we are doing. Instead of assuming a fixed structure like in traditional regression, the system is effectively searching over different program structures and improving them based on performance. In our case, this is happening through OpenEvolve, where code is being mutated and evaluated rather than just fitting parameters to a known model.

The work so far has been focused on understanding how these concepts translate into practice, especially how OpenEvolve evolves solutions over time and how evaluation metrics influence that process. This builds on the earlier work with AlgoTune based tasks and helps connect the idea of symbolic regression to real algorithm optimization.

Overall, LLM-SRBench helped provide context and understanding, while AlgoTune and OpenEvolve are what we are actually using to run experiments and evaluate performance improvements. Additionally, ideas from LLM-SR’s implementation, along with tracking how performance changes across iterations, were used to improve both the stability and understanding of our OpenEvolve experiments. To better understand how the system behaves in practice, we analyze two OpenEvolve runs: one where no improvement occurred, and one where the system successfully improved over time.

Example 1: No Improvement (0% Improvement Rate)	Example 2: Successful Evolution (75% Improvement Rate)	
Initial combined_score: 0.8994	Iteration 0:	combined_score = 0.8957 (initial program)
Iteration errors:	Iteration 9:	combined_score = 0.9481 (+0.0524) ← NEW BEST
Iteration 1 → No valid diffs	Iteration 10:	combined_score = 0.8941 (decline)
Iteration 2 → No valid diffs	Iteration 11:	combined_score = 0.9296 (recovery)
Iteration 3 → No valid diffs	Iteration 12:	combined_score = 0.9368 (improvement)
...	Final Results:	
Iteration 10 → No valid diffs		
Final Results:		
Improvement rate: 0.00%		
Best program metrics:		
accuracy: 0.9517		
precision: 0.9139		
recall: 0.6732		
f1_score: 0.7753		
combined_score: 0.8994		

This demonstrates that the evolution process is nonlinear, where improvements may be followed by temporary declines before stabilizing. Despite these fluctuations, the overall trend shows consistent progress, resulting in a significantly higher final score compared to the initial solution. These results show that OpenEvolve does not guarantee improvement at every iteration but instead explores the solution space in a way that may include failures, recoveries, and breakthroughs. The 75% improvement rate indicates that most iterations contributed positively, while the presence of declines highlights the importance of tracking performance across iterations and retaining high performing solutions. This highlights that the effectiveness of the system is not solely determined by the final score, but by how consistently it is able to explore, recover, and converge toward better solutions over time.

Noise Robustness Analysis

To evaluate how the system performs under imperfect data conditions, it is important to consider how noise is handled during the tracking process. In real world scenarios, observations contain measurement uncertainty, which can impact the accuracy of track association if not properly accounted for.

Noise is implemented directly into the tracking model through uncertainty-aware covariance modeling within the Kalman filtering process. Measuring uncertainty is derived from parameters such as range and line-of-sight uncertainty, which are used to construct the observation noise matrix.

During deterministic initialization, each observation is compared against predicted track states using a Mahalanobis distance-based gating approach. This allows the system to evaluate how consistent an observation is with a predicted track relative to its uncertainty.

Example of Kalman Filter + Gating in Track Association

```
# Predict next state
x_pred = F @ x_prev
P_pred = F @ P_prev @ F.T + Q

# Residual between prediction and observation
y = z - H @ x_pred

# Covariance including uncertainty
S = H @ P_pred @ H.T + R_mat

# Mahalanobis distance
d2 = float(y.T @ np.linalg.solve(S, y))

# Gating decision
if d2 < GATE_THRESHOLD:
    best_score = d2
    best_track_index = k
```

The gating threshold of 9.0 corresponds to approximately three standard deviations in three-dimensional space, meaning that only observations that are statistically consistent with the predicted track are accepted.

This approach was implemented to improve robustness in the presence of noisy measurements. Without incorporating uncertainty, the system would rely on simple distance-based comparisons, which can lead to incorrect associations when observations are noisy or when uncertainty varies across measurements. As well as when the system becomes complex with large datasets.

By incorporating uncertainty directly into the association process, the system is able to make probabilistic decisions, improving stability and reducing the impact of noise on overall tracking performance. This is particularly important for UCTP, where incorrect associations can propagate errors throughout the tracking pipeline. Which are also calculated in the evaluator script and used to feed into the OpenEvolve evolution pipeline to improve and lower those False Positives and False Negatives, and therefore evolving a stronger performing model.

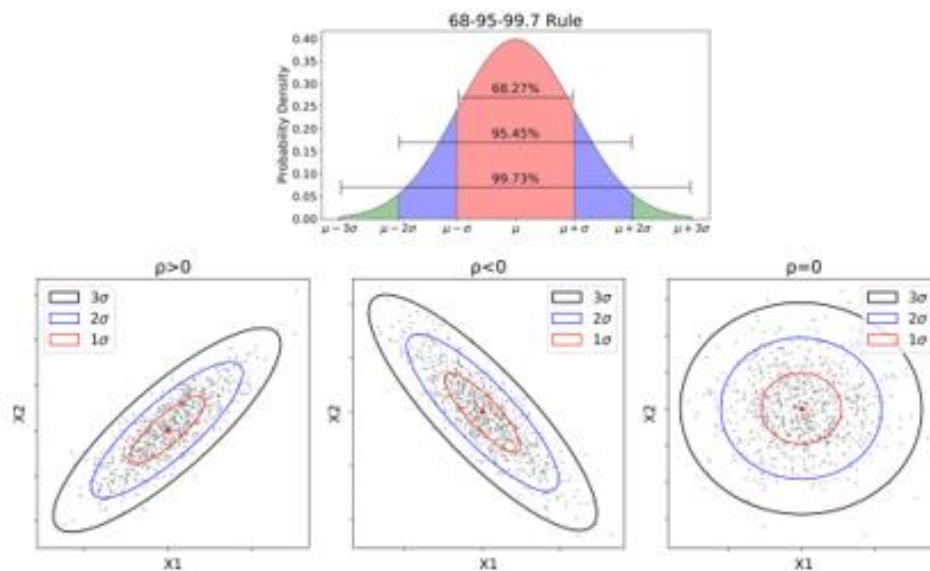
To further analyze robustness, different levels of measurement uncertainty can be considered using a Gaussian noise model, where the standard deviation (σ) represents the level of noise in

observations. In this system, σ is derived from sensor uncertainty parameters such as line-of-sight uncertainty, which are used to construct the observation noise matrix.

For example, when σ is very small, observations contain minimal noise, and track associations are expected to be highly accurate. As σ increases, observations deviate further from the predicted state, which increases the residual error. And directly affects the Mahalanobis distance used in the gating process.

Since the Mahalanobis distance is scaled by the covariance matrix, higher uncertainty allows for larger deviations and are still considered to be valid, while lower uncertainty results in stricter gating. This enables the system to adapt its association decisions based on the level of noise present in the observations. This demonstrates how the system balances sensitivity and robustness. If we have low noise, it leads to stricter associations, while higher noise requires more flexible gating to maintain correct track continuity.

Covariance and Uncertainty Representation



To understand how uncertainty is modeled in the tracking system, we begin with a Gaussian (normal) distribution, often visualized as a bell curve. The mean (μ) represents the most likely value, while the standard deviation (σ) describes how spread out the data is around that meaning.

For example, in a one-dimensional Gaussian distribution, approximately 68% of values lie within one standard deviation of the mean, 95.45% within two standard deviations, and 99.7% within three. Which provides a probabilistic interpretation of uncertainty, where values closer to the mean are more likely, and values further away have lower probability. This means we do not know exactly how far away an object is but instead define it as most likely a certain distance away with some uncertainty. We try to illustrate this idea using probability distribution, whereas we move further away from the mean, there is a smaller chance that the object is actually that far from its estimated position. This is important because it means that we do not assume an object is exactly at a single point, but rather that it is most likely near a predicted position with some uncertainty. This uncertainty is represented using variance, which defines how wide or narrow the distribution is. So, if we have a higher standard deviation it results in a wider distribution, meaning there is a higher chance that the object is further away than expected. And a smaller standard deviation results in a narrower distribution, meaning we are more confident that the object is close to the estimated position.

When we apply this idea to multiple dimensions, such as position in space, we consider not only the variance in each direction but also how those directions relate to each other. For example, in two dimensions, we have variance in the x and y directions, as well as covariance between them. Covariance represents how two variables vary together, indicating whether they are correlated. This idea helps us understand the relationship between variables as we are dealing with six-dimensional state vectors.

These values are combined into a covariance matrix. In two dimensions, this is a 2×2 matrix; in three dimensions, a 3×3 matrix; and in this project, the full state consists of three position and three velocity components, resulting in a 6×6 covariance matrix. To understand what this represents, if we take many measurements of the same object, we will not get the exact same value every time due to noise and other sources of uncertainty. Instead, we obtain a distribution of possible values. The mean of this distribution represents the most likely estimate, while the covariance describes how much variation exists around that estimate.

The state vector defines the mean of the distribution (the most likely position and velocity), while the co-variance matrix defines the uncertainty around that estimate. When put together, they describe a probability distribution over possible object states rather than a single deterministic value.

This representation is critical for UCTP, as it allows the system to compare not only the difference between estimated states, but also how confident those estimates are. And by incorporating covariance,

the system can evaluate whether two observations are statistically consistent, which directly supports the uncertainty-aware gating and association logic used in the tracking pipeline.

Design Considerations

One of the key design decisions in this project was to implement a structured baseline for track association prior to applying OpenEvolve. This was done to provide a structured starting point for the optimization process, rather than relying on fully generated solutions from an LLM.

Without a structured baseline, the LLM-guided search may generate overly complex or irrelevant modifications, which can reduce efficiency and lead to unstable behavior during evolution. By starting with a deterministic and physically grounded implementation, the system is able to focus on refining meaningful logic rather than exploring arbitrary solutions.

After researching some of the previous work in LLM-based optimization, it focused on the importance of structure in the generation process. And approaches that relied on fully generated solutions often produced unstable or irrelevant outputs, depending on the problem it was solving. And this was especially noticeable when domain specific constraints are not enforced. This was also observed during experimentation, where unconstrained models tended to generate solutions that were syntactically valid but logically incorrect or not meaningful. In some cases, the models exposed weaknesses in the evaluation setup to produce outputs that appeared correct based on the metric, but did not represent valid or realistic solutions, more on this in the next section.

In contrast, methods that guide generations using structured representations or partial initial program templates are able to produce more meaningful and evolutionary results. This reduces unnecessary exploration and improves overall optimization efficiency. This is the baseline structure I used when experimenting with OpenEvolve. From this I noticed that the system is able to focus on refining existing logic rather than generating entirely new solutions, resulting in real stable solutions.

Degenerate Solution Behavior and Evaluation Limitations

A key observation during experimentation was the emergence of a degenerate solution, where the system collapsed to assign all observations into a single track. In this case, the model learned that

grouping all observations together guaranteed perfect recall, since no true associations are missed. However, this comes at the cost of significantly increasing false positives, resulting in poor precision and zero specificity. So, it essentially learned that it could cheat the system to get the desired output by grouping everything into one track. From here, I needed to adjust the constraints and optimize my metrics so that it did not allow such solution to be awarded.

Example:

```
Evolution complete!
Best program metrics:
  accuracy: 0.1239
  precision: 0.1239
  recall: 1.0000
  specificity: 0.0000
  balanced_accuracy: 0.5000
  f1_score: 0.2204
  combined_score: 0.0000
  num_tracks: 1.0000
  artifacts: {'num_tracks': 1, 'total_pairs': 706266, 'tp': 87476,
             'fp': 618790, 'fn': 0, 'tn': 0}
```

This behavior revealed a limitation in the evaluation function. Since the objective is focused on minimizing false negatives, the system is incentivized to prioritize recall over all correctness. As a result, it converges with a solution that maximizes the metric without producing a meaningful or valid result. This can be useful since the system is exposing weaknesses in the scoring function. And interesting to note that rather than improving the underlying tracking logic, the model adapted to the evaluation criteria itself.

So, if the objective function does not balance competing metrics, the system may converge to degenerate solutions that satisfy the metric but fail in practice. The key takeaway to this is that the system did not degrade in capability but instead learned to optimize the evaluation function. This highlights the importance of carefully designing scoring functions to ensure that improvements in metrics correspond to meaningful improvements in system performance. This approach also reduces unnecessary exploration during the optimization process, allowing OpenEvolve to make more targeted improvements and converge more efficiently toward higher scoring solutions.

This design choice aligns with broader observations in machine learning, where methods that incorporate prior knowledge tend to be more robust under noisy conditions compared to purely data-based approaches. By constraining the search space to meaningful and physically consistent solutions, the system avoids unstable outputs when data quality degrades. Overall, providing structure through a

baseline implementation improves both the efficiency and stability of the optimization process, while enabling OpenEvolve to effectively refine the underlying tracking logic.

Influence of LLM-SR Design Principles

The design of this system was also influenced by concepts from LLM-SR, particularly its use of structured optimization and experience-based learning. One key idea is maintaining and tracking strong performance solutions over time, rather than treating each iteration independently. In LLM-SR, this is implemented through an experience buffer that stores and prioritizes stronger program candidates.

A similar concept is applied in this project through tracking best programs, improvement rates, and iteration performance changes. This allows the system to retain useful solutions while continuing to refine them across iterations.

Another important idea is the use of multiple exploration paths, where different candidate solutions are evaluated in parallel. This helps prevent the system from converging too early to a suboptimal solution and encourages broader exploration of the solution space.

Example (Buffer Implementation using Softmax)

```
def _softmax(logits, temperature):
    logits = np.array(logits, dtype=np.float32)
    return scipy.special.softmax(logits / temperature, axis=-1)

cluster_scores = np.array([]) # scores of candidate programs

probabilities = _softmax(cluster_scores, temperature)

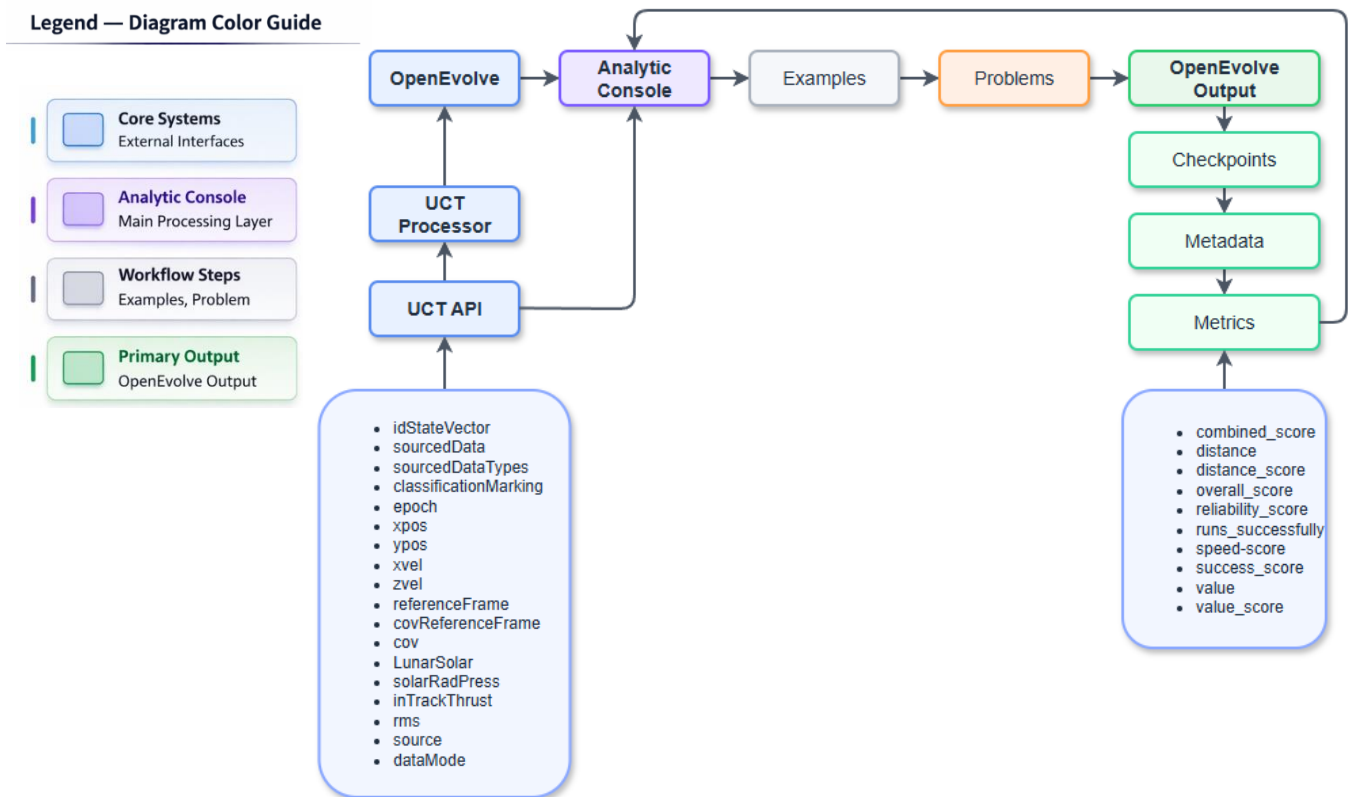
selected_index = np.random.choice(
    len(cluster_scores),
    p=probabilities
)
```

These concepts influenced the decision to incorporate structured tracking and evaluation into the OpenEvolve pipeline, enabling more stable and efficient optimization over time.

8.5 Analytics Console Finalization Overview

The Analytics Console has changed considerably from the original Figma design schematics created during Senior Design I. Over time, new features were introduced to address updated sponsor requirements. Because the Analytics Console became the central focus for the sponsors, Senior Design II concentrated heavily on its continued development to meet those expectations and provide a more effective tool for the researchers using it.

First, we must go over what data is being used in the Analytics Console alongside how it works:



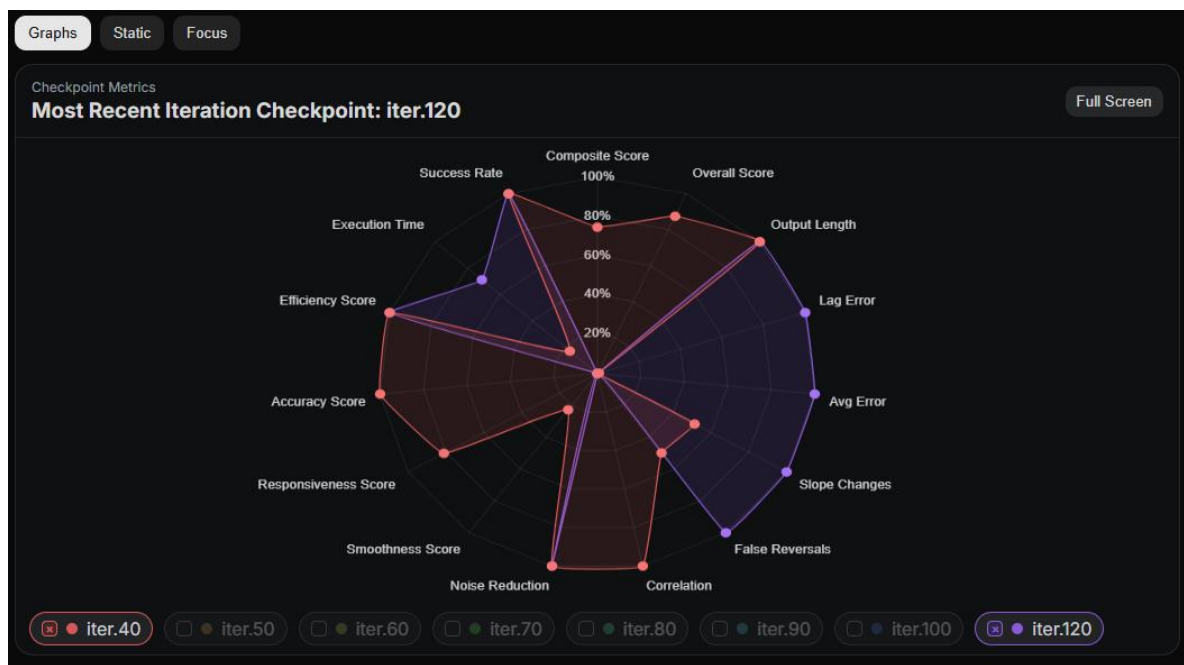
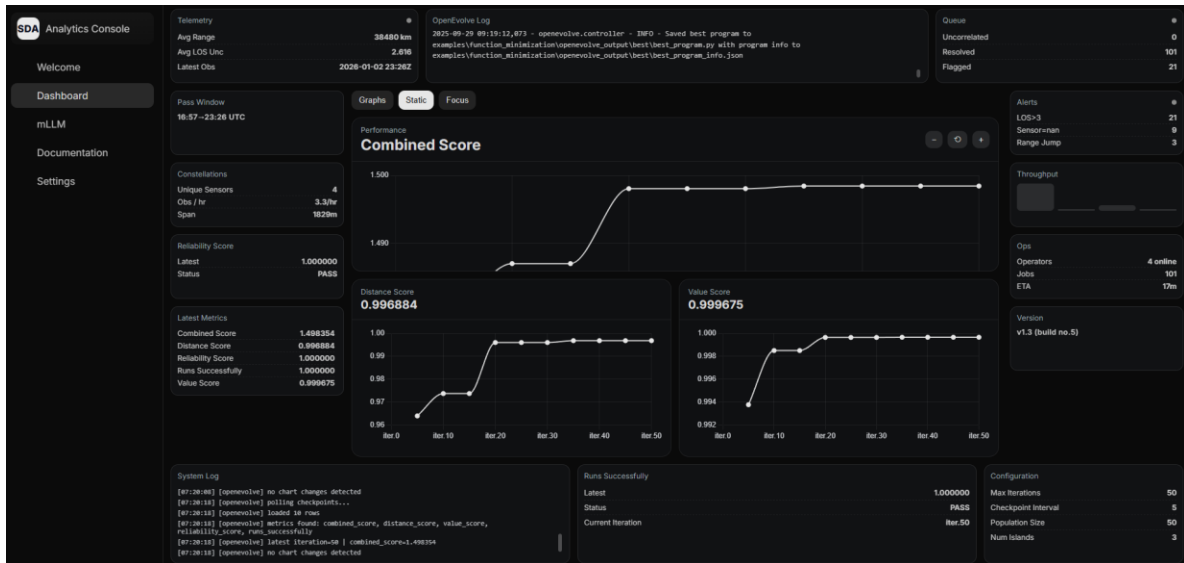
In the dashboard page, many different data types are being transmitted through.

- OpenEvolve's Checkpoints:** One of the primary data sources in the dashboard comes from OpenEvolve's checkpoints. After each evolution cycle, OpenEvolve outputs the best programs along with their associated metrics at each checkpoint defined in the configuration and evaluation files. These checkpoints are written to the disk as folders,

and each checkpoint folder contains a `best_program_info.json` file that stores the iteration data and a metrics object. Every 10 seconds, the backend scans these checkpoint folders, sorts them in checkpoint or iteration order, and normalizes the numerical values. It then exposes the processed data through a REST-style API endpoint. On the front end, the dashboard continuously polls this endpoint, converts each metric into a time series where the x-axis represents the current iteration and the y-axis represents the metric value, and passes that data into Chart.js for graph rendering. The panels that rely on this polling process include the Performance panel, Metric panels, Runs Successfully panel, and the Graph display. If any issues occur during this process, they can typically be observed in the system log terminal, which provides updates related to the API endpoints.

- **UCT API:** The dashboard also displays dataset information using dummy data. This data is stored in the assets folder within the UI-DEVELOPMENT directory. One example is the “1 object 1000 observation” dataset provided by our sponsors, which follows the UCT benchmarker structure used to input raw data from the Unified Data Library and into the Processors. This data is also polled into the dashboard and is mainly used for the outer panels, including Constellations, Telemetry, Queue, Alerts, and Ops. Although it is currently dummy data, it allows the interface to simulate how real benchmark or observation-based information would appear in the final system.
- **OpenEvolve’s Configuration:** The Analytics Console is also capable of rerouting configuration data through the settings webpage. This capability is primarily used by the Configuration panel in the lower-left corner of the dashboard, allowing users to control where configuration information is sourced.
- **OpenEvolve’s Log:** In addition, the dashboard can display OpenEvolve log data. These logs are automatically generated during OpenEvolve execution and correspond to the messages normally shown in the terminal while a run is in progress. The Analytics Console retrieves these updated logs and displays them in the frontend, improving visibility into system activity without requiring users to monitor the terminal separately.

Dashboard Page:

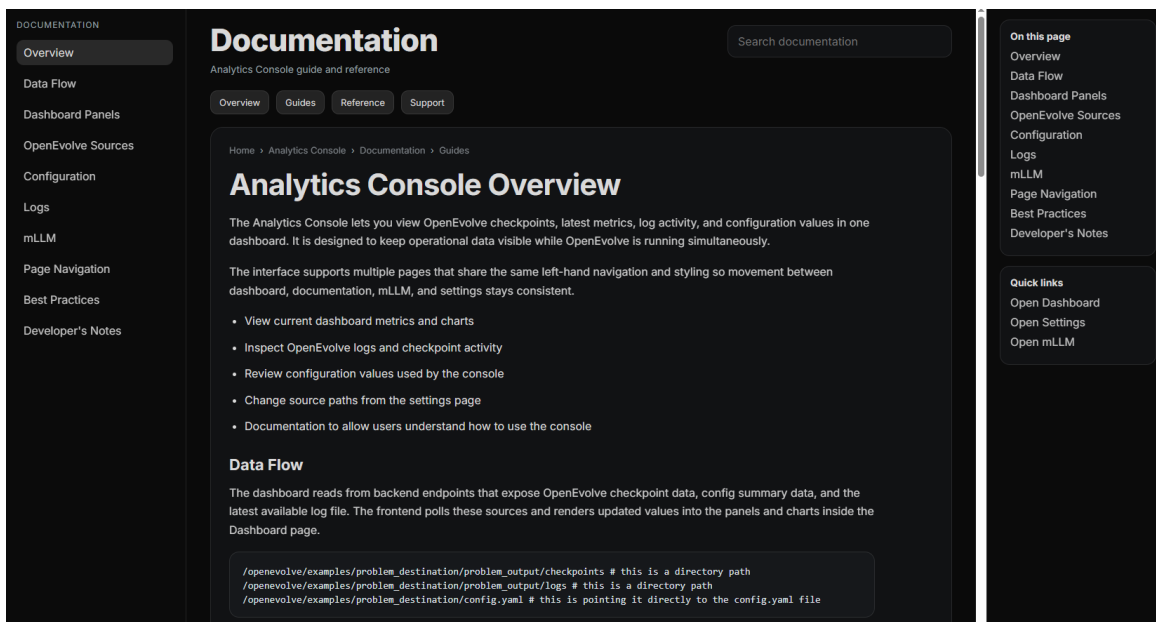


These changes were specifically requested by the sponsors to ensure that researchers have access to as much relevant data as possible when using OpenEvolve for automatic algorithm discovery and optimization of uncorrelated track processors. In addition to these enhancements, supporting webpages were developed to improve the overall usability of the Analytics Console and reduce the need for researchers to modify code when deploying the system across different environments, including HPC systems and local government machines.

These supporting webpages include the Documentation and Settings pages within the Analytics Console.

Documentation: The Documentation page is designed primarily to help first-time users understand and navigate the Analytics Console. It explains how data is collected and displayed, how directories should be configured, and how to set up the system so that real-time updates are shown during OpenEvolve runs. In addition, the page outlines best practices for configuring and using the console, provides developer notes on the current build, and identifies potential areas for future development that were beyond the scope of the current team and may be addressed by future teams.

Documentation Page:



Settings: The Settings page is a critical component of the Analytics Console because it allows users to configure the directories from which data is pulled. Because this data is often stored on high-performance computing systems, the required file paths can vary depending on the environment and specific run setup. By allowing users to update these paths directly through the interface, the page makes the console more flexible and easier to deploy across different systems. Most importantly, it enables researchers to quickly switch between directories for different OpenEvolve runs they are monitoring, eliminating the need to manually modify code whenever a new run or environment is used.

Settings Page:

Settings

Program	Data Pull	API Key
Program Name initial-program Click the Program Name label to rename the project.	Pathway C:\Users\kfg4s\Downloads\SENIOR DESIGN\openevolve\ Click the Pathway label or use the pencil to change the checkpoint path.	Key OPENAL_API_KEY This reads and writes the real api_key value from your OpenEvolve YAML config.
Configuration configuration.yaml This dataset is to transfer configuration data onto the Analytic Console.	Source C:\Users\kfg4s\Downloads\SENIOR DESIGN\openevolve\ Click the Source label or pencil to change the YAML config path.	— No API key used for this row.
OpenEvolve Logs logs.txt This row controls where the OpenEvolve Log panel pulls its latest log file from.	Source C:\Users\kfg4s\Downloads\SENIOR DESIGN\openevolve\ Click the Source label or pencil to change the logs folder used by the OpenEvolve Log panel.	— No API key used for this row.

SDA TAP LAB

Transferring to a new environment?

This page shows you how current data is being processed into the Analytics Console. You can also manage and reconfigure new paths for new environments above. [Learn more](#)

Lessons learned and Insights gained: (Kyle Galang)

One of the main things I wish I had done differently during the development of the Analytics Console was to spend more time learning how to scale the system and optimize the way it pulls data. As Senior Design II came to an end and I began preparing for the final project demonstration, I realized there were many areas I would have liked to improve.

One major improvement would have been creating a more flexible method for pulling data from higher-level directories and automatically accessing files within nested subdirectories, regardless of how deep the file structure extends. A design like this would have simplified the API endpoints and made the overall backend architecture more efficient and easier to maintain.

Another area I wish I had explored further was the scalability of the project itself. I came to realize that researchers working on algorithms for uncorrelated track processors are likely not focused on just a single algorithm, but on many algorithms at once, since UCT is a highly multidisciplinary field. Because of this, I would have liked to expand the Analytics Console so it could support multiple

OpenEvolve instances running at the same time. This would allow users to monitor several runs simultaneously, observe how each one evolves from its initial function, and compare results side by side.

If I were to approach this project again, I would place a much stronger emphasis on restructuring the architecture of the Analytics Console and thinking more deeply about the overall system design. In particular, I would focus on how the console could support multiple users and multiple concurrent OpenEvolve instances, rather than being limited to a single user monitoring a single run. I would also explore integrating a database to store OpenEvolve runs, metrics, and generated graphs. In the future, this could function similarly to version control for OpenEvolve experiments, allowing researchers to preserve, compare, and revisit different runs over time. This would be especially valuable because different researchers may prompt OpenEvolve to solve the same problem in different ways, making it important to track and compare those variations systematically.

Overall, I am satisfied with the design I developed, particularly because it was driven by my passion for frontend design and user experience. I included developer notes in the Documentation page of the console to outline future enhancements that I was unable to contribute to during this project. My long-term vision for the Analytics Console is for it to serve as a sandbox- or whiteboard-style environment where researchers can experiment, organize their work, and interact with OpenEvolve more efficiently, resulting in faster, cleaner, and more intuitive workflows when developing algorithms.

8.6 mini-Large Language Model Finalization Overview

Ultimately, the chatbot described earlier in this document changed significantly from its original concept. Initially, the goal was to train the chatbot to reference existing OpenEvolve example problems that had already been tested, while also incorporating future training using custom programs developed for basic UCT processor functions. However, as the project progressed, it became clear that the complexity of uncorrelated track processing required a much broader foundation of knowledge, including mathematics, physics, statistics, and astrophysics.

As a result, the team restructured its approach to the chatbot. Instead of first focusing narrowly on OpenEvolve-specific examples, the revised goal was to begin with generalized training and then fine-tune the model for code generation based on OpenEvolve's specific requirements. Due to these challenges and shifts in project scope, the chatbot gradually became more of a stretch goal by the end of

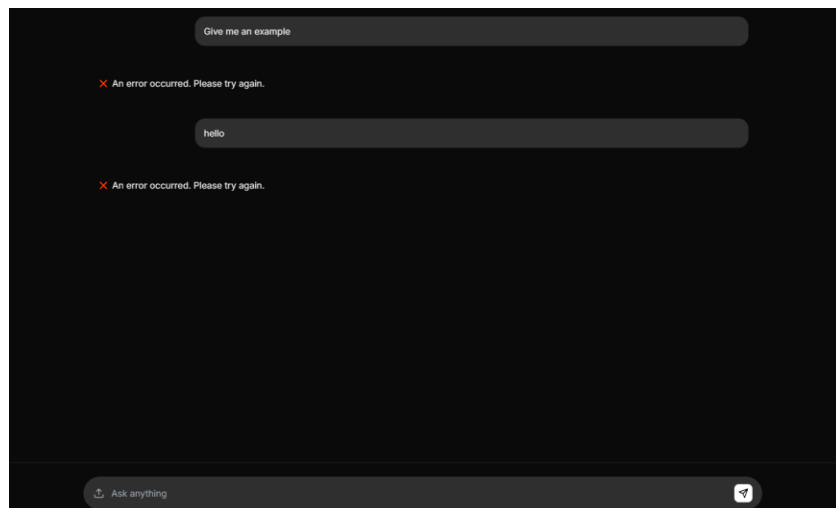
the project, as the primary focus shifted toward the development of the Analytics Console in order to better support the researchers.

In the past, the chatbot faced many limitations, particularly in determining how many tokens would be required to make it viable for code generation and reasoning tasks. Tokenization was consistently a challenge throughout development, as it significantly affected both model performance and training feasibility. At present, the Analytics Console includes a placeholder for the mLLM, but there is not yet a fully trained language model integrated into the console for live prompting. This is largely due to the shift in how the model is being developed and trained.

Within the current file system, there is a localized model that was trained for approximately four hours on a single NVIDIA RTX 3060 Ti GPU. So far, this model has only been trained on basic tokenization, and it still struggles to produce responses that are both complete and consistently correct. The model is based on nanochat, an experimental LLM training framework created by Andrej Karpathy, former Director of AI at Tesla and one of OpenAI's founding researchers. Nanochat focuses primarily on the pretraining and fine-tuning stages of language model development.

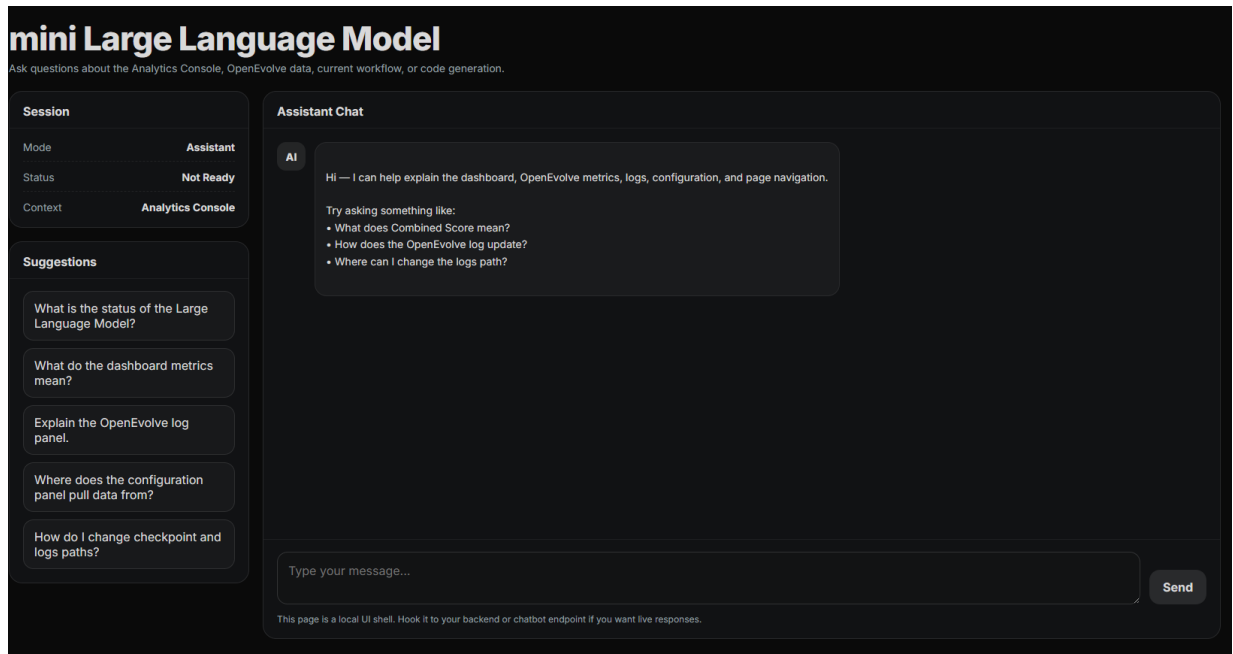
Although the team did not have the time or computational resources needed to fully train and deploy this model, the work has been preserved in the project repository on GitHub so that future teams can continue development. The long-term goal for this model was to train it using eight H100 GPUs for approximately three to four hours, at an estimated cost of \$50 to \$100. At that scale, the model could potentially reach GPT-2-level performance and later be fine-tuned further to better support UCT-related tasks and OpenEvolve codegeneration.

Previous Chatbot Page:



The previous chatbot interface was highly unoptimized, as loading the webpage for prompting was often slow and would frequently cause the Analytics Console to freeze during page transitions. In addition, the chatbot was limited to pulling only from OpenEvolve’s example problems, which made it function more like a dictionary or lookup tool rather than a true reasoning and code generation platform.

Current mini-Large Language Model Page:



This page currently serves as the placeholder interface for the mini Large Language Model (mLLM). It was designed and optimized to avoid unnecessary feature overhead that could freeze or slow down the Analytics Console. In addition, the page was built with a simple and flexible architecture so that a model trained using nanochat’s training harness can be integrated into it more easily in the future.

Lessons learned and Insights gained: (Kyle Galang)

After learning how to train a model through nanochat, I realized how limiting token constraints can be when trying to use AI for specialized single-task applications. If I had more time to continue training nanochat and refine its pretraining process, particularly in terms of the data being fed into the model, I believe we could have developed a strong LLM capable of demonstrating automatic code generation for OpenEvolve initial programs and evaluator files. This experience also made me more appreciative of the engineering community, especially the experts who openly share their work so others can learn and build from it. In particular, it gave me a greater understanding of localized models, including how effective their performance can be relative to their cost. That insight has inspired me to

continue pursuing this kind of work in the future, with the goal of developing my own localized LLM that I can fully own, run, and use independently without relying on internet access or larger models from industry.

With regard to future work, the model has already undergone limited training through the tokenization of simple training scripts from `base_train` and `mid_train` using a 3060 Ti GPU. If the goal is to train the model to achieve GPT-2.0-level performance or better, a strong next step would be to rent eight H100 GPUs through Lambda, which would cost approximately \$50 to \$100. From there, the model could be further fine-tuned on basic UCT processor code that has been translated into OpenEvolve initial programs and evaluator files. Once refined, the model could then be integrated back into the Analytics Console's mini-Large Language Model page, allowing researchers to use real-time prompting to help automatically generate code for OpenEvolve inputs.

Chapter 9. References for Diagrams and Past Work

9.1 Introduction

This final section of the document serves to reference a lot of science, mathematics, diagrams, and methodologies discussed in the document from previous researchers and project contributors. This is to reference the diagrams used from previous work and other works correlated to the project.

9.2 UCTP & SDA TAP Lab

Website

Code

Louis Caves¹, Miles Puchner², Binyamin Stivi², Gabriel Lundin⁴, Cameron Smith⁵

¹ University at Buffalo, ² University of California, San Diego, ³ Texas A&M University, ⁴ Ohio State University

9.3 MITRE

Dr. Jon C. Cline

Lead Information Systems Engineer, MITRE, Center for Connected Government, VA

9.4 Virginia Tech Applied Research Corporation

Melissa Hills

Educational Coordinator, Instructional Designer, Data Scientist, Colorado Springs, CO

9.5 OpenEvolve

Contributors

Code

@software{openevolve,

```

title = {OpenEvolve: an open-source evolutionary coding agent},

author = {Asankhaya Sharma},

year = {2025},

publisher = {GitHub},

url = {https://github.com/algorithmicsuperintelligence/openevolve}

}

```

9.6 LLMSR-Bench

[Paper](#) [Code](#)

Parshin Shojaee^{*1}, Ngoc-Hieu Nguyen^{*2}, Kazem Meidani^{3 4}, Amir Barati Farimani³, Khoa D Doan²,
Chandan K Reddy¹

¹Virginia Tech, ²VinUniversity, ³Carnegie Mellon University, ⁴Capital One

9.7 AlgoTune

[Paper](#) [Code](#)

[Ori Press](#) [Brandon Amos](#) [Haoyu Zhao](#) [Yikai Wu](#) [Samuel K. Ainsworth](#) [Dominik Krupke](#) [Patrick Kidger](#) [Touqir](#)
[Sajed](#) [Bartolomeo Stellato](#) [Jisun Park](#) [Nathanael Bosch](#) [Eli Meril](#) [Albert Steppi](#) [Arman](#)
[Zharmagambetov](#) [Fangzhao Zhang](#) [David Pérez-Piñeiro](#) [Alberto Mercurio](#) [Ni Zhan](#) [Talor Abramovich](#) [Kilian](#)
[Lieret](#) [Hanlin Zhang](#) [Shirley Huang](#) [Matthias Bethge](#) [Ofir Press](#)

UNIVERSITÄT TUBINGEN, PRINCETON UNIVERSITY